

项目学习手册 · 从零读懂 nsF5 隐写

nsF5 Steganography Project

2026 年 9 月 9 日

目录

导读 · 这本手册怎么用	6
0.1 读者对象与起点	6
0.2 本手册的阅读约定	6
0.3 12 周快速入门路线总览 (6–12 个月深入版见附录 F)	7
0.4 学完以后, 你应该能做到	8
0.5 项目地图: 先认识你要学的代码	8
0.6 学习小贴士	9
0.7 v1.4.0 更新说明 (2026-09-06)	9
第 1 章 · 数字图像与二进制 (第 1 周)	11
1.1 一张数字图像, 其实就是一堆数字	11
1.2 二进制、字节与位	11
1.3 为什么图像里 “有空地” 可以藏东西	12
1.4 位平面: 把一张图拆成 8 层	12
1.5 回到项目: 先认识 cover 与 stego	14
1.6 小结与自我检查	14
第 2 章 · 准备工具: Python、NumPy 与图像读写 (第 2 周)	15
2.1 安装环境	15
2.2 用 NumPy 思考图像	15
2.3 图像读写: 认识 image_io.py	17
2.4 第一次端到端运行: run_e2e.py	18
2.5 常见报错速查	18
第 3 章 · LSB 隐写与它的统计 “指纹” (第 3–4 周)	19
3.1 加密 vs 隐写: 先回答第一个问题	19
3.2 最小可运行的 LSB 隐写	19
3.2.1 先亲眼看一眼: LSB 改动到底有多 “隐蔽”	20
3.2.2 像素级: 到底改了多少、改成什么	21
3.3 为什么会露馅: 卡方检验	21

3.4 RS 分析: LSB 的“结构塌缩”	22
3.5 项目的综合判读: analyze()	23
3.6 动手实验: 藏一句英文, 再抓它出来	24
3.7 小结与自我检查	24
第 4 章 · 矩阵编码与 F5: 少改一点, 藏得更多 (第 5 周)	26
4.1 从“每像素 1 位”到“平均每次改动藏 p 位”	26
4.2 汉明码速成: 校验矩阵与伴随式	26
4.3 手算一个 $p=3$ 的例子	27
4.4 为什么高效很关键: 容量、改动率与可检测性	28
4.5 F5: 矩阵编码 + JPEG 系数减幅	30
4.6 回到项目: MatrixEmbedding	30
4.7 小结与自我检查	30
第 5 章 · nsF5 与湿纸编码: 项目的核心算法 (第 6 周)	32
5.1 先把问题拧清楚: F5 的收缩	32
5.2 湿纸编码: 先划湿点, 再解方程	32
5.3 读懂项目: nsF5Pixel._embed()	33
5.4 再看三个求解函数	34
5.4.1 一个能算的 GF(2) 例子 ($p=3$, 干列足够)	34
5.4.2 什么时候才需要高斯消元? —— 干列张成整个 GF(2)	35
5.4.3 自由变量置 0 = 尽量少改	35
5.5 验证实验: 效率与往返	36
5.6 小结与自我检查	36
第 6 章 · 口令、SHA-256 与“键控”安全设计 (第 6–7 周)	37
6.1 如果嵌入位置是固定的, 会怎样?	37
6.2 两条种子规则: 先认图, 再找正文	38
6.3 确定性置乱: splitmix64 + Fisher–Yates	38
6.4 篡改感知: 到底能感知什么	40
6.5 动手实验: 口令错误与像素篡改	40
6.6 小结与自我检查	41
第 7 章 · 机器学习基础 (写给第一次学的人) (第 7–12 周)	42
7.0 本章路线图: 建议怎么学 (约 3–6 周)	42
7.1 隐写检测, 其实就是“猜一张图藏没藏东西”	42
7.2 一张图怎么变成“数字”: 特征与标签	43
7.3 分类 = 画一条线, 把两类分开	44
7.4 从“分数”到“概率”: 为啥要套一层 sigmoid	45

7.4.1 为什么不直接用 0/1, 而要用 “概率”?	47
7.5 模型怎么 “学会”: 损失与训练	47
7.5.1 从 “一步到位” 到 “一步步走”, 为什么非得这么绕?	48
7.5.2 一个 “防过头” 的旋钮: C 与正则化	49
7.6 怎么不让模型 “作弊”: 交叉验证与分组	49
7.7 评估: 别只盯 “准确率”	50
7.7.1 ROC 曲线与 AUC: 只看 “能不能把含密排前面”	52
7.7.2 阈值怎么选: ROC 上的 “操作点”	52
7.7.3 为什么 “准确率” 在这里会骗人	54
7.8 进阶: 项目还做了两个 “加分项” (看得懂就行)	54
7.9 小结与自我检查	56
第 8 章 · 用机器学习做隐写检测 (第 8–14 周)	57
8.0 本章路线图 (约 4–8 周)	57
8.1 一个反直觉的问题: 为啥不直接让 AI “看图”?	57
8.2 认识 11 个特征: 它们在 “量” 什么	58
8.2.1 RS 分析: 核心信号 G_n 怎么来的	58
8.2.2 卡方检验: 灰度对是不是 “被抹匀了”	59
8.2.3 熵: 图像 “天然有多乱”	60
8.3 数据怎么来: make_dataset.py 的 “1+6” 设计	60
8.4 训练脚本怎么读: train_model.py	61
8.4.1 四种分类器, 各是什么路数 (只讲思想 + 真实对比)	61
8.4.2 哪个特征更重要: 看看 LR 学到的系数	64
8.4.3 一个很关键的细节: OOF 与 “别用自己考自己”	65
8.5 怎么读结果: ROC、AUC 与按档检出率	65
8.6 灵敏度: 在 GUI 里怎么切换 “松紧”	66
8.7 动手实验: 从训练到单图判定	66
8.8 v1.4.0 更新: SRM、143 维特征与双版本模型 (2026-09)	67
8.8.1 SRM 高通滤波: 同源有收益, 跨源反而亏	67
8.8.2 v2 143 维特征与 12 档变体	68
8.8.3 真实 JPEG 干净图暴露的部署问题与修复	68
8.8.4 双版本部署: 143d 稳健 vs 53d 可解释	68
8.9 小结与自我检查	69
第 9 章 · 工程化: C++、GPU、GUI 与测试 (第 10 周)	71
9.1 系统架构: 分层而不耦合	71
9.2 C++ 加速: 为什么是 “热路径”, 以及如何保证没错	72
9.3 GPU 版: 把 11 维特征 “批量向量化”	73
9.4 GUI: 把研究工具变成可教学的应用	74

目录	5
9.5 测试与 CI：让重构不翻车	74
9.6 代码阅读路线图（按需选用）	75
9.7 v1.4.0 更新：SRM、多源数据与模型工程（2026-09）	75
第 10 章 · 综合实践：从“读懂”到“做出来”（第 11–12 周）	77
10.0 实验方法论：先学会“怎么算才算数”	77
10.1 方向 A：复现并批判性验证（最稳）	78
10.2 方向 B：功能扩展（最有成就感）	78
10.3 方向 C：教学演示再包装	79
10.4 论文与代码对照阅读表	79
10.5 汇报/答辩常见问题与应答要点	79
10.6 交付清单（最后一周）	80
第 11 章 · 边界、伦理与继续前进（穿插学习）	81
11.1 法律与伦理边界	81
11.2 项目已知局限（看懂边界才算真正读懂）	81
11.3 现代信息隐藏与隐写分析地图	82
11.4 结语	82
附录 A · 术语中英对照	83
附录 B · 常用命令速查	86
附录 C · 12 周打卡检查表	88
附录 D · 概念 → 代码定位表	89
附录 E · 推荐资源	90
书籍（中文先入门）	90
核心论文（按阅读顺序）	90
在线资料与工具	90
附录 F · 6–12 个月深入自学路线图	92
F.1 四段式：每一段的目标与产出	92
F.2 四条主线：贯穿“广、全、实”	93
F.3 可选的“往外走”课题（按主线挑 1–2 个）	94
F.4 与导读 0.3（12 周路线）的关系	94

导读 · 这本手册怎么用

这是一份“跟着代码学”的手册。它不代替教科书，也不代替论文，而是把 F:\Steganography 项目背后的信息隐藏与机器学习知识，按零基础本科生可以接受的顺序拆开：先用直觉和例子讲清楚“为什么”，再带你看看项目里“怎么实现”，最后让你亲手跑实验、改代码。

0.1 读者对象与起点

你只需要具备这些前提，其余内容手册会逐步补齐：

- 会用鼠标操作电脑、能安装软件（Windows 环境）；
- 上过大学数学公共课或愿意遇到公式时“先跳过、再回头”；
- 有一点编程基础更好，但没有也没关系——第 2 章会带你把 Python 用到够用；
- 至少有 12 周（约 3 个月）的耐心，每周 6–8 小时；想深入 6–12 个月请看附录 F。

新手提示 |

如果你连 Python 都没装过，不要慌：前两周就是专门为这种情况设计的。你不需要成为程序员，只需要成为“能读懂并修改这个项目的人”。

0.2 本手册的阅读约定

手册里反复出现五种小方块，含义如下：

标记	含义
新手提示	把容易卡住的概念换成大白话或类比，让你顺利往下走
避坑提醒	初学者最容易踩的坑：数值类型、参数不一致、误读统计量等
动手做	必须亲自动手的小实验，只读不练等于没学
想一想	不需要标准答案的思考题，用来检验自己是否真的理解

回到项目看代码	指出应当打开哪个文件、读哪一段，建立“概念 → 代码”的映射
---------	--------------------------------

0.3 12 周快速入门路线总览（6–12 个月深入版见附录 F）

路线遵循“载体基础 → 隐写算法 → 隐写分析 → 机器学习 → 工程加速 → 综合实践”的自然顺序。每一周都对应可运行的代码或可复现的实验：

周次	主题	对应章节	动手成果
第 1 周	数字图像与二进制	第 1 章	能亲手查看一张图的像素与位平面
第 2 周	Python/NumPy/Pillow	第 2 章	跑通 GUI 或 run_e2e.py，能读写图像数组
第 3–4 周	LSB 隐写与盲分析	第 3 章	藏一句话 → 用卡方/RS 检测到它
第 5 周	矩阵编码与 F5	第 4 章	用汉明码做到“改动更少、藏得更多”
第 6 周	nsF5、湿纸与哈希键控	第 5–6 章	读懂 ns5_core.py，解释湿纸求解
第 7 周	机器学习基础	第 7 章	能讲清特征/标签/过拟合/AUC
第 8–9 周	机器学习隐写检测	第 8 章	跑通 v1 11 维与 v2 143 维两套管线；理解 SRM 与 143d/53d 双版本模型
第 10 周	C++/GPU/数据工程/GUI	第 9 章	看懂加速原理、自校验机制与多源数据集管线 (SRM/BOSSbase)
第 11–12 周	综合项目与汇报	第 10–11 章	完成一个可演示的改进实验

动手做 |

请把这张表抄成一张打卡表贴在桌前，或直接使用附录 C 的检查表。每周结束时勾掉一行，并回答该章末尾的“想一想”。

想深入、想做出成果？

上面是 12 周的“入门”路线；如果你有 6–12 个月，请看附录 F 的深入路线图——同样是这些内容，但按“基础 / 提高 / 深化 / 实战”四段放慢加深，并给出一条可选的“算法 / 检测 / 工程 / 研究”主线。

0.4 学完以后，你应该能做到

- 用自己的话解释：隐写、隐写分析、嵌入效率、收缩、湿纸编码、伴随式；
- 说出 LSB 隐写为什么会被卡方检验与 RS 分析发现；
- 手算一个 $p=3$ 的汉明矩阵嵌入例子（块内最多改 1 位）；
- 解释 nsF5 与 F5 的本质区别，以及为什么“无收缩”更重要；
- 解释图像哈希键控如何实现解码自同步，并说明它的局限；
- 解释机器学习二分类的完整流程，包括为什么要按照片分组交叉验证；
- 读懂 v1 的 11 维与 v2 的 143 维特征（含 SRM 统计），能解释 143d 稳健版与 53d 可解释版的双模型策略；
- 讲清 C++ DLL 与 GPU 加速分别加速了什么，为什么需要一致性校验；
- 独立完成一个小的改进实验并写出诚实的结果分析。

0.5 项目地图：先认识你要学的代码

整个项目是一个“研究工具”：既能做算法实验，也打包成带 GUI 的软件。先记住这些关键文件，后面每一章都会回来找它们：

文件	角色	学习顺序
src/image_io.py	图像读写封装：打开/保存 8bit 灰度与彩色图	第 2 章
src/ns5_core.py	算法核心：汉明码、湿纸、哈希键控、嵌入/解码	第 3-6 章
src/steganalysis.py	盲隐写分析：卡方、RS、综合概率与判定	第 3 章
src/efficiency.py	码族与嵌入效率理论/实测曲线	第 4 章
src/matrix_demo.py	伴随式查找的教学演示逻辑	第 4 章
src/fsfeatures.py + cpp/fsfeatures.dll	v1 11 维统计特征的 ctypes 绑定 (v2 143 维见 featurize_v2.py)	第 8 章
src/make_dataset.py / train_model.py	数据集生成 (v1/v2 特征、SRM、多源) 与训练/评估	第 8 章
src/ml_predict.py	单图 ML 判定封装 (143d/53d 双版本 + 灵敏度)	第 8 章

src/cppembed.py + cpp/nsf5embed.dll	C++ 嵌入/置乱热路径与自校验	第 9 章
gpu/*.py	PyTorch 批量特征 (v1 11 维 / v2 143 维) 与 GPU 训练	第 9 章
src/gui.py	tkinter 图形界面, 含教学面板	第 9 章
src/test_core.py 等测试	验证嵌入往返、分析与误报的回归测试	全程
src/featurize_v2.py + src/srm_filter.py + gpu/featurize_v2_gpu.py	v1.4 新增: 30 核 SRM 预处理与 143 维 v2 特征 (CPU/GPU 双实现)	第 8-9 章

回到项目看代码 |

建议从现在起保持项目窗口打开。手册里提到某个文件时, 就切过去找到它; 第 2 章之后, 多数知识点都要求你 “先运行、再阅读”。

0.6 学习小贴士

- **先跑通, 再读懂。**很多概念在纸上绕, 跑一次端到端脚本就通了;
- **公式分两步消化。**第一遍只看结论和直觉, 第二遍再回到推导;
- **用自己的话复述。**每章小结能讲给同学听, 才算掌握;
- **记录实验日志。**改了哪些参数、结果如何, 是最后综合项目最宝贵的材料;
- **善用测试。**项目自带的 test_core.py / test_steg.py 是最快的 “对不对” 裁判。

想一想 |

在你开始之前, 先花 5 分钟回答: 加密和隐写有什么不同? 如果你说不清, 太好了——这正是第 3 章要解决的第一个问题。

0.7 v1.4.0 更新说明 (2026-09-06)

本手册 9 月 3 日初稿对应项目 v1.3; 9 月 6 日项目更新到 v1.4.0, 本版手册已同步。隐写算法部分 (第 1-6 章) 不受影响; 机器学习与工程部分补充了 8.8 与 9.7 两节, 并把旧指标标注为 “v1 基线”。

- ML 检测从 11 维特征 + LR/XGB (AUC 约 0.75-0.79) 升级为 143 维 LGB 默认版 (8 折平均 0.9085) 与 53d 可解释版 (0.9227), 详见 8.8;

- 新增 SRM 高通滤波预处理、143 维 v2 特征与 12 档变体, 并补入真实 JPEG 干净样本与 BOSSbase 全量多源数据, 详见 8.8 与 9.7;
- 项目迁移到新 GitHub 仓库 (Yukinoshita-lin/nsf5-steganography), 许可证改为 Apache-2.0 并新增 NOTICE;
- 术语表、命令速查、12 周打卡表与概念 → 代码定位表已同步补充 v1.4 内容。

第 1 章 · 数字图像与二进制 (第 1 周)

动手做 |

本章目标：把“图像 = 数字矩阵”的直觉焊死；会手算二进制与字节；能说出为什么人眼和统计都能容忍一点“像素级扰动”。我们用项目真实载体 (img/cover.png) 亲眼拆开一张图。

1.1 一张数字图像，其实就是一堆数字

把一张 8bit **灰度图** 放大到像素级，你会看到它是一张由 0-255 整数组成的二维表格。0 代表全黑，255 代表全白，中间是不同深浅的灰。计算机只保存这张表格的“尺寸”和“数字”，不保存你肉眼看到的画面。

彩色图则是三张这样的表格叠在一起：R（红）、G（绿）、B（蓝）三个通道。常见的 RGB 图每个像素有 3 个 0-255 的值。比如一个橙色像素可以写成 $R=255$ 、 $G=128$ 、 $B=0$ 。

在代码里，灰度图就是一个二维数组，彩色图就是一个三维数组。项目的 `src/image_io.py` 把这件事封装成了两个函数：`load_as_gray()` 负责把任意支持的图片读成灰度数组，`save_image()` 负责把数组保存成 PNG。

新手提示 |

数学里我们用“行、列”描述矩阵，图像处理里则常说“高、宽”：一张 512×512 图就是 512 行、512 列。数组形状 (512, 512) 的写法正是“行数在前”。

1.2 二进制、字节与位

计算机底层只认识 0 和 1。一个 0 或 1 叫 1 bit (**位**)，8 个 bit 组成 1 byte (**字节**)。8 个 bit 一共能表示 $2^8=256$ 种组合，正好对应灰度值 0-255。

例如十进制 $200 = 128 + 64 + 8$ ，写成 8 位二进制是 11001000；它的**最低有效位 (LSB)** 就是最右边的那个 0。把 200 的 LSB 改成 1，得到 201——在人眼里，灰度 200 和 201 几乎无法区分。

“最低有效位”是整本手册最重要的概念之一：LSB 隐写，就是偷偷修改像素的 LSB，把秘密比特藏进去。

动手做 |

打开一个 Python 交互环境 (或写个小脚本), 验证下面这段:

体验: 十进制、二进制与 *LSB*

```
v = 200
print(bin(v))           # 0b11001000
print(v & 1)             # 最低位 = 0
print(v ^ 1)            # 翻转最低位 = 201
print(v & 0b11111110)   # 把最低位清零 = 200
print(v & 0b11111110 | 1) # 先清零再置1 = 201
```

1.3 为什么图像里 “有空地” 可以藏东西

图像里藏着大量**冗余**, 这要从两个层面理解:

- **视觉冗余**: 人眼对微小亮度差、高频细节不敏感。把某个像素从 128 改成 129, 几乎没人看得出来;
- **统计冗余**: 自然图像相邻像素高度相关, 大部分信息集中在低频。JPEG 等压缩格式就是靠去掉这些冗余才把文件变小;
- **LSB 平面近似随机**: 自然照片的最低几位看起来 “像噪声”, 这恰好提供了天然的 “掩护环境”。

于是出现了两类使用冗余的方式: **压缩**想把冗余去掉让文件变小, **隐写**想在冗余里塞秘密而尽量不破坏视觉与统计特征。二者争夺的是同一片 “可修改空间”, 这也是为什么很多隐写算法要针对特定压缩格式设计。

避坑提醒 |

不要以为 “看不见 = 安全”。肉眼看不见只是最弱的要求; 真正专业的检测者用统计工具, LSB 直接替换会留下非常明显的统计痕迹, 这正是第 3 章卡方检验与 RS 分析能抓住它的原因。

1.4 位平面: 把一张图拆成 8 层

把灰度值写成 8 位二进制后, 可以按 “第几位” 把图像拆成 8 张二值图, 称为**位平面**。最高位 (MSB) 平面决定画面大轮廓; 最低位 (LSB) 平面几乎全是细碎的噪声点。

先用项目真实载体 (*img/cover.png*) 亲手拆开:

图 1-1 (真实数据: 最左是原图 (0-255); 右边 8 格依次是 *bit 7* (最高位, $\times 128$) 到 *bit 0* (最低位/*LSB*, $\times 1$) 的位平面。*bit 7* 决定大轮廓; *bit 0* 几乎全是细碎噪声)

图 1-1 一张 8bit 灰度图折成 8 个位平面 (bit7=最高位, bit0=最低位/LSB)

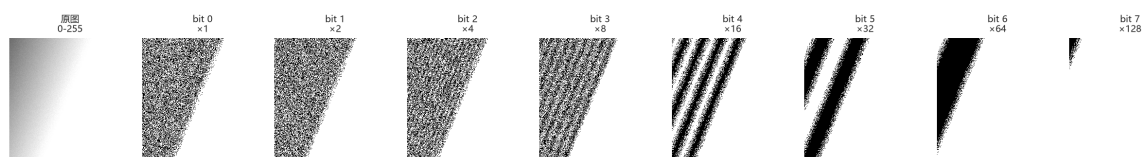


图 1: 图 1-1 位平面分解

读图要点:

- 越高的位平面越“有结构” (bit 7、6、5 很像原图的轮廓);
- 越低的位平面越“像噪声” (bit 0、1、2 布满随机斑点);
- 正因为 LSB 平面本就接近随机, 往里面再塞伪随机秘密比特, 肉眼和简单直方图都很难察觉——但统计手段能测出“它被随机化过” (第 3 章)。

再把“叠加还原”也亲手做一遍, 理解“每一层必须乘它的权重再相加”:

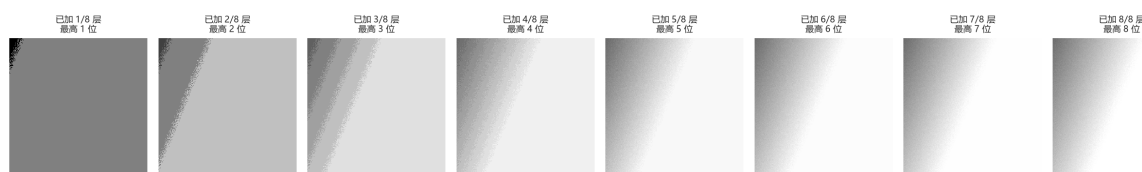
图 1-2 从最高位逐层累加: 加得越多越清晰, 8 层全加 = 精确还原原图
原图 = $\sum (\text{bit}_k) \times 2^k$, 每一层必须乘其权重再相加

图 2: 图 1-2 逐层累加还原

图 1-2 (真实数据: 从最高位开始逐层累加——先只有 bit7, 画面是灰暗的块状轮廓; 每加一层越清晰; 加满 8 层就精确等于原图)

这就是“叠加”最关键的一句话:

$$\text{原图} = \sum_{k=0}^7 \text{bit plane}_k \times 2^k$$

位平面本身只是 0/1, **必须乘以它所在的权重 2^k 之后才能相加**。如果只是把 8 层“或”起来或无视权重直接相加, 得到的根本不是原图。这也是为什么项目里“改 LSB”用的是 `& 0xFE | bit` (把最低位清零再置位) 这样的位运算, 而不是做整数的加减。

新手提示 |

图 1-1 里你看到“高位置轮廓、低位像噪声”。这个直觉非常重要: **修改高位 = 肉眼可见; 修改低位 = 肉眼不可见但统计可测**。隐写只碰低位, 恰恰是因为“低位本身就像噪声, 改了也不显眼”——这既是隐写的机会, 也是检测的突破口。

动手做 |

打开互动实验室 (webapp/index.html) 的第 1 个区块 (位平面实验室): 用 “抽看单层” 把 bit 0、bit 4、bit 7 分别拖出来对比; 再切到 “叠加还原”, 关掉 bit 0、1、2 看画面怎样变 “平”, 把 8 层全开看它怎样精确还原原图。

数一数像素与 LSB 的取值 (项目里到处是这样的 *numpy* 操作)

```
import numpy as np
img = np.asarray([[200, 201], [128, 129]], dtype=np.uint8)
print(img.shape, img.dtype)    # (2, 2) uint8
print(img & 1)                  # LSB 位平面 [[0,1],[0,1]]
print(np.unpackbits(np.array([200], dtype=np.uint8)))
# [1 1 0 0 1 0 0 0] ← 最高位在前
```

1.5 回到项目: 先认识 cover 与 stego

隐写领域里有两个固定称呼: 没有秘密的原始载体叫 **cover (载体)**, 藏入秘密后的图像叫 **stego (含密图)**。项目里这两个词随处可见: `img/cover.png` 是演示用载体, `output/stego_*.png` 是嵌入后的含密图。

回到项目看代码 |

打开 `src/run_e2e.py` 第 1 步: 它用 *numpy* 生成一张 256×256 的平滑 “封面图” 并保存到 `img/cover.png`。仔细看生成方式——`np.linspace` 制造渐变背景, 再加一点小噪声。理解这段代码, 你就理解了一张自然感图像是如何从数字 “长” 出来的。(我们图 1-1/1-2 用的正是这张图。)

1.6 小结与自我检查

- 灰度图 = 0-255 整数矩阵; 彩色图 = 三个通道矩阵;
- LSB 是像素值二进制的最低位, 翻转它对视觉影响极小 (图 1-1);
- 压缩消除冗余, 隐写利用冗余, 二者针锋相对;
- `cover` = 原始载体, `stego` = 含密图;
- 高位置轮廓、低位像噪声——这是 “隐写藏低位、检测测低位” 的根源 (图 1-2)。

想一想 |

一张纯黑色 (全 0) 的图适不适合做 LSB 隐写的载体? 一张纯随机噪声图呢? 把想法写下来, 学完第 3 章再回来对照。(提示: LSB 平面是不是 “接近随机” 决定了它好不好藏、好不好被抓。)

第 2 章 · 准备工具：Python、NumPy 与图像读写 (第 2 周)

动手做 |

本章目标：装好环境、跑通端到端脚本，会用 NumPy 读写图像数组。从此以后，你做的每个实验都能立刻在项目里验证。我们先用一张真实照片，把「图像 = 数组」焊进脑子。

2.1 安装环境

项目依赖很轻：核心只需要 NumPy 与 Pillow，机器学习部分再用 scikit-learn、joblib 等。建议用较新的 Python 3.9+ (README 示例使用 3.14，低版本同样可用)。

在项目根目录安装并做第一次自检

```
cd F:\Steganography
pip install numpy pillow
python src\test_core.py      # 核心算法自测
python src\test_steg.py     # 隐写分析自测
python src\run_e2e.py        # 端到端：嵌→入解→码分→析绘图
```

避坑提醒 |

如果系统默认 python 没有 tkinter，GUI 无法启动。README 给出的办法是使用带 tkinter 的解释器（例如 C:\Python314\python.exe）；只跑命令行脚本则不受影响。遇到 ModuleNotFoundError 时先确认你安装到了“运行它的那个解释器”里：python -m pip install ... 更稳妥。

2.2 用 NumPy 思考图像

NumPy 数组的核心概念只有四个：**形状 (shape)**、**类型 (dtype)**、**切片 (slice)** 与**向量化运算**。隐写算法本质上都在做：取出数组 → 按某种顺序改写部分元素的 LSB → 存回数组。

先用一张真实照片看“图 = 数组”：

图 2-1 一张图 = 一个数组: 彩色是 (H,W,3), 灰度是 (H,W), 类型都是 uint8
(真实相机照片, 原尺寸 4096×3072×3)

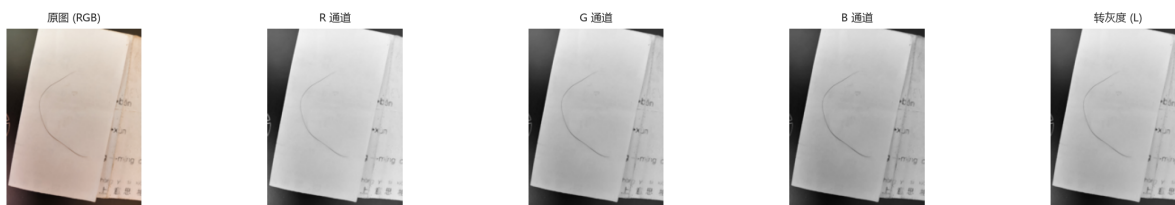


图 3: 图 2-1 图像 = 数组

图 2-1 (真实相机照片: 彩色是 $(H, W, 3)$ 的三通道数组, 转灰度是 (H, W) 的单通道, 类型都是 `uint8`。R/G/B 三个通道分别是一张“这个颜色有多亮”的灰度图; 灰度则是按人类亮度感知加权的结果)

读图要点:

- 彩色图: 三个 0-255 的数字叠在每个像素上, 形状 $(H, W, 3)$;
- 灰度图: 每个像素只一个 0-255, 形状 (H, W) ;
- 类型都是 `uint8` (0-255 的 8 位无符号整数) ——这也是为什么“改 LSB”是位运算而不是 ± 1 。

四个核心概念的 10 行速览

```
import numpy as np
a = np.arange(12).reshape(3, 4)      # 3 行 4 列
print(a.shape, a.dtype)              # (3,4) int64
b = a[:, 1]                          # 取第 2 列
c = a[0:2, 0:3]                      # 左上角 2×3 块
x = np.zeros((64, 64), dtype=np.uint8)
x[10:20, 5:15] = 255                # 画一个白色方块
print((x & 1).sum())                 # 统计 LSB=1 的数量
```

注意 `dtype=np.uint8`: 图像像素是 0-255 的 8 位无符号整数。做加减时要小心**溢出回绕**: `np.uint8(200) + 100` 会得到 44 而不是 300。项目在翻转 LSB 时用“异或 1”而不是“ ± 1 ”, 正是为了避免把 0 减成负数或把 255 加溢出。

图 2-2 (真实计算: 一个 `uint8` 数组; `uint8` 溢出回绕—— $200+100=44$ 而非 300, 所以改位用 1; 转置/切片得到的是“视图”, 内存不连续, 保存给底层库前常要 `np.ascontiguousarray` 转一下)

动手做 |

在交互环境里敲这段, 亲眼看到 `np.uint8(200)+np.uint8(100)==44`, 以及 `a.T` 与 `np.ascontiguousarray(a.T)` 的 `flags` 差异。第 2 周的“会读代码”就从读懂这些小坑开始。

图 2-2 dtype 与内存布局: 两条最容易被绕进去的坑

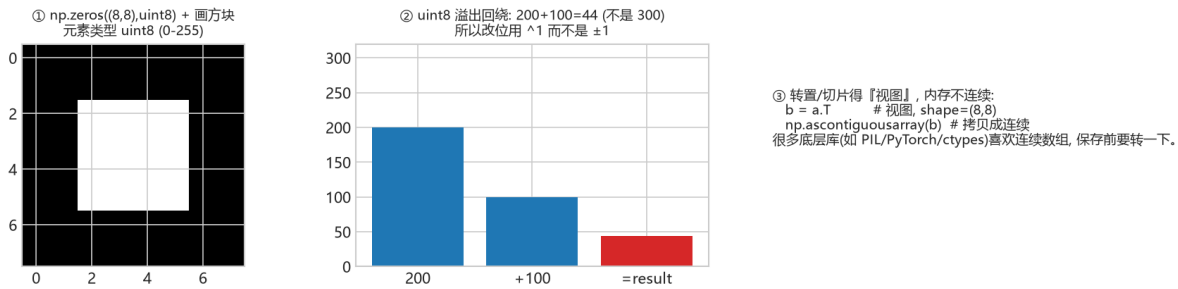


图 4: 图 2-2 dtype 与内存布局

2.3 图像读写: 认识 `image_io.py`

`project/src/image_io.py` 的核心接口

```
from PIL import Image
import numpy as np

def load_as_gray(path):
    im = Image.open(path).convert("L") # 强制转灰度
    return np.asarray(im, dtype=np.uint8)

def save_image(image, path):
    im = Image.fromarray(np.ascontiguousarray(image))
    im.save(path)
```

回到项目看代码 |

自己打开 `src/image_io.py` 全文, 注意 `load_image()` 与 `load_as_gray()` 的差异, 以及 `SUPPORTED` 支持的扩展名。然后运行下面这段, 读入项目自带的封面图:

动手: 读图、看形状、存灰度版

```
import sys; sys.path.insert(0, "src")
import image_io as IO
img = IO.load_as_gray("img/cover.png")
print(img.shape, img.dtype) # (256,256) uint8
print(img.min(), img.max(), img.mean())
IO.save_image(img, "output/my_copy.png")
```

注意 `load_as_gray` 里 `.convert("L")`：它把一切输入强制成灰度；而 `save_image` 里 `np.ascontiguousarray` 保证内存连续。这一“读成灰度、写成连续”的封装，是后面所有算法（嵌入/分析/ML）统一的输入接口——保证大家拿到的都是同一规格的数组。

2.4 第一次端到端运行：run_e2e.py

运行 `python src/run_e2e.py`，脚本会依次完成五件事：生成封面图 → 以 nsF5 p=3 嵌入一段英文消息（分别用空口令与口令） → 解码比对 → 对干净图与含密图做盲隐写分析 → 绘制码族/效率图。

动手做 |

仔细阅读终端输出，找到三组数字并抄下来：嵌入比特数、改动像素数、干净图与含密图的分析概率。下一章你就能解释它们为什么不同。

你还看不懂嵌入内部不要紧——第 2 周的目标只是“让代码在你机器上跑起来”，以及建立“数组改 LSB”的直觉。等你学完第 8 章回来看 `train_model.py`，会发现这里的每个“数组操作”都在为“特征向量”服务——这就是本手册“跟着代码学”的主线。

2.5 常见报错速查

报错 / 现象	原因	处理
ModuleNotFoundError	包装到了别的解释器	用运行脚本的解释器执行 <code>python -m pip install</code>
图像太小, 无法容纳头部 + 正文	像素数少于消息容量	换更大的图，或调小 p / 缩短消息
UnicodeDecodeError / 乱码	默认只支持 ASCII 消息	嵌入文本保持 ASCII；中文方案见第 10 章拓展思路
GUI 秒退 / TclError	解释器无 tkinter	换带 tkinter 的 Python 运行 <code>src/gui.py</code>

想一想 |

为什么 `image_io` 保存时要把数组转成“连续数组” (`np.ascontiguousarray`)？提示：切片与转置可能产生非连续内存视图，很多底层库不喜欢它们。再想：既然灰度是 (H,W) 单通道，为什么第 8 章的“特征”能到 143 维？（提示：那些是“统计量”，不是像素本身。）

第 3 章 · LSB 隐写与它的统计“指纹” (第 3—4 周)

动手做 |

本章目标：亲手完成一次“藏进去 → 检测出来”的最小闭环；理解卡方检验与 RS 分析各自抓住了什么；看懂 steganalysis.py 的判读逻辑。我们先用真实载体 (img/cover.png) 直观看到 LSB 改动有多“隐蔽”，再看统计如何戳穿它。

3.1 加密 vs 隐写：先回答第一个问题

维度	加密 (Encryption)	隐写 (Steganography)
保护对象	消息内容：没有密钥就看不懂	通信行为：别人看不出有秘密
可见性	密文明显“不像正常数据”	载体看起来完全正常
典型场景	传输密码、证书、加密文件	隐蔽信道、版权水印、取证研究
相互关系	可先加密再隐写 (强烈推荐)	隐写隐藏的是“秘密存在”这一事实

新手提示 |

一句口诀：**加密保护“内容”，隐写保护“存在”**。专业隐写系统通常两者都用：先把消息加密，再把密文藏进载体。加密后密文近似随机，反而更不容易在统计上露馅。

3.2 最小可运行的 LSB 隐写

最朴素的隐写叫 **LSB 替换 (LSB replacement)**：把秘密比特串逐位写进像素的 LSB。灰度图有 $M \times N$ 个像素，就能藏 $M \times N$ 个比特。

教科书版最小 *LSB* (仅用于理解；项目实际用的是第 4-5 章的矩阵编码)

```
import numpy as np

def text_to_bits(s):
    raw = s.encode("ascii")
    # 16 bit 长度头 + 每个字符 8 bit
    head = [(len(raw) >> i) & 1 for i in range(16)]
```

```

body = np.unpackbits(np.frombuffer(raw, np.uint8))
return np.concatenate([head, body]).astype(np.uint8)

def lsb_embed(cover, bits):
    flat = cover.ravel().copy()
    k = min(len(bits), flat.size)
    flat[:k] = (flat[:k] & 0xFE) | bits[:k]    # 清最低位再写入
    return flat.reshape(cover.shape)

def lsb_extract(stego, nbytes):
    flat = stego.ravel() & 1
    return bytes(np.packbits(flat[16:16 + nbytes*8]))

```

避坑提醒 |

有损格式是 LSB 的天敌。 PNG/BMP 无损保存, 位面不会变; JPEG 有损压缩会把 LSB 打得面目全非。所以项目默认保存 PNG, 而 JPEG 域隐写需要另做一套 (yccstego 扩展正是走量化 DCT 系数路线)。

为什么消息开头要先放 16 bit 长度头? 因为解码端必须知道“读多少字节”才能还原消息。项目 `src/ns5_core.py` 的 `encode_string()` 正是先写 16 位小端长度、再写 ASCII 正文比特——你在 3.2 的例子中已经把结构复制了一遍。

3.2.1 先亲眼看一眼: LSB 改动到底有多“隐蔽”

用项目的演示载体 `img/cover.png` 做一次真实的 LSB 替换 (藏随机比特), 看有 25% 像素的 LSB 被改写后, 图片变成什么样:

图 3-1 肉眼几乎一模一样, 但差异放大与 LSB 位平面会露出端倪

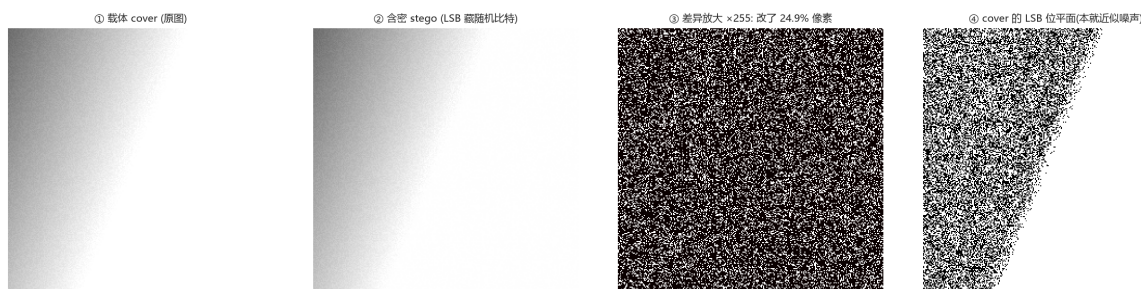


图 5: 图 3-1 载体 vs 含密 vs 放大差异

图 3-1 (真实数据: 载体 `img/cover.png`; 用朴素 LSB 藏入随机比特后的含密图; 把 “/ 载体-含密 /” 放大 255 倍, 才看清改动遍布全图; 载体本身的 LSB 位平面——本就近似噪声)

读图要点:

- **vs 肉眼几乎无差别**——这正是 LSB 隐写的“隐蔽”所在；
- **差异放大 $\times 255$ 后**，才能看到散落全图的噪点，这就是“被改动的像素位置”；
- **cover 的 LSB 位平面本就是细碎噪声**——所以往里面再藏随机比特，直方图/肉眼都难以察觉。

3.2.2 像素级：到底改了多少、改成什么

再放大到一小块真实像素值，看嵌入前后每个像素的变化：

图 3-2 像素级对比：只是个别像素的 LSB 变了 0 $\leftrightarrow$ 1，灰度几乎不变

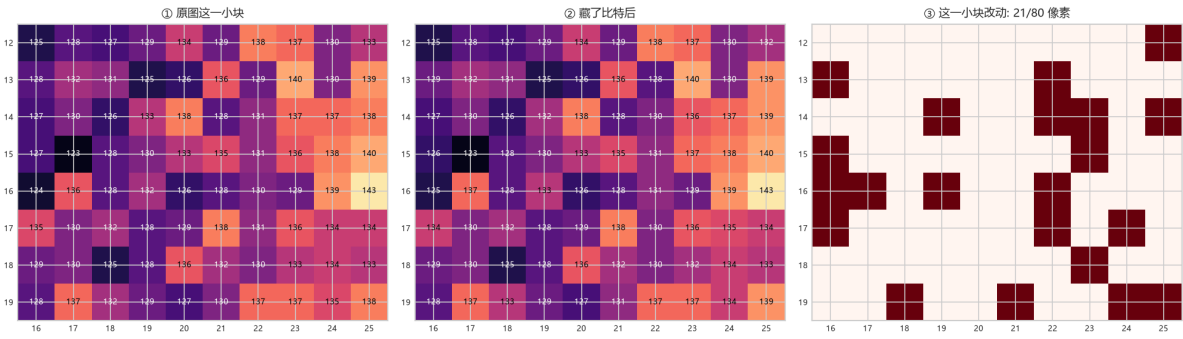


图 6: 图 3-2 像素级对比

图 3-2 (真实数据： / 分别是同一小块区域嵌入前/后的灰度值； 标出被改的 21/80 个像素。注意每处改动只差 1——因为只是 LSB 在 0 1 之间翻转)

读图要点：

- 每个被改的像素，值只 ± 1 (如 127 $\rightarrow$ 128、135 $\rightarrow$ 134)，因为变的是**最低位**；
- 人眼对 1 级灰度差完全无感——这就是“视觉冗余”；
- 但正是这种“把奇偶拉平”的改动，给统计检测 (3.3 卡方、3.4 RS) 留下了可测的指纹。

新手提示 |

记住两个词：**视觉冗余** (人眼看不出) 与**统计冗余** (检测器看得出)。LSB 隐写利用了前者，却恰恰破坏了后者——这成为所有统计检测的突破口。

3.3 为什么会露馅：卡方检验

自然图像里，相邻灰度 $2i$ 与 $2i+1$ (例如 100 与 101) 出现的次数通常**不相等**。LSB 替换会强制把偶数与奇数“配对拉平”：当某像素从 $2i$ 改成 $2i+1$ 或反过来，两个灰度的频数会趋向均衡。Westfield 的卡方检验做的就是这件事：

1. 统计整张图的 256 级灰度直方图，按相邻对 (0,1)、(2,3)、...、(254,255) 配对；
2. 假设“已嵌入”，则每对的两个频数应近似相等，期望值取对之和的一半；
3. 计算统计量 $\sum(\text{观测}-\text{期望})^2/\text{期望}$ (只统计频数和大于 0 的灰度对)；
4. 把统计量换算成 p 值：p 高表示“观测与均匀假设吻合” → 疑似 LSB 随机化；
5. 项目还会把图切成 20 个前缀段分别计算 p 值，取中位数，得到更稳的判据。

注意 p 值的含义容易反：这里 p 越大越可疑，因为它在说“如果 LSB 已被完全随机化，看到这种分布的几率很大”——而干净的平滑图通常不会这么均匀。

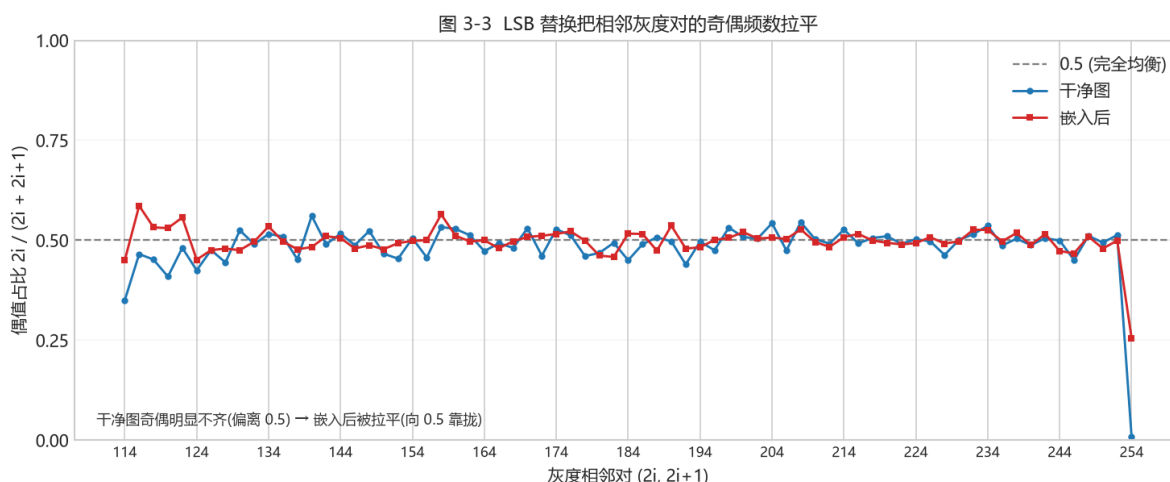


图 7: 图 3-3 相邻灰度对奇偶频数：干净 vs 嵌入后

上图对比了干净图与 LSB 嵌入后、每个相邻灰度对 (2i, 2i+1) 里偶值所占的比例：干净图 (蓝) 明显偏离 0.5 (奇偶频数不齐)；嵌入后 (粉) 被“拉平”，向 0.5 靠拢。这正是卡方检验要抓的信号——越均衡，p 值越高，越可疑。

避坑提醒 |

天然噪声图会骗过卡方检验。 数码照片本身 LSB 就接近随机，卡方 p 天然很高。所以项目引入了“内容本底随机度”修正：先用灰度差分熵估计图像本身有多‘随机’，再决定是否把高 p 当作嵌入证据。这是把启发式做成可用的关键工程细节。

3.4 RS 分析：LSB 的“结构塌缩”

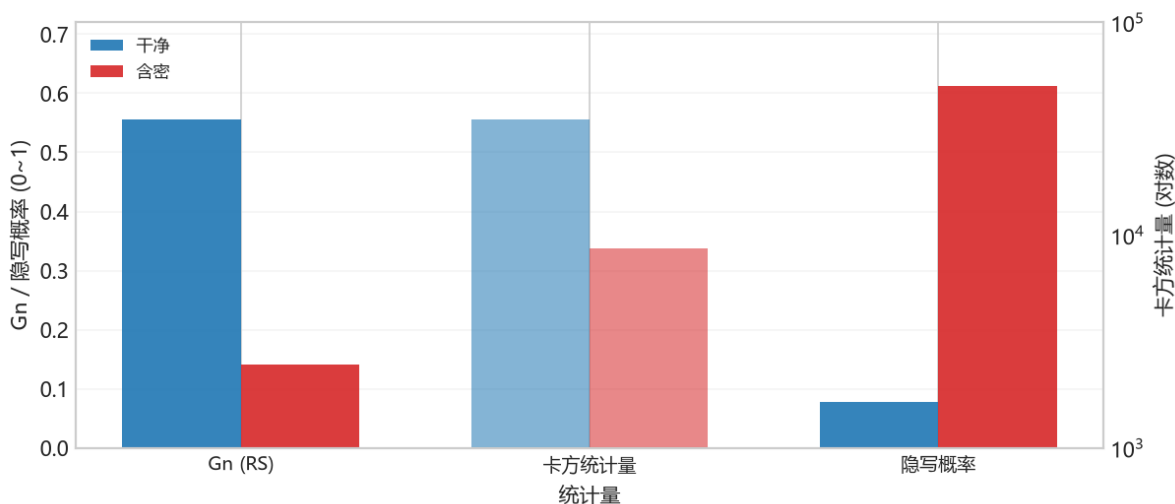
Fridrich 等人的 RS 分析换了一个角度：不比较灰度对频数，而是观察 LSB 位面的**空间结构**。做法是把像素流切成 4 像素一组，定义一个平滑度判别式 f (组内相邻差绝对值之和)，再分别用“正掩码”与“负掩码”翻转组内部分像素的 LSB，统计翻转后 f 变大 (规则 R) 还是变小 (奇异 S) 的组数。

干净的自然图像在负掩码下有明显的“常规倾向”，常用量 $G_n = (R_n - S_n)/N$ 显著为正（项目里干净平滑图常在 0.3~0.7）。LSB 替换让位面随机化后，这个结构“塌缩”， G_n 明显下降。项目同时输出 G_r 、估计嵌入率等，共同组成“RS 证据”。

项目 `steganalysis.py` 中 `RS` 的核心逻辑（伪代码，仅示意分组与统计思想）

```
groups = flat[:m].reshape(-1, 4)           # 每 4 像素一组
fg = np.abs(np.diff(groups, axis=1)).sum(axis=1) # 原平滑度 f
pos = (groups ^ np.array([0,1,1,0])) & 0xFF    # +M 翻转
neg = (groups ^ np.array([1,0,0,1])) & 0xFF    # -M 翻转
Rm = (f(pos) > fg).sum(); Sm = (f(pos) < fg).sum()
Rn = (f(neg) > fg).sum(); Sn = (f(neg) < fg).sum()
Gn = (Rn - Sn) / n                          # 干净图明显为正，随机化≈后0
```

图 3-4 三道证据对照：RS 缺口塌缩（基于真实数据）



含密图(粉)的 G_n 明显塌缩、卡方统计量下降、隐写概率升高——这就是三道判据的合力

图 8: 图 3-4 三道证据对照：RS 缺口 / 卡方统计量 / 隐写概率

上图用项目真实统计量对比了干净图（蓝）与含密图（粉）：RS 缺口 G_n 从明显为正（结构）塌缩、卡方统计量下降（奇偶被抹平）、ML 含密概率上升。LSB 嵌入把位面的“秩序”破坏掉了——这组柱状图就是 RS/卡方/ML 三道证据的直观对照。

3.5 项目的综合判读：analyze()

`src/steganalysis.py` 的 `analyze(image, sensitivity=...)` 并不只看一个统计量，而是组合四路信号：

- 卡方中位 p 值（判断灰度对是否被拉平）；

- RS 塌缩幅度 (G_n 相对内容自适应基线的下降比例);
- 灰度差分熵与 LSB 差分熵 (判断“随机”是不是图像本底自带);
- 前缀 p 值序列 (观察随机化是局部还是整体)。

三档灵敏度 (严格/均衡/宽松) 调整“可能/高度可能”的判定阈值与概率牵引系数, 本质是在**误报与漏报**之间选择操作点。

回到第 7 章看全貌 |

第 7 章会把这套启发式升级成“特征 + 监督分类器”, 并画出更完整的 ROC / AUC (图 7-6) 与特征分布 (图 8-1/8-2)。

3.6 动手实验：藏一句英文，再抓它出来

利用项目真实 API: 嵌入 → 保存 → 分析 (可在 *Jupyter* 中逐段执行)

```
import sys; sys.path.insert(0, "src")
import numpy as np
import image_io as IO
from ns5_core import embed_string, extract_string
import steganalysis as SA

clean = IO.load_as_gray("img/cover.png")
stego, report, nbits = embed_string(
    clean, "Hello_nsF5!", method="nsF5", p=3, password="")
IO.save_image(stego, "output/lab3_stego.png")
print("改动像素:", report["cover_changed"])
print("解码:", extract_string(stego, method="nsF5", p=3))
for name, im in (("干净", clean), ("含密", stego)):
    r = SA.analyze(im)
    print(name, "Gn=%.3f" % r["RS_Gn"],
          "chi2p=%.3f" % r["chi2_pvalue"],
          "prob=%.2f" % r["stego_probability"], r["verdict"])
```

动手做 |

把消息改成 5000 个字符再跑一次 (参考 `src/test_steg.py`), 观察改动像素占比、 G_n 下降幅度与隐写概率的变化。再换一张噪声较多的照片, 看看卡方与 RS 是否还能区分。

3.7 小结与自我检查

- LSB 嵌入利用**视觉冗余** (图 3-2 每处只差 1), 肉眼不可辨 (图 3-1);

- LSB 替换会拉平相邻灰度对频数 (卡方可测) 并破坏位面结构 (RS 可测);
- p 值高 $\neq$ 安全; 要先判断图像本底是否天然随机;
- 误报/漏报不可兼得, 灵敏度设置就是选择操作点;
- 项目 `analyze()` 是 “统计信号 + 内容基线” 的工程化组合。

想一想 |

纯 LSB 替换 “藏 1 bit/像素”, 改动率平均 50%。如果只藏少量消息、改动不到 1% 像素, 卡方还查得到吗? RS 呢? 带着这个问题进入第 4 章。再想一层: 既然图 3-1 的 LSB 位平面本就近似噪声, 为什么卡方还能抓住它? (提示: 关键在于 “干净图的奇偶频数原本不齐”, 而不是 “LSB 看起来乱”。)

第 4 章 · 矩阵编码与 F5：少改一点，藏得更多（第 5 周）

动手做 |

本章目标：理解“嵌入效率”为什么重要；能手算 $p=3$ 的汉明伴随式嵌入例子；看懂矩阵编码的原理图与效率曲线；知道 F5 的“收缩”问题出在哪。

4.1 从“每像素 1 位”到“平均每次改动藏 p 位”

朴素 LSB 每藏 1 bit 就平均改动 0.5 个像素，嵌入效率很低。如果有一种编码：每读 n 个像素组成的块，就能通过**最多改动 1 个像素**写入 p 位消息，改动次数就会大幅下降。这个想法叫**矩阵嵌入 (Matrix Embedding)**，它用到的数学工具是二元汉明码。

定义嵌入效率 α = 平均每次修改所携带的比特数。理论极限是每次修改最多携带 p 位， $\alpha(p) = p \cdot 2 / (2 - 1)$ ： $p=2$ 时约 2.67， $p=3$ 时约 3.43， $p=4$ 时约 4.27，随 p 增大而上升，但需要的块长 $n=2^{p-1}$ 也在变大。

新手提示 |

为什么 p 越大“越划算”？因为汉明码用 $n=2^{p-1}$ 个位置承载 p 位消息； p 越大， n 相对 p 的“浪费”越少，平均改 1 次能寄的信息越多。代价是块变大、可嵌入消息对载体结构更挑剔。我们先看图 4-1 怎么直观体现“7 个像素装 3 位、只翻 1 个像素”。

4.2 汉明码速成：校验矩阵与伴随式

设块长 $n=2^{p-1}$ 。构造一个 $p \times n$ 的**校验矩阵 H** ，它的 n 列恰好是 $GF(2)$ 里的全部非零向量： $p=3$ 时，七列就是二进制 1..7。把块内 n 个像素的 LSB 写成列向量 x ，定义伴随式 $s = H \cdot x \pmod{2}$ ，它是一个 p 位向量。

嵌入 p 位消息 m 时，我们想通过翻转个别位，让新伴随式等于 m 。因为 H 的每一列都不同，翻转第 j 位只会把伴随式变成 $s \oplus H_j$ ——也就是说，**任意需要的伴随式变化都对应某一行**。

1. 计算当前伴随式 $s = H \cdot x$;
2. 若 $s == m$ ，块内无需改动;

3. 否则求差值 $d = s \oplus m$ (p 位非零向量);
4. 在 H 中找到等于 d 的那一列, 记为第 j 列;
5. 翻转第 j 个像素的 LSB, 完成 ($H \cdot x = m$)。

图 4-1 矩阵编码: 7 个像素的 LSB 藏 3 位, 通常只需翻 1 个像素
 $x=1000011$ $m=100$ flip col=1

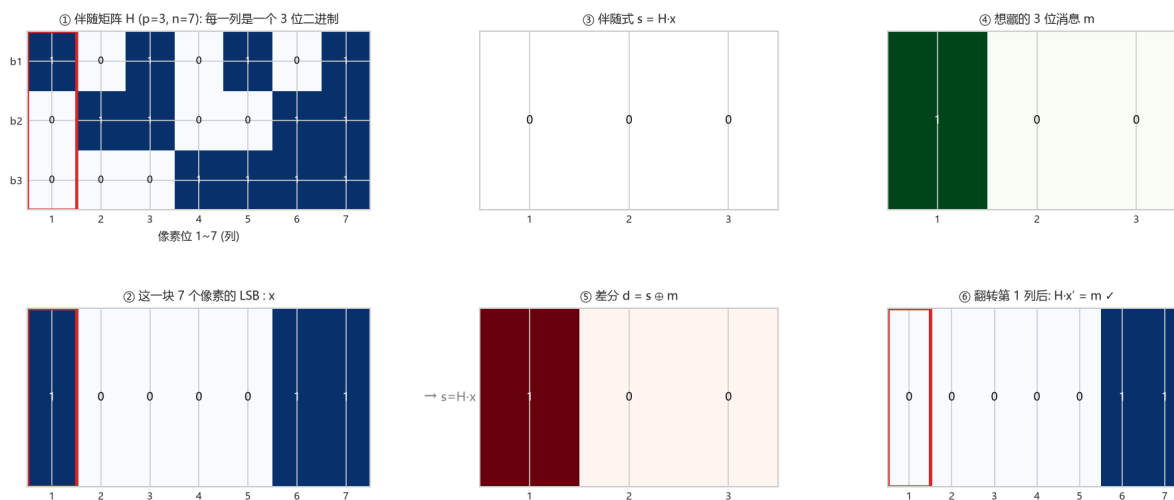


图 9: 图 4-1 矩阵编码伴随式示意

图 4-1 (真实计算: 校验矩阵 H ($p=3, n=7$), 红色框是高亮的“目标列”; 这一块的 7 个像素 LSB: x ; 当前伴随式 $s = H \cdot x$; 想藏的 3 位消息 m ; 差分 $d = s \oplus m$ 。因为 d 恰好等于 H 的第 1 列, 所以 里只需翻转第 1 个像素的 LSB, 就得到 的新伴随式 $= m$)

把这张图读透, 你就掌握了矩阵编码的全部:

- H 有 7 列, 每一列是一个 3 位二进制 (对应 17);
- $s = H \cdot x$: 把 x 里为 1 的位置对应的列异或起来, 就得到当前伴随式;
- 想让伴随式变成 m , 只需把 “等于 $d = s \oplus m$ ” 的那一列对应的像素 LSB 翻转;
- 结论: 藏 3 位消息, 绝大多数情况只需改 1 个像素。

4.3 手算一个 $p=3$ 的例子

设块内 7 个像素的 LSB 为 $x = [0, 1, 0, 1, 1, 0, 1]$ 。列向量按二进制顺序 (低位在上): $H = [1, 0, 0], H = [0, 1, 0], H = [1, 1, 0], H = [0, 0, 1]$ 依此类推。先算伴随式: x 中取 1 的位置是第 2、4、5、7 列, 做异或: $H \oplus H \oplus H \oplus H = [0, 1, 0] \oplus [0, 0, 1] \oplus [1, 0, 1] \oplus [1, 1, 1] = [0, 0, 1]$, 所以 $s = [0, 0, 1]$ 。

想嵌入 $m = [1, 1, 0]$: $d = s \oplus m = [0, 0, 1] \oplus [1, 1, 0] = [1, 1, 1]$ 。 $H = [1, 1, 1]$, 命中第 7 列, 于是只翻转第 7 个像素的 LSB: $x = [0, 1, 0, 1, 1, 0, 0]$ 。再算一次 $s = H \cdot x = [1, 1, 0] = m$, 成功。

7 个像素承载 3 位消息, 但平均只需要改动 $(2^3-1)/2^3 \approx 87.5\%$ 的块里改 1 位——约 0.875 次/块, 折算每 3 位约 0.875 次改动, 即每次改动约带 3.43 位。

动手做 |

项目里 `src/matrix_demo.py` 把这个过程变成了可点击的可视化面板 (GUI 的“矩阵编码演示”标签页)。先用 `demo_step(p, x, m_bin)` 看返回的字典里 `s`、`d`、命中的列号, 再手工验证 $H \cdot x == m$ 。

4.4 为什么高效很关键：容量、改动率与可检测性

单纯“藏得进去”不算本事；真正要紧的是同样的消息量，改得越少，统计痕迹越小，越难被检测。我们用真实公式把三种量随 p 的变化画出来，你就明白矩阵编码的取舍了。

图 4-2 矩阵编码为什么比朴素 LSB 高效：p 越大，每次改动携带越多的秘密位

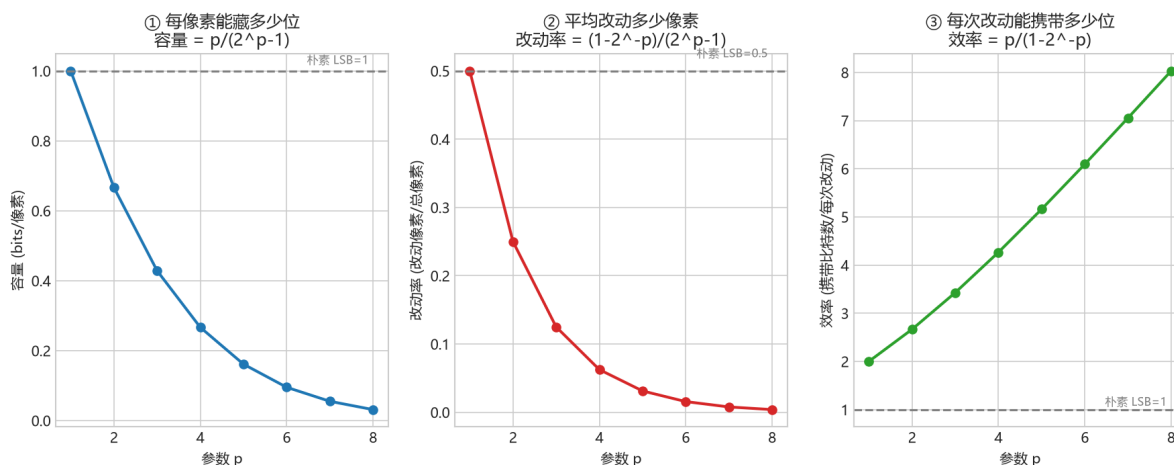


图 10: 图 4-2 嵌入效率

图 4-2 (真实计算：随参数 p 增大——每像素容量 $p/(2^p-1)$ 单调下降；平均改动率 $(1-2^{-p})/(2^p-1)$ 大幅下降；每次改动携带的比特数（嵌入效率） $p/(1-2^{-p})$ 单调上升。虚线是朴素 LSB 的基准：容量 1、改动率 0.5、效率 1)

读图要点：

- **改动率** () 是“痕迹”的直接度量：p 越大，藏同样的消息改动的像素越少，越难被统计检测抓住；
- **效率** () 衡量“每次改动有多少价值”：p=3 时约 3.43 位/改动，是朴素 LSB 的 3 倍多；
- **代价** ()：p 越大，每像素容量越小，要藏同样多的消息需要更大的载体。

把它们合到一张“容量-改动率”权衡图上，取舍一目了然：

图 4-3 (真实计算：蓝线是矩阵编码在不同 p 下的“容量 (横轴) vs 改动率 (纵轴)”；红色虚线是朴素 LSB。可以看出，矩阵编码整条线都明显“低很多”——同样的改动率下能藏更多，或同样的容量下改得更少、痕迹更小。这就是为什么第 5 章的 *nsF5* 坚持用矩阵嵌入)

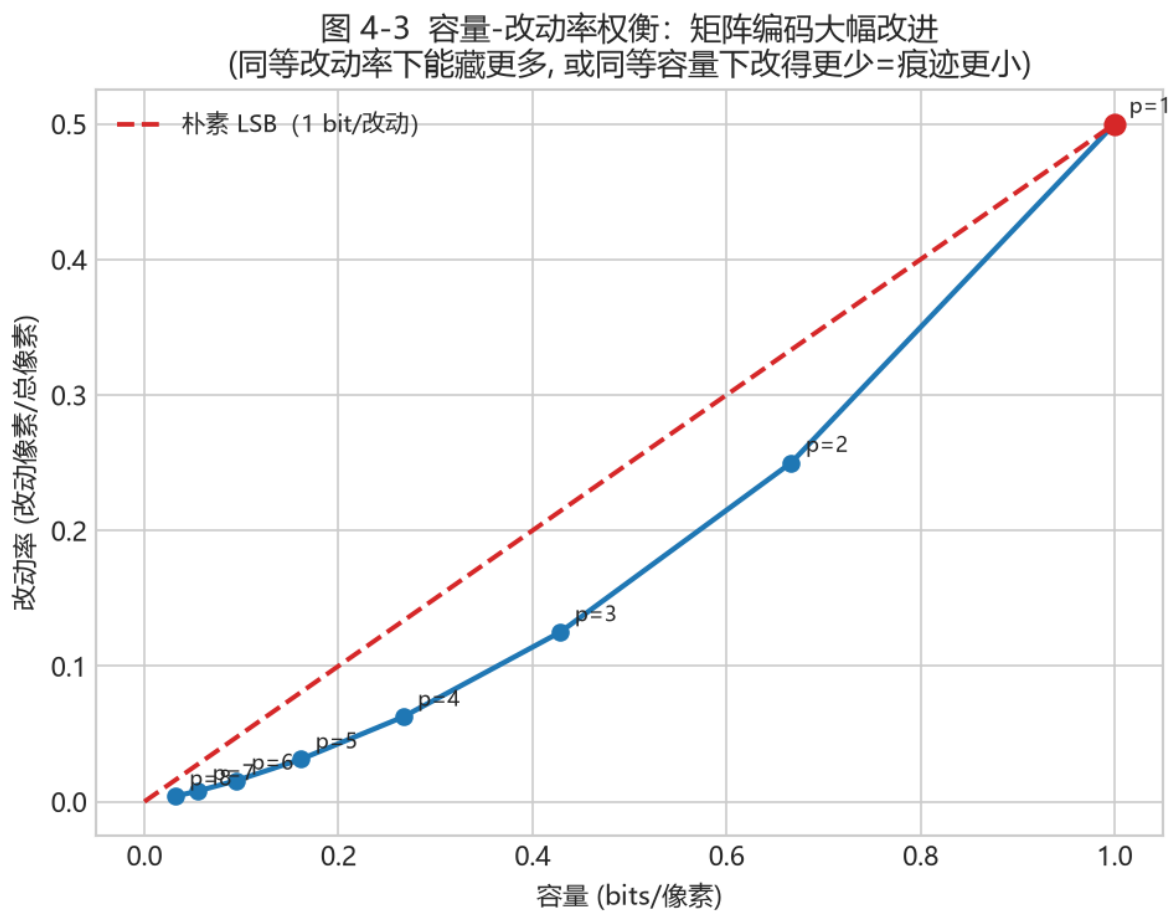


图 11: 图 4-3 容量-改动率权衡

新手提示 |

隐写性能通常用两个数衡量：**容量**（能藏多少）与**失真/改动率**（会被察觉多少）。矩阵嵌入的意义就是在**同样的失真下把容量做上去**，或在同样容量下把失真压下来。这一“改得越少越安全”的思想，贯穿第 5 章 nsF5 与第 8 章检测。

4.5 F5: 矩阵编码 + JPEG 系数减幅

F5 是 Westfeld 提出的 JPEG 隐写：在量化 DCT 交流系数上做矩阵嵌入。修改“位”不是翻转 LSB，而是让系数的**绝对值减小 1**（例如 $+3 \rightarrow +2$ 、 $-5 \rightarrow -4$ ），这样既改变 LSB 奇偶，又不容易引入显著统计异常。

问题出在“收缩”：当绝对值已经是 1 的系数被减幅，会直接变成 0。而 0 系数在 JPEG 解码里意味着“没有这个频带分量”，F5 认为它不再是合法载体，于是丢掉这块、重试后续消息。**收缩**导致实际载荷变小、修改效率低于理论值，还留下可被统计的特征。

4.6 回到项目：MatrixEmbedding

回到项目看代码 |

打开 `src/ns5_core.py` 的 `MatrixEmbedding` 类：`_embed()` 正是 4.2/4.3 步骤的直接代码——计算 `s`、比较 `m`、找列、翻转。而图 4-1/4-2/4-3 就是这段代码背后数学的可视化。

MatrixEmbedding._embed() 核心（真实代码节选）

```
x1 = (c[pos] & 1).astype(np.uint8)    # 取块内 LSB
s = syndrome(H, x1)                   # s = H·x
if np.array_equal(s, m): continue      # 恰好命中，不改
d = (s ^ m).astype(np.uint8)           # 差值
col = int(np.where(np.all(H == d[:, None], axis=0))[0][0])
c[pos[col]] ^= 1                       # 翻转命中像素
```

回到项目看代码 |

`src/efficiency.py` 的 `plot_code_family_and_efficiency()` 也会画出码族与效率图（保存到 `output/efficiency.png`），可与图 4-2 互相印证。

4.7 小结与自我检查

- 矩阵嵌入以“块”为单位： n 个位置承载 p 位、至多改 1 位（图 4-1）；
- H 列互不相同 $\Rightarrow$ 任意伴随式差值都能定位到唯一需要翻转的位置；

- 嵌入效率 $\alpha = p \cdot 2 / (2^p - 1)$, 随 p 增加但块长也增加 (图 4-2);
- **容量-改动率权衡**: 改得越少越难检测 (图 4-3), 这是 “为什么矩阵编码” 的根本答案;
- F5 在 JPEG 系数上减幅嵌入, 遇到 $|\text{系数}|=1$ 减到 0 就 “收缩”。

想一想 |

如果 F5 遇到收缩就 “跳过重试”, 消息还可能正确解码吗? 解码端如何知道哪块被跳过? 想不通没关系——第 5 章的 nsF5 用湿纸编码彻底绕开了这个难题。再想一层: 结合图 4-3, 如果我只想藏 0.2 bit/像素, 矩阵编码能把改动率压到多少 (相比朴素 LSB 的 0.1)? 这直接决定了隐写检测的难度。

第 5 章 · nsF5 与湿纸编码：项目的核心算法（第 6 周）

动手做 |

本章目标：说清“收缩”为什么伤效率；理解湿纸编码“湿点不动、干点解方程”的思想；能对照 ns5_core.py 讲出 nsF5 嵌入的完整流程。我们配了两张真实计算图（图 5-1 / 图 5-2）。

5.1 先把问题拧清楚：F5 的收缩

F5 在 JPEG 量化系数上做减幅修改：系数绝对值每被减 1，LSB 奇偶就翻转一次。但绝对值 = 1 的系数（例如 +1、-1）再减就变成 0。对 JPEG 来说，0 系数是“缺席的频带”，不适合继续当载体，F5 只能放弃这一块重新寻找位置——这就是**收缩 (shrinkage)**。

- 收缩让**实际容量**低于理论值：块被浪费，消息需要更多位置；
- 收缩让**嵌入效率**低于 $\alpha(p)$ ：为补偿损失需要更多次修改；
- 收缩还会留下可检测的统计特征：被改到 0 的系数变多，直方图出现异常。

项目在空间域（像素）上做了一个巧妙类比：把像素值减去 128 变成“有符号值” $x_v = \text{像素} - 128$ ，于是 $|x_v| \leq 1$ 的像素（127、128、129）就是“减幅会撞到零附近”的危险位置。矩阵嵌入在 JPEG 上的收缩难题，在像素域里变成了“127/128/129 不能作为普通修改对象”的同一问题。

5.2 湿纸编码：先划湿点，再解方程

湿纸编码 (Wet Paper Coding) 是 Fridrich 等人提出的框架，用来回答一个问题：如果载体里有一部分位置“碰不得”（像被水浸湿的纸，写了也看不见），能不能只用其余“干”位置完成消息嵌入？

答案是可以，而且不需要通信双方预先商量哪些位置是湿的——只要算法能从图像内容本身重建出同样的湿/干划分即可。项目正是这样做的：

1. 对每个块，先找出“减幅会变成零/危险”的位置，标记为湿点；
2. 真正的问题是：在**干位置**集合上找一个修改向量 e ，使翻转这些位置后整块的伴随式恰好等于目标 m ；

- 3. 即解方程 $H[:, \text{干}] \cdot y = d$, 其中 $d = s \oplus m$ 是期望的伴随式变化;
- 4. 项目按 “单个干列命中 $\rightarrow$ 两个干列异或命中 $\rightarrow GF(2)$ 高斯消元兜底” 的顺序求解, 尽量少改;
- 5. 解出的 y 是 0/1 向量: $y=1$ 的位置就是实际要减幅的干位置。

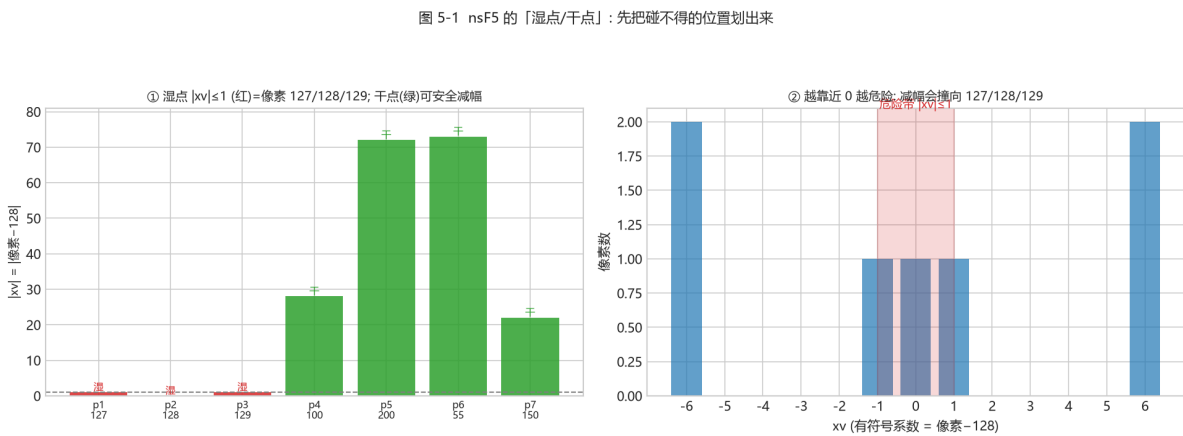


图 12: 图 5-1 湿点/干点划分

图 5-1 (真实计算: 一个 $p=3$ 的块, 像素 127/128/129 因 “ $|xv| \leq 1$ 、减幅会撞向 0” 被判为湿点 (红), 其余为干点 (绿); 把像素-128 映成有符号系数 xv , 越靠近 0 越危险)

读图要点 |

第一步永远是 “先划湿点”: 把减幅后可能变成非法值 (0 系数 / 撞零) 的位置提前排除。这样后面的嵌入只会在安全的位置上发生——这正是 nsF5 能不收缩、不重试的关键。

新手提示 |

为什么 “先标记湿点” 就不收缩了? 因为收缩的根源是 “改到一半才发现改成了 0”。nsF5 把所有会撞零的位置提前排除, 只在保证安全的干点上求解, 于是每个块都能成功嵌入, 既不用重试, 也不会出现多余的 0 系数。

5.3 读懂项目: nsF5Pixel._embed()

src/ns5_core.py 里 nsF5Pixel 类继承自 MatrixEmbedding, 只重写了 _embed()。逐行读这段代码, 你会看到完整的 nsF5 决策树:

nsF5Pixel._embed() 决策树 (伪代码流程, 逻辑与原代码一致)

```
xv = c[pos].astype(np.int16) - 128      # 像→素有符号系数
s = syndrome(H, (xv & 1).astype(np.uint8))
```

```
if s == m: continue # 伴随式已匹配
tc = 命中列(H, s ^ m) # 最“直接”的候选位置
if abs(xv[tc]) > 1: # 减幅不会撞零 → 直接改
    c[pos[tc]] -= sign(xv[tc]) # 向 128 靠拢一步
else: # 候选位置是湿点
    dry = [k for k in range(n) if abs(xv[k]) > 1]
    e = solve_wet_paper(H, dry, d) # 只在干点上解方程
if e: 按 e 减幅所有命中干点
else: 兜底翻转目标位 LSB（保证可解码）
```

5.4 再看三个求解函数

函数	作用	关键点
solve_wet_paper(H, dry_cols, target)	在干列里找尽量稀疏的解	权重 1 → 权重 2 → 高斯兜底
gauss_solve_GF2(cols, b)	GF(2) 高斯消元解线性方程组	自由变量置 0 压低改动数
permute_index(total, seed)	确定性伪随机置换	splitmix64 + Fisher-Yates, 可 C++ 加速

5.4.1 一个能算的 GF(2) 例子 (p=3, 干列足够)

设 p=3。build_hamming(3) 生成的第 j 列就是二进制数 j 的 3 位展开（最低位在上），把它写成 $h_1, \dots, h_7$ ：

$$H = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}, \quad h_1 = \begin{smallmatrix} 1 \\ 0 \\ 0 \end{smallmatrix}, h_2 = \begin{smallmatrix} 0 \\ 1 \\ 0 \end{smallmatrix}, h_3 = \begin{smallmatrix} 1 \\ 1 \\ 1 \end{smallmatrix}, h_4 = \begin{smallmatrix} 0 \\ 0 \\ 1 \end{smallmatrix}, h_5 = \begin{smallmatrix} 1 \\ 0 \\ 1 \end{smallmatrix}, h_6 = \begin{smallmatrix} 0 \\ 1 \\ 1 \end{smallmatrix}, h_7 = \begin{smallmatrix} 1 \\ 1 \\ 1 \end{smallmatrix}$$

假设块内像素 1、2、3 是湿点 ($|xv| \leq 1$ ，即 127/128/129)，干列是 {4,5,6,7}。再假设期望的伴随式差 $d = s \oplus m = (1, 1, 0)^T$ 。

图 5-2（真实计算： H 矩阵——湿列（淡红框）不参与，干列（绿框）可解，绿粗框 = 命中的列；期望的伴随式差 d；解：在干列上找尽量少的列使其异或等于 d。本例单列无命中，双列命中 $h \oplus h = d$ ，于是改第 4、7 两列）

读图要点 |

- **单列命中**：看干列里有没有一列恰好等于 d（本例没有）；- **双列命中**：看有没有两列异或等于 d（本例 $h \oplus h = [0,0,1] \oplus [1,1,1] = [1,1,0] = d$ ，命中）；- 若都不行，才上 GF(2) 高斯消元兜底。**核心：只在“干点”上解方程，湿点完全不动。**

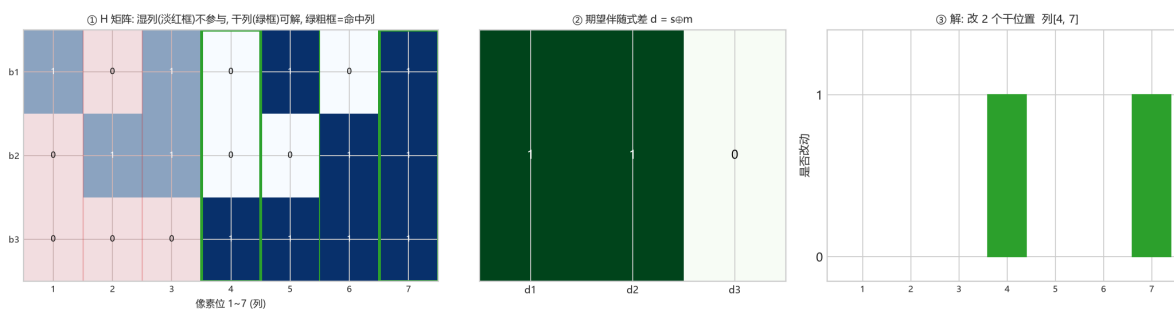
图 5-2 湿纸编码：在干列上解 $H\mathbf{y} = \mathbf{d}$ （本例： $\mathbf{h}_4 \oplus \mathbf{h}_7 = [0, 0, 1] \oplus [1, 1, 1] = [1, 1, 0] = \mathbf{d} \checkmark$ ）

图 13: 图 5-2 湿纸编码求解

第一步，单列命中：看四个干列有没有一个恰好等于 $\mathbf{d} = (1, 1, 0)^\top$ ： $\mathbf{h}_4 = (0, 0, 1)^\top$ 、 $\mathbf{h}_5 = (1, 0, 1)^\top$ 、 $\mathbf{h}_6 = (0, 1, 1)^\top$ 、 $\mathbf{h}_7 = (1, 1, 1)^\top$ 。都没有等于 $(1, 1, 0)^\top \rightarrow$ 单列无命中。

第二步，双列异或：找两个干列异或等于 $\mathbf{d}$ 。例如 $\mathbf{h}_5 \oplus \mathbf{h}_6 = (1, 0, 1)^\top \oplus (0, 1, 1)^\top = (1, 1, 0)^\top = \mathbf{d}$ 。**命中！**于是只改这两个干位置（对应的系数），无需高斯。（代码按随机顺序找第一对命中，实际可能采到 $\mathbf{h} \oplus \mathbf{h}$ 这一组，与 $\mathbf{h} \oplus \mathbf{h}$ 等价；二者都满足 $\text{syndrome}(\mathbf{H}, \mathbf{e}) = \mathbf{d}$ 。）

5.4.2 什么时候才需要高斯消元？——干列张成整个 $\text{GF}(2)$

上面干列 $\{4, 5, 6, 7\}$ 有 4 个，它们张成整个 $\text{GF}(2)^3$ ，所以绝大多数 $\mathbf{d}$ 都能用单列或双列命中。但如果干列太少或线性相关，张成空间就缩水。

例：只有干列 $\{4, 5\}$ （湿点更多），它们及两两异或能覆盖的伴随式只有 4 种： $(0, 0, 0)$ 、 $\mathbf{h}_5 = (1, 0, 1)^\top$ 、 $\mathbf{h}_6 = (0, 1, 1)^\top$ 、 $\mathbf{h}_5 \oplus \mathbf{h}_6 = (1, 1, 0)^\top$ 。若 $\mathbf{d} = (0, 0, 1)^\top$ ，这几种都覆盖不到——**无解**。

为什么？ $\text{GF}(2)$ 高斯消元把干列拼成 $A = [\mathbf{h}_5 \ \mathbf{h}_6]$ ，这是 3×2 矩阵，秩最多为 $2 < p=3$ ，**不可能张成 3 维空间**。只有当 $\mathbf{d}$ 落在 $\text{span}(A)$ 里才有解； $\mathbf{d} = (0, 0, 1)^\top \notin \text{span}\{(1, 0, 1)^\top, (0, 1, 1)^\top\}$ ，所以 `gauss_solve_GF2` 返回 `None`，`solve_wet_paper` 也返回 `None`。**这就是“干点不足”的数学本质：秩 $< p \Rightarrow$ 某些消息位无法实现。**

可解性判据 |

要在干点上任意嵌入 p 位消息，干列数通常得 $\geq p$ ，并且这些干列要张成整个 $\text{GF}(2)$ 。这正是“湿纸编码需要足够干点”的原因（nsF5 里干点几乎总是够用；极端情形代码会兜底翻转目标位以保证可解码）。

5.4.3 自由变量置 0 = 尽量少改

高斯消元解 $A\mathbf{y} = \mathbf{d}$ 一般有无穷多个解（当列数多于秩时）。`gauss_solve_GF2` 把**自由变量置 0**，等效于“只在必需的干列上改动”，从而**压低改动像素数**——这既提升嵌入效率，也让统计痕迹更轻。

回到项目看代码 |

打开 `src/ns5_core.py` 第 145–215 行，对照 5.4 的表格逐段读。建议先读函数注释，再用 `src/test_core.py` 的 `test_wet_paper_needed()` 验证：它把像素强制设成 127/128/129 制造湿点，仍能完成嵌入/解码往返。

5.5 验证实验：效率与往返

运行核心自测 + 对比两种方法

```
python src\test_core.py
# 对比 matrix 与 nsF5 改动像素数：
import sys; sys.path.insert(0, "src")
import numpy as np; from ns5_core import embed_string
img = np.random.default_rng(0).integers(0, 256, (64, 64), np.uint8)
for method in ("matrix", "nsF5"):
    stego, rep, nb = embed_string(img, "Hello_nsF5!", method=method, p=3)
    print(method, "改动", rep["cover_changed"], "像素_/_容量", nb)
```

对同一张随机图、同一条消息，`matrix` 与 `nsF5` 的改动数不一定谁更少——因为 `nsF5` 的保护范围 (127/128/129) 在随机图上很少触发。真正的差异出现在“危险像素密集”的图上：`nsF5` 依然能无收缩完成嵌入，`matrix` 则可能反复失效。

动手做 |

把 `img` 换成一整块都是 127/128/129 的“危险图”，再对比两种方法的成功次数与改动数，直观感受湿纸编码的价值。

5.6 小结与自我检查

- 收缩 = 减幅撞零导致块作废，损伤容量、效率与统计安全（图 5-1 的“危险带”）；
- 湿纸编码把“不能改的位置”预先划为湿点，只在干点解伴随式方程（图 5-2）；
- `nsF5` = 矩阵嵌入 + 湿纸编码，无收缩、无重试；
- 项目在像素域用 $xv = \text{像素} - 128$ 模拟 JPEG 系数， $|xv| \leq 1$ 视为湿点。

想一想 |

为什么湿纸编码“不需要预先约定湿点位置”？解码端怎么知道哪些像素不能用来承载消息？提示：解码端只需要读 LSB 算伴随式，它不需要知道嵌入时绕过了谁。再想一层：如果一块里湿点太多、干点张不成整个 $GF(2)$ ，会发生什么？（结合 5.4.2 的秩论证。）

第 6 章 · 口令、SHA-256 与 “键控” 安全设计 (第 6—7 周)

动手做 |

本章目标：理解为什么嵌入位置要“由密钥决定”；会讲清解码自同步与篡改感知的原理；知道这套设计的边界在哪里。我们配了两张真实计算图（图 6-1 / 图 6-2）。

6.1 如果嵌入位置是固定的，会怎样？

假设每次都是从左上角开始顺序嵌。攻击者知道算法后，可以：直接读取前 N 个像素的 LSB 还原消息；把消息原位复制到另一张“看起来相同”的图像上（位置替换攻击）；对固定区域做针对性扰动，既破坏你的消息又不改动其余图像。

因此现代隐写会把嵌入路径做成**密钥控制的伪随机顺序**：没有口令（或不知道内容哈希）的人，不知道消息在哪一块，也无法进行位置级攻击。

图 6-1 嵌入位置是「由密钥决定的伪随机顺序」：两张不同口令给出完全不同的路径

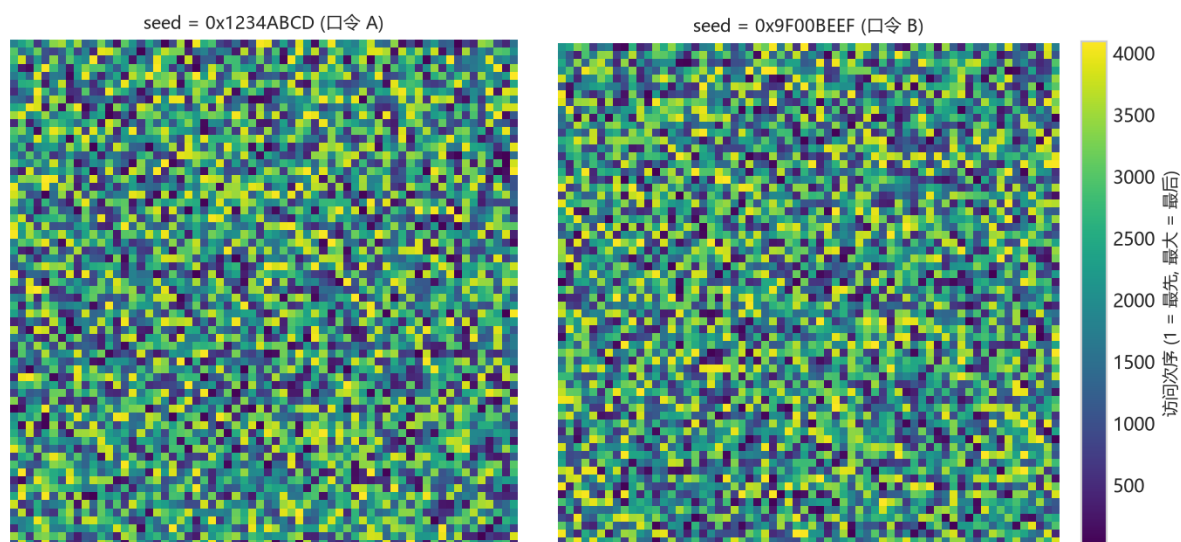


图 14: 图 6-1 键控置换路径

图 6-1 (真实计算：用项目 `permute_index()` 的 `splitmix64 + Fisher-Yates` 算法，在 64×64 格子上画出 “访问顺序”。向左用口令 *A*，向右用口令 *B* —— 两幅图的 “先后次序” 完全不同，看起来都像随机噪声)

读图要点 |

颜色越亮越 “先被访问”。换成不同口令，整张图的次序就彻底不同 —— 攻击者不知道口令，就无法预测 “消息藏在哪些位置”。这就是**键控**的意义：**安全来自 “不知道位置”，而不是 “藏得看不见”**。

6.2 两条种子规则：先认图，再找正文

项目把像素分成两个池：**头部池** (开头 `N_h` 个块) 与**正文池** (其余像素)。密钥派生用两个独立的种子：

- `seed0` = **仅由口令派生**：决定头部池的置乱顺序。头部里预埋的是 `cover_hash` —— 由原图像字节计算的 SHA-256 截断前 16 字节 (128 位)；
- `seedB` = `cover_hash` + **口令联合派生**：决定正文池的置乱顺序与分块。

解码时，接收方先用自己的口令重建 `seed0`，读回头部取出 `cover_hash`，再与口令一起重建 `seedB`，才能找到正文。口令错误或头部被破坏，后续路径立刻失效 —— `test_pwd_mismatch()` 测的就是这一点。

项目中的种子派生 (真实代码)

```
def derive_seed(image_bytes, password=""):
    base = hashlib.sha256(image_bytes).hexdigest()
    if password:
        base = hashlib.sha256(
            base.encode() + password.encode()).hexdigest()
    return int(base, 16)
```

6.3 确定性置乱：splitmix64 + Fisher-Yates

有了种子还不够，还要把它变成 “1..N 的一个确定排列”。项目用 `splitmix64` 作为伪随机源，执行 Fisher-Yates 洗牌：从末尾向前，每一步用伪随机数选一个位置交换。同样的种子必然产生同样的排列 —— 这正是 “自同步” 的数学基础。

图 6-2 (真实计算：横轴 = 像素位置，纵轴 = 访问次序。口令 *A* (蓝)、口令 *B* (红) 各自都是一条 “每点只出现一次” 的置换；灰色虚线是不置乱的顺序 (可直接预测 = 危险)。同一口令总是同一条曲线 → 自同步；换口令则完全打散)

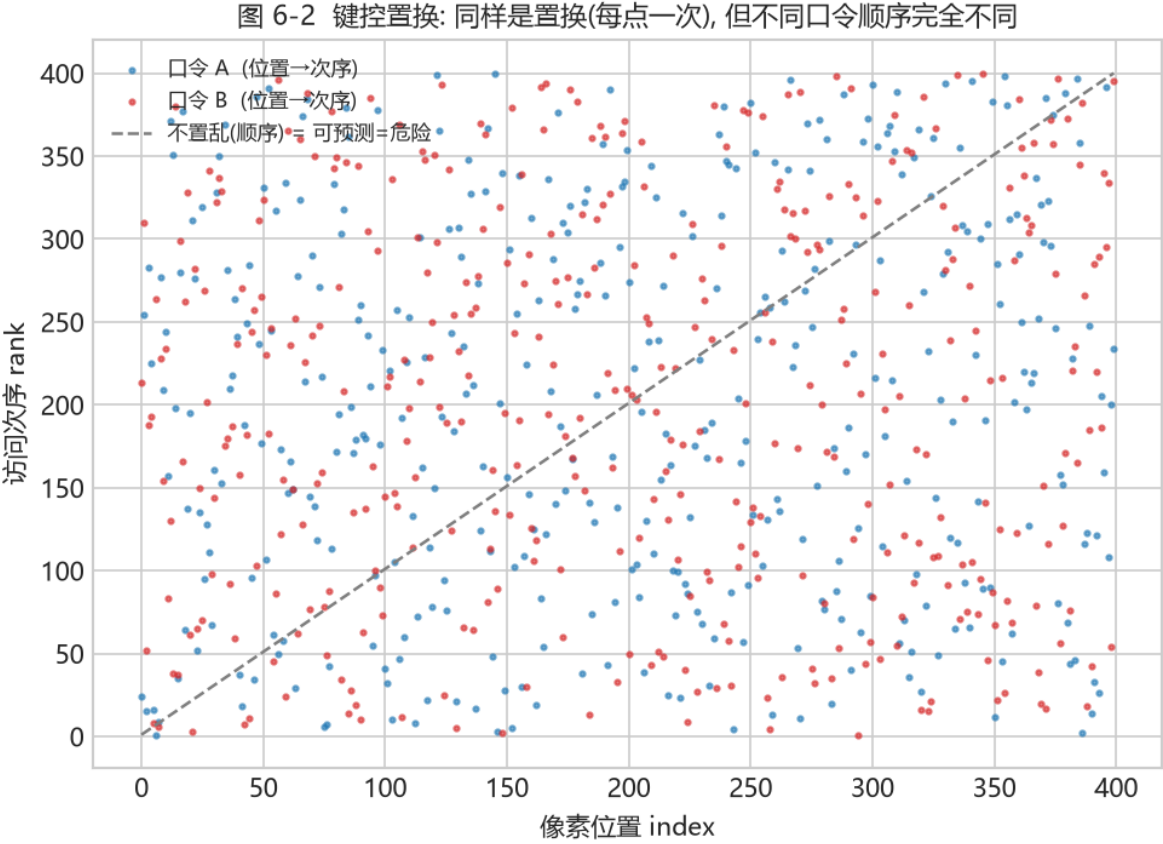


图 15: 图 6-2 键控置换散点

回到项目看代码 |

打开 `src/ns5_core.py` 的 `permute_index()`: DLL 可用时调用 C++ 的 `nsf5_permute`, 否则走 Python 同算法 fallback。两套实现必须给出完全相同的排列, 否则用 DLL 嵌入的图在无 DLL 的机器上无法解码。`src/cppembed.py` 的 `selfcheck()` 专门做这种跨语言一致性校验。

6.4 篡改感知：到底能感知什么

嵌入端在报告中给出 `cover_hash` (原图哈希)。任何像素被改动, 图像字节的 SHA-256 都会改变。把收到的图像重新计算哈希, 与嵌入端报告的 (或头部解出的) 哈希比较: 不一致 = 图像被动过。更妙的是, 由于正文路径依赖这个哈希, 即使攻击者只改了一个像素, 正文置乱种子也随之错位, 解码结果会立刻崩溃或乱码。

同时要诚实地说清边界: 这套机制是**键控 (keying) 与篡改感知**, 不是密码学意义上的**完整性认证**。头部哈希本身没有用密钥做 MAC, 理论上有能力重嵌整张图的攻击者可以同步更新头部哈希。论文与 README 都把这个列为后续工作 (例如引入密钥化 MAC), 阅读时注意区分 “检测普通篡改” 与 “抵御恶意重嵌”。

新手提示 |

一句话区分: **篡改感知** = “我发现图被动了”; **完整性认证** = “我能证明这图一定没被动过、且动过的人无法掩盖”。前者是 “检测”, 后者是 “抵御”, 强度不同。

6.5 动手实验：口令错误与像素篡改

实验 A: 口令不匹配应失败; 实验 B: 改 1 像素后解码

```
import sys; sys.path.insert(0, "src")
import numpy as np; import image_io as IO
from ns5_core import embed_string, extract_string, get_image_hash

img = IO.load_as_gray("img/cover.png")
stego, rep, _ = embed_string(img, "secret", method="nsF5",
                             p=3, password="A")
print(extract_string(stego, method="nsF5", p=3, password="B"))
tampered = stego.copy(); tampered[0, 0] ^= 1  # 只改 1 个像素
print(get_image_hash(stego)[:16], "vs", get_image_hash(tampered)[:16])
print(extract_string(tampered, method="nsF5", p=3, password="A"))
```


避坑提醒 |

实验 B 的失败方式可能是 `ValueError`、乱码或截断消息，取决于被改像素落在哪块。不要期待 “总是抛异常” ——篡改感知的工程表现就是：与哈希比对失败，且正文无法自同步。

6.6 小结与自我检查

- 固定路径 = 可预测 = 可被复制与针对；键控路径让隐藏位置由口令 + 内容决定 (图 6-1)；
- 头部放 128 位内容哈希，正文种子依赖该哈希 → 解码无需外部同步；
- 确定性置换 (`splitmix64` + Fisher–Yates) 是自同步的数学保证 (图 6-2)；
- 键控 $\neq$ 完整性认证；强对抗场景需要密钥化 MAC。

想一想 |

如果两张不同的原图被嵌入了同一条消息，含密图的隐藏位置会一样吗？为什么？（提示：正文种子由什么决定？）再想一层：既然安全性靠 “不知道位置”，那攻击者能不能通过大量统计 “猜” 出置乱规律？（参见第 8 章，这正是检测的切入点。）

第 7 章 · 机器学习基础（写给第一次学的人）（第 7—12 周）

动手做 |

本章目标：完整、扎实地学会四件事——**一张图怎么变成数字（特征）** → **模型怎么学会判断** → **怎么防止“作弊”** → **怎么判断模型好不好**。我们从一个完全没接触过机器学习的人也能读懂的角度讲起，配合真实数据生成的图，循序渐进，不赶时间。

7.0 本章路线图：建议怎么学（约 3—6 周）

如果你有半年到一年的学习周期，完全可以把这一章当成**第一次认真学机器学习**的入门课，而不仅仅是“看懂项目代码”的速成。建议这样走：

1. **第 1 步（约 1 周）**：只读 7.17.2，建立「数据 → 特征 → 标签」的整体印象，能说出自己平时见过哪些“用数字描述一张图”的例子。
2. **第 2 步（约 1 周）**：读 7.37.5，配合图 7-1/7-2/7-3，理解“画一条线 + 把线变成概率 + 让概率尽量对”。**这里不要求会推公式**，能看着图复述大意即可。
3. **第 3 步（约 1 周）**：读 7.67.8，重点理解**数据泄漏**和**怎么评估**。这是最容易在真实项目里踩坑的地方，也最值得多花时间。
4. **第 4 步（约 1 周）**：把本章所有代码摘录在你自己电脑上跑一遍（python src\train_model.py），对着输出确认每行数字的来历。

学习方法建议 |

每节末尾都有「想一想」。请把答案写下来，而不是只在心里过一遍——写出来的过程才是真正在学。

7.1 隐写检测，其实就是“猜一张图藏没藏东西”

换一个角度看问题：一张照片，要么是干净的（标成 0），要么往里面藏过消息（标成 1）。机器学习要做的，就是学会一个判断：给它一张图，尽量猜对是 0 还是 1。

这和“老师把很多带答案的题讲给学生，学生考试时对没见过的题也能做对”是一回事——机器学习不会“背”，它学的是**规律**。整个监督学习只有四样东西：**数据、特征、模型、评估**。下面一个来，每一个都对应到项目代码。

术语小字典 |

本章你会反复看到这四个词，先记住它们的“人话版”：- **数据** Data：一堆“带答案的例子”。- **特征** Feature：把每张图压缩成的一串数字（相当于“用尺子量图”）。- **模型** Model：一个“从特征到答案”的判断规则。- **评估** Evaluation：判断这个规则到底靠不靠谱。

7.2 一张图怎么变成“数字”：特征与标签

电脑不认识“图片”，只认识数字。所以第一步是把一张图**压缩成一小串数字**。项目 v1 版把每张图压成 **11 个数字**，v1.4 又扩展成 143 个（第 8 章再讲）。这 11 个数字就叫**特征向量**，它等于“用 11 把尺子去量这张图”。

- **特征 (feature)**：这 11 个数字。比如“像素差值乱不乱”“亮暗分布对不对”。
- **标签 (label)**：这张图到底藏没藏，0 = 干净，1 = 含密。
- **样本 (sample)**：一行 = 一张图，11 个特征数字 + 1 个标签。

数据集长什么样（前几列是特征，*label* 是答案，*photo_id* 先别管）

```
Rm,Sm,Rn,Sn,RS_Gr,RS_Gn,chi2_pvalue,diff_entropy,lsb_diff_entropy,median_prefix_p
,chi2_stat,label,photo_id,...
41316,4574,36608,5057,0.56,0.48,0.56,1.99,0.986,2.7e-166,1719.3,0,0,...
40488,4907,35701,5342,0.54,0.46,0.61,2.02,0.989,2.2e-155,1675.9,1,0,...
```

data/dataset.csv (真实前两行，数值截断显示)

新手提示 |

看到 *chi2_pvalue*、*RS_Gn* 这种名字不用慌，它们就是“第几把尺子量出来的数”。第 8 章会逐个说每把尺子量的是什么。

把表写成数学记号，后面所有公式都用它（别怕，就是“第 i 行、第 j 列”的写法）：

- **样本**：一行一个样本，共 N 个，记 $i = 1, \dots, N$ ；
- **特征**：第 i 个样本的 11 个数字记为向量 $\mathbf{x}_i$ ；全部行拼成矩阵 X (N 行 $\times$ 11 列)；
- **标签**： $y_i \in \{0, 1\}$ ，拼成向量 $\mathbf{y}$ 。

7.3 分类 = 画一条线，把两类分开

有了一堆“带答案的数”，模型要学的是什么？就是在数字空间里画一条线，一边是干净、一边是含密。可以想象成：横轴是“乱的程度”，纵轴是“亮暗是否均匀”，干净图大多在一角，含密图在另一角。模型学的就是那条分界线。

图 7-1 决策边界：逻辑回归把特征空间分成两类
(真实数据·取两个特征示意)

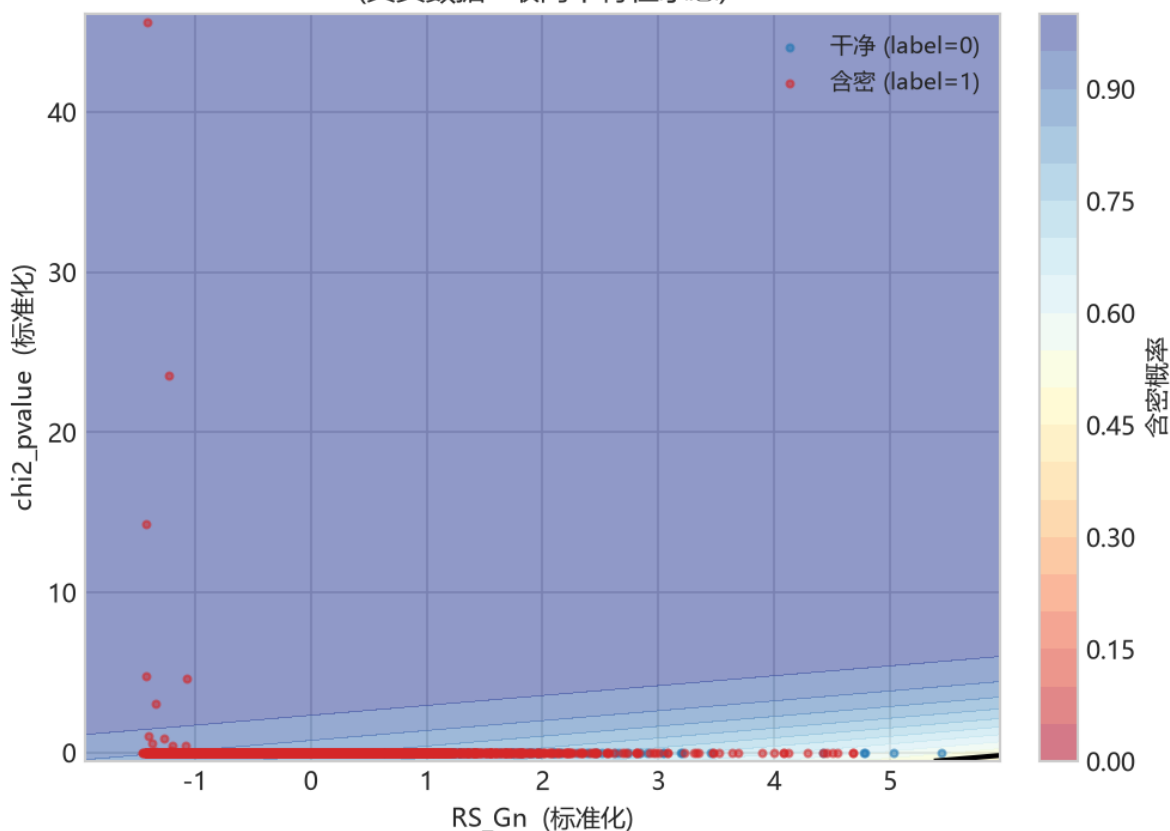


图 16: 图 7-1 决策边界

图 7-1 (真实数据: 用 RS_Gn 与 $chi2_pvalue$ 两个特征, 逻辑回归学出的决策边界; 蓝 = 干净, 红 = 含密, 色块 = 模型认为的含密概率)

把这张图看明白，是理解整章的关键。注意几点：

1. **两类不是完全分开的**：边界附近蓝点和红点混在一起。说明这两个特征还不足以完美区分，但已经能“大致”分开了。
2. **色块是“概率”**：颜色越红，模型越认为含密；越蓝越认为干净。中间那条黑线就是“概率 = 0.5”的分界。
3. **维度变多也一样**：真实模型用 11（或 143）个特征，就是在这 11（或 143）维空间里画一个“超平面”。维度高了画不出来，但道理和这张二维图完全一样。

这条线在数学上就叫**线性判别函数**，其实就是一个“打分式子”：

$$z = w_1x_1 + w_2x_2 + \cdots + w_{11}x_{11} + b.$$

别被符号吓到，它就是一个加权求和：

- $x_1, \dots, x_{11}$ 是那 11 个特征数字；
- $w_1, \dots, w_{11}$ 是**权重**，决定“哪个特征重要、往哪个方向倾斜”。比如某个特征对判断特别有用，它的 w 就大；
- b 是**偏置**，相当于把整条线上移或下移一点，用来定位置。

判的时候很简单： z 大于等于 0 判成含密，小于 0 判成干净。**这一整行代码** `LogisticRegression` 就是在自动找最合适的 w 和 b 。

想一想 |

图 7-1 里，你觉得“RS_Gn”和“chi2_pvalue”哪个对判断更重要？如果只保留一个特征，你会选哪个？——这个直觉，会在 7.7 的“特征重要度”图里被验证。

7.4 从“分数”到“概率”：为啥要套一层 sigmoid

上面那个 z 是一个可正可负、没有上限的“分数”。但用户更想知道的是一个 0 到 1 之间的**概率**：“这张图有多大可能是含密”。于是加一层 sigmoid 把分数压到 01：

$$\hat{p} = \frac{1}{1 + e^{-z}}.$$

- z 很大 (分数高) $\rightarrow \hat{p}$ 接近 1 (多半是含密)；
- z 很小 $\rightarrow \hat{p}$ 接近 0 (多半干净)；
- $z = 0 \rightarrow$ 恰好 0.5 (一半一半)。

图 7-2 (*sigmoid* 把无上界、可正可负的“得分 z ”压缩成 01 的“概率”。左半蓝区 = 更像干净，右半红区 = 更像含密)

新手提示 |

你可以把 sigmoid 想成一个“百分比换算器”：把没边界的分数，换成很好懂的 0%100%。名字叫“**逻辑回归**”，其实干的还是“画一条线 + 输出概率”。

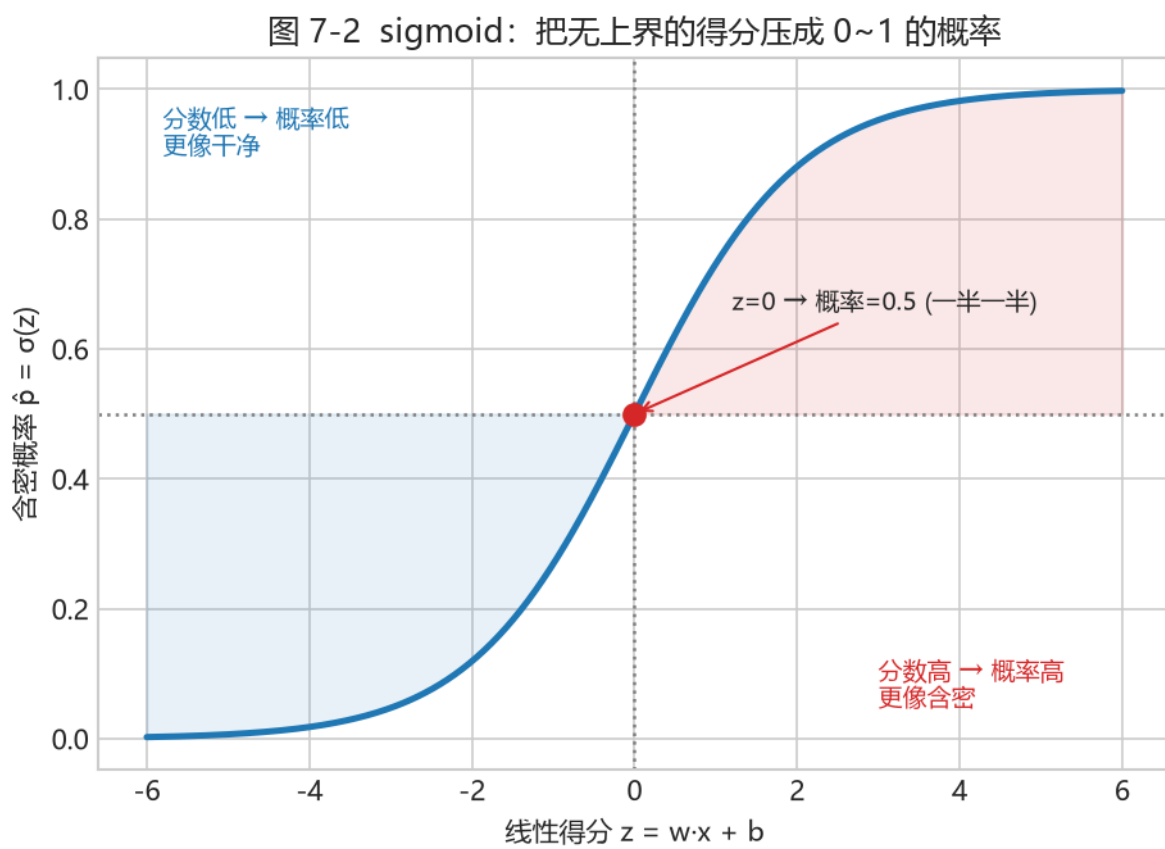


图 17: 图 7-2 sigmoid

7.4.1 为什么不直接用 0/1，而要用“概率”？

好问题。因为真实世界没有“非黑即白”。图 7-1 里边界附近那些点，模型也不确定。与其让它勉强说“0”或“1”，不如老老实实给出“我有 73% 把握认为含密”。这样：

- 用户可以自己决定**阈值**（加到 7.7 讲）；
- 可以评估模型**到底有多确定**（概率校准）；
- 可以画出 ROC / AUC 这种**与阈值无关**的指标（7.7.1）。

术语小字典 |

输出概率的分类器叫**软分类** (soft classification)；直接输出 0/1 的叫**硬分类** (hard classification)。逻辑回归是软分类，这也是它能算 AUC（曲线下面积，衡量模型把含密排在干净前面的能力，越大越好，详见 7.7.1）的前提。

对照项目代码——`src/train_model.py`：

```
def lr(seed=0): return make_pipeline(StandardScaler(), LogisticRegression(
    max_iter=2000, C=0.1, random_state=seed))
```

代码	人话解释
StandardScaler()	先把每个特征调成“差不多大”，免得某个数字太大把分界线带偏（见 7.4.2）
LogisticRegression	就是上面“画线 + sigmoid 输出概率”那套，自动找 w 、 b
max_iter=2000	最多调整 2000 次，防止它没算完就停
C=0.1	一个“让它别太用力记答案”的旋钮（正则化，见 7.5.2）

想了解再点 |

为什么先标准化？因为“卡方统计量”可能是几千，“Gn”却只有 01。权重对它俩一样敏感的话，梯度会拼命去迁就那个大数字，线就被带歪了。标准化的公式是 $x'=(x-\mu)/\sigma$ ，把每个特征变成“均值 0、标准差 1”，大家就公平了。注意 μ 、 σ 只能从训练集算，别把测试集偷偷用进去。

7.5 模型怎么“学会”：损失与训练

“学会”到底在干嘛？就是**不断调整 w 和 b ，让输出概率尽量接近正确答案**。怎么衡量“接近”呢？用一个**损失 (loss)**——你可以理解成“猜错要扣多少分”。经典做法是交叉熵 (log loss)：

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right].$$

大白话：当正确答案是 1（藏了）时，只看 $\hat{p}_i$ ——它越接近 1，扣分越少；当正确答案是 0 时，只看 $1 - \hat{p}_i$ ——它越接近 0，扣分越少。猜得越离谱，扣得越狠。

那怎么让扣分变少？一点点试：算一下“往哪个方向微调、扣分降得最快”，就沿那个方向挪一小步。这叫**梯度下降**：

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}.$$

图 7-3 梯度下降：沿损失下降最快的方向一步步走到谷底

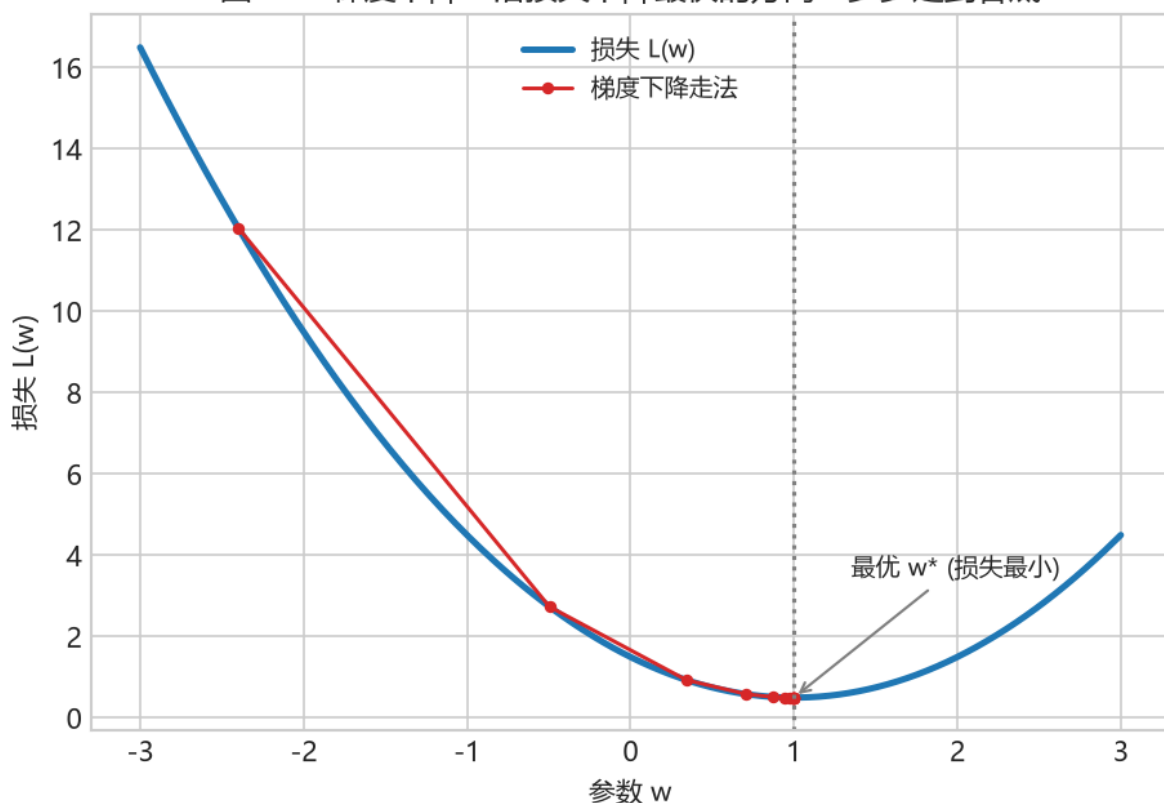


图 18: 图 7-3 梯度下降

图 7-3（一个简单的“损失 vs 参数”曲线。红色阶梯就是梯度下降：从左边出发，一步步走到谷底——损失最小的那个 w^* 。蓝线是损失函数，谷底就是最优参数）

新手提示 |

你完全不用会推导数。只要记住三句话：**损失 = 模型错得有多离谱**；**训练 = 找参数让损失最小**；**梯度下降 = 顺着“让损失变小最快的方向”一点点挪**。这些 scikit-learn 都帮你封装好了。

7.5.1 从“一步到位”到“一步步走”，为什么非得这么绕？

理论上，逻辑回归的损失是“凸函数”，可以一步算出最优解。但真实项目里：

- 数据量一大，直接求逆矩阵慢且不稳定；
- 我们要的是**能把问题推广到很多模型**（树、神经网络）的通用方法；
- 梯度下降是**任何**把“最小化损失”当成目标的模型的共同引擎。

所以理解梯度下降，等于同时理解了线性模型、树模型、甚至神经网络的“训练”是怎么一回事。这就是它的价值。

7.5.2 一个“防过头”的旋钮：C 与正则化

如果特征之间高度相关或样本很少，模型可能把权重 w 调得巨大，去死记硬背训练数据——这是**过拟合**的征兆。防的办法之一是给损失加一项“惩罚”，要求权重别太大（L2 正则）：

$$\mathcal{L}_{\text{reg}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \lambda \sum_j w_j^2.$$

scikit-learn 用的是**逆正则化参数** $C = 1/\lambda$ ：

- $C=0.1 \rightarrow \lambda = 10$ ，强正则，权重被狠狠压缩，模型偏保守；
- $C=1.0 \rightarrow \lambda = 1$ ，中等； C 越大正则越弱。

回到项目看代码 |

`train_model.py` 里基模型用 `LogisticRegression(C=0.1)`（想让基模型“克制、别乱记”），而堆叠器的 meta 学习器用 `C=1.0`（放松一点，让它自由综合）。同一个类，两个 C ，体现“基模型要稳、meta 学习器要灵”的取舍。

7.6 怎么不让模型“作弊”：交叉验证与分组

模型在见过的题上拿高分很容易，真正要看的是**没见过的题**。如果模型把训练样本硬背下来，遇到新题就崩——这叫**过拟合**。

防过拟合的标准办法是**交叉验证**：把数据切成几份，轮流拿一份当“小考”，其他当“复习”。但这里有个特别容易踩的坑。

避坑提醒 |

同一张照片有 7 个变体（1 个干净 + 6 个藏过东西），它们互相**太像了**。如果随便切，训练里可能会混进某张测试照片的“亲兄弟”，等于提前把答案告诉了模型——这叫**数据泄漏**，分数会虚高到离谱。项目的做法是**按 photo_id 分组**：同一张照片的所有变体，要么全部进训练、要么全部进测试，绝不分开。

图 7-4（示意：每个 P 是一张照片，一行有 7 个变体。上排“随机切分”会把同一张照片的 7 个变体拆进不同折——漏题；下排“*GroupKFold*”让同一张照片的 7 个变体进同一折——诚实）

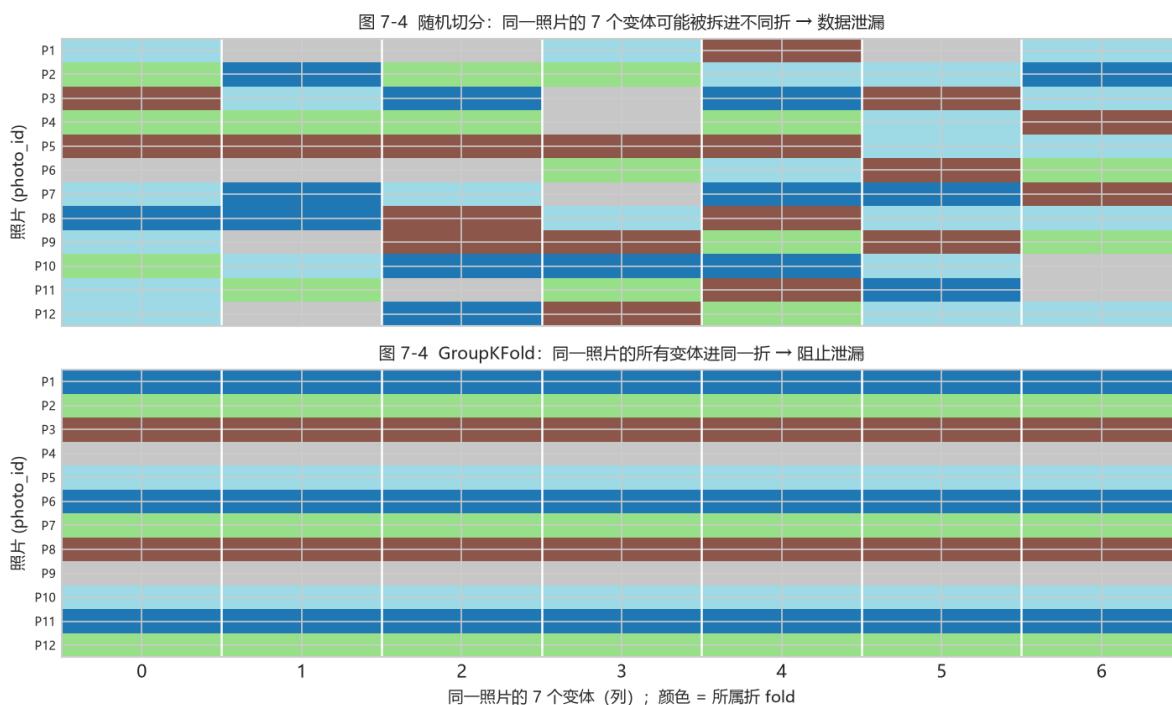


图 19: 图 7-4 GroupKFold

对照代码——`src/train_model.py::cv_oof()` 的核心：

```
from sklearn.model_selection import GroupKFold
gkf = GroupKFold(n_splits=5)
for tr, va in gkf.split(X, y, groups):    # groups = photo_id, 按照片分组再切
    clf.fit(X[tr], y[tr])
    p = clf.predict_proba(X[va])[:, 1]    # 只在这折“小考”上打分
```

关键就是 `groups` 这个参数。它告诉切分器：“同一组的样本不能拆散”。换成普通的 `KFold` 就会把同一张照片的“亲兄弟”拆到两边，分数就作弊了。

想一想 |

为什么“同一张照片的 7 个变体”算不上一份独立样本？试着设计一个最小例子：假如只有 10 张照片，随机切分有多大概率把某张照片的“亲兄弟”拆到训练/测试两边？这样的“泄漏”，对分数影响有多大？（提示：想想 7.7.1 的 0.5 随机线）

7.7 评估：别只盯“准确率”

模型输出的是概率，得定一个阈值才能说 0 还是 1。评估一套指标，先看混淆矩阵：

图 7-5（真实数据在 *Youden* 阈值下的混淆矩阵：*TN*= 正确拒绝、*FP*= 误报、*FN*= 漏报、*TP*= 正确检出。数字是真实样本数）

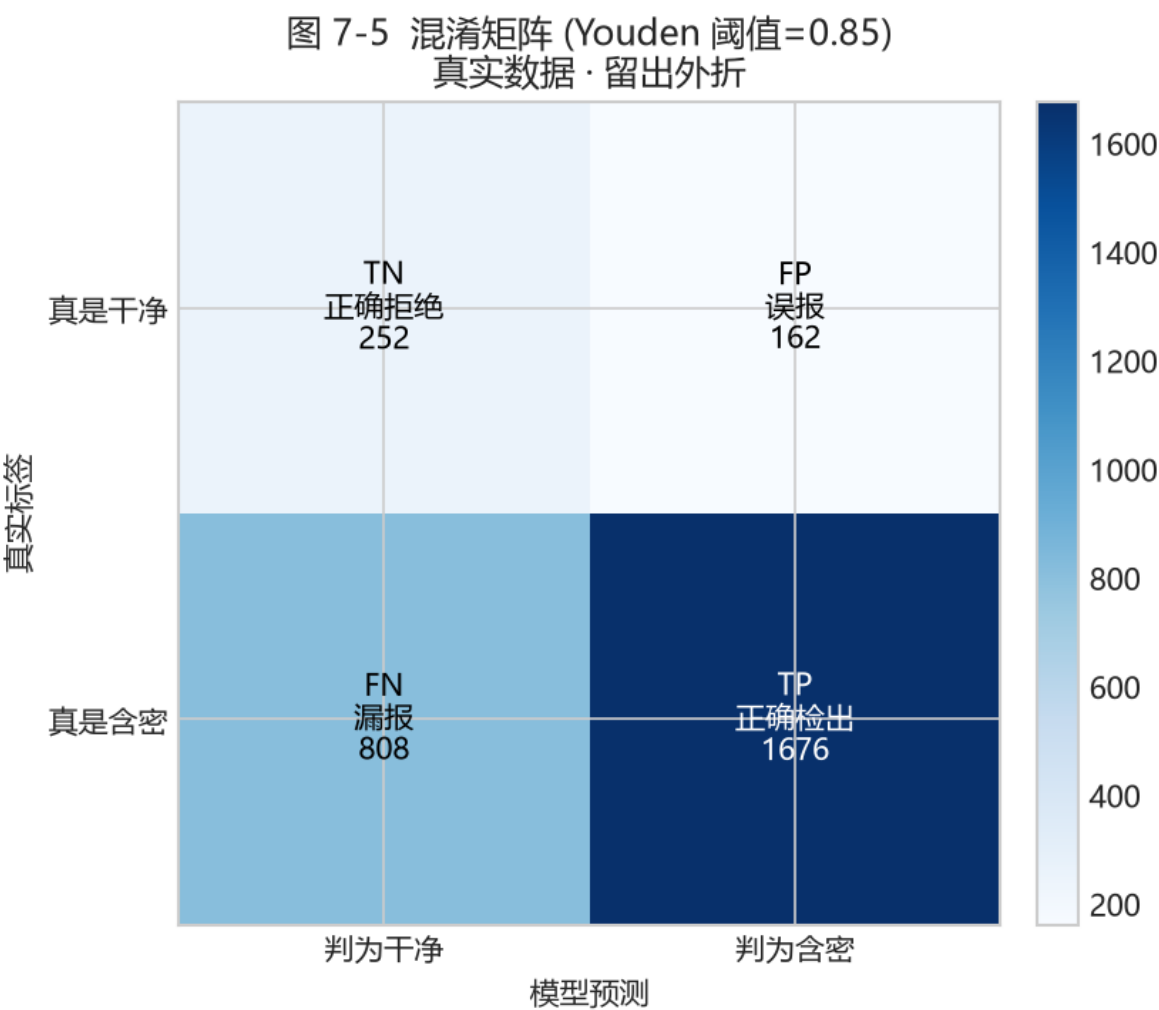


图 20: 图 7-5 混淆矩阵

由这四个数（TN、FP、FN、TP）定义的常用指标：

$$\text{accuracy} = \frac{TN + TP}{TN + FP + FN + TP}, \quad \text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

- **准确率**：整体猜对的比例。**坑**：数据里含密多、干净少，模型全部判“含密”，准确率照样很高，但把所有干净图都冤枉了。
- **精确率**：被判成含密的人里，有多少是真的。
- **召回率**：真正的含密图，抓到了多少。
- **F1**：精确率和召回率的折中。

指标	回答的问题	直觉
准确率	整体猜对多少	不平衡时骗人
精确率	判为含密里真含密	误报低 → 精确率高
召回率	真含密里抓到多少	漏报低 → 召回率高
F1	两者都要时	折中
ROC / AUC	排序能力强不强、跟阈值无关	0.5 等于瞎猜

7.7.1 ROC 曲线与 AUC：只看“能不能把含密排前面”

核心理想：一个好模型，应该把含密的图排在干净图前面。把阈值从高到低扫一遍，就能画出一条 **ROC 曲线**，横轴是“误报率”，纵轴是“检出率”。曲线越靠近左上角越好。

图 7-6（真实数据生成的 *OOF* ROC 曲线：AUC≈0.70。红色阴影区域就是 AUC——“随机抽一张含密图、一张干净图，模型把含密图排在前面”的概率。红色圆点 = Youden 最佳操作点）

AUC 就是曲线下面积：0.5 是瞎猜（对角虚线），1.0 是完美排序。0.7 意味着“多数时候能把含密排在干净前面”，但还不够好——这正对应弱密度难检出的现实。

关键点 |

AUC 与阈值无关！不管你把阈值定在 0.5 还是 0.7，AUC 都不变。因为 AUC 只关心“排序”，不关心“具体在哪个点切”。这在下图（7-7）能直接看出来。

7.7.2 阈值怎么选：ROC 上的“操作点”

虽然 AUC 与阈值无关，但**真正部署时**你必须选一个阈值。选高 → 保守（误报少、漏报多）；选低 → 激进（检出多、误报多）。看下图就一目了然。

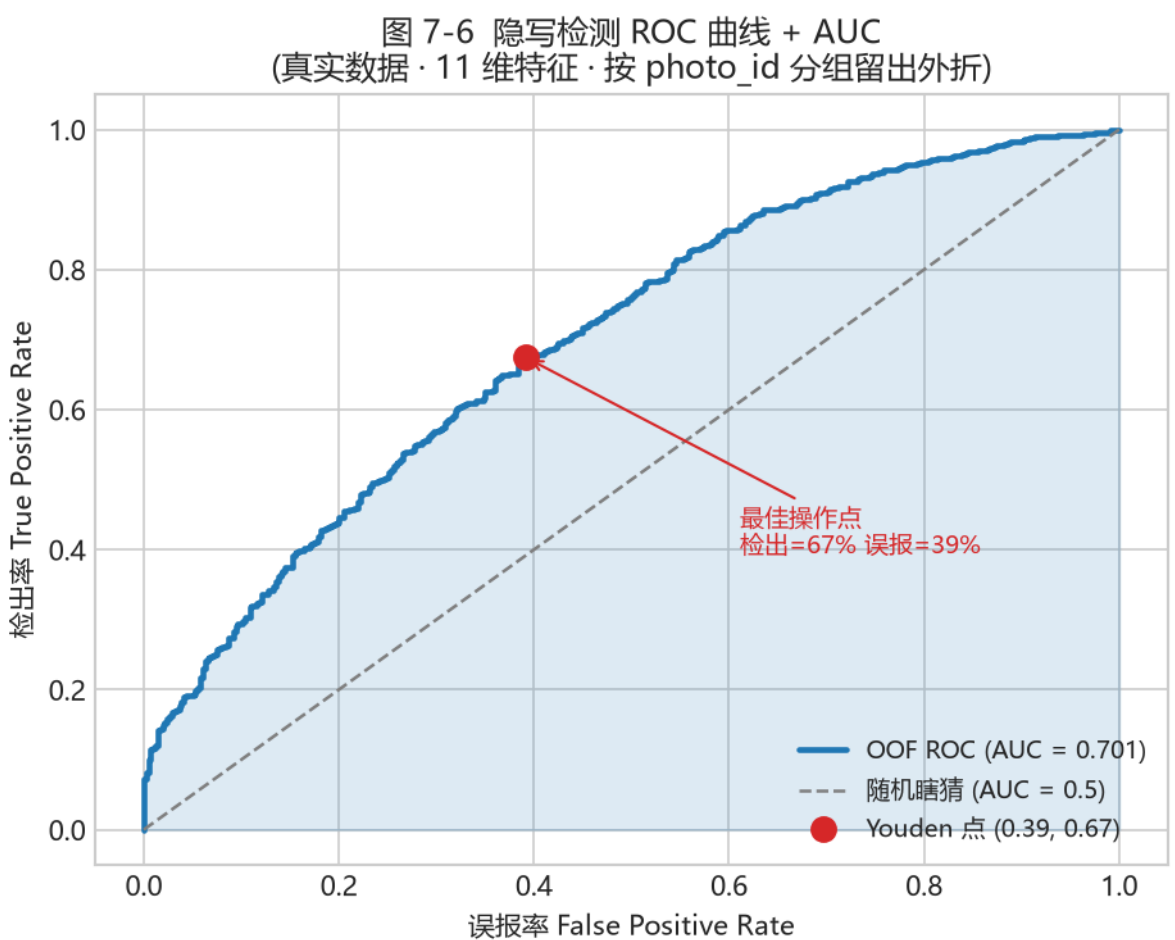


图 21: 图 7-6 ROC + AUC

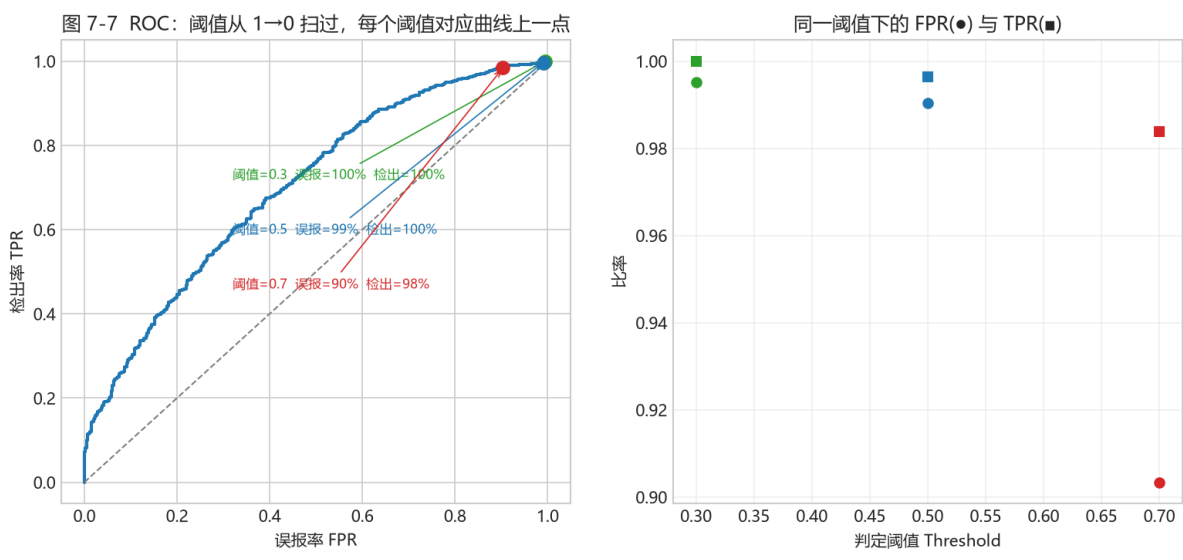


图 22: 图 7-7 ROC 操作点

图 7-7（同一组测试，左图：阈值分别取 0.3/0.5/0.7 时在 ROC 上的三个点；右图：同一阈值下“检出率 $TPR()$ ”和“误报率 $FPR()$ ”怎么变。阈值越低，检出越高但误报也越高——这就是一对不可避免的矛盾）

回到项目代码 |

train_model.py 里用 Youden 准则自动选阈值，就是找“检出率 - 误报率”最大的那个点（离随机线最远的点），再另算一个“低误报”阈值（误报率 $\leq 10\%$ ）供严格档使用：

```
from sklearn.metrics import roc_curve
fpr, tpr, th = roc_curve(y_true, proba)
j = tpr - fpr                # Youden J = 检出率 - 误报率
best = float(th[np.argmax(j)]) # 让 J 最大的阈值，就是离随机线最远的点
```

7.7.3 为什么“准确率”在这里会骗人

想象 2898 个样本中 2484 个是含密：只要模型**全部判含密**，准确率就是 $2484/2898 \approx 85.7\%$ ，看起来很高，但它是把 414 张干净图**全误报**了。所以准确率在类别不平衡时没意义，要看 AUC、召回率、F1 这类指标。

我特别画了一张图，把“阈值怎么影响各项指标”完整展示出来，请看：

图 7-8（真实数据：随阈值从 1 降到 0，精确率（绿）、召回率（蓝）、误报率（红）的变化。你选的阈值，就是一个“操作点”——在这里做取舍）

读图要点：

- 阈值很低时：召回率高（几乎都抓到），但精确率低、误报率高（误杀很多干净图）；
- 阈值很高时：精确率高（判为含密的基本都是真的），但召回率低（漏掉很多）；
- 中间某处，你会选一个符合自己需求的“折中点”。

动手做 |

运行一次 `python src\train_model.py`。不用看懂全部，先找三行：CV-AUC、测试集 AUC、低误报点检出率。第 8 章会逐行解释。

7.8 进阶：项目还做了两个“加分项”（看得懂就行）

train_model.py 里还有两件事，第一次学知道个大概即可：

- **概率校准**：逻辑回归常“过分自信”（该 0.3 的说成 0.95）。项目加了一层校准，让概率更接近真实比例，这样定阈值才有意义。

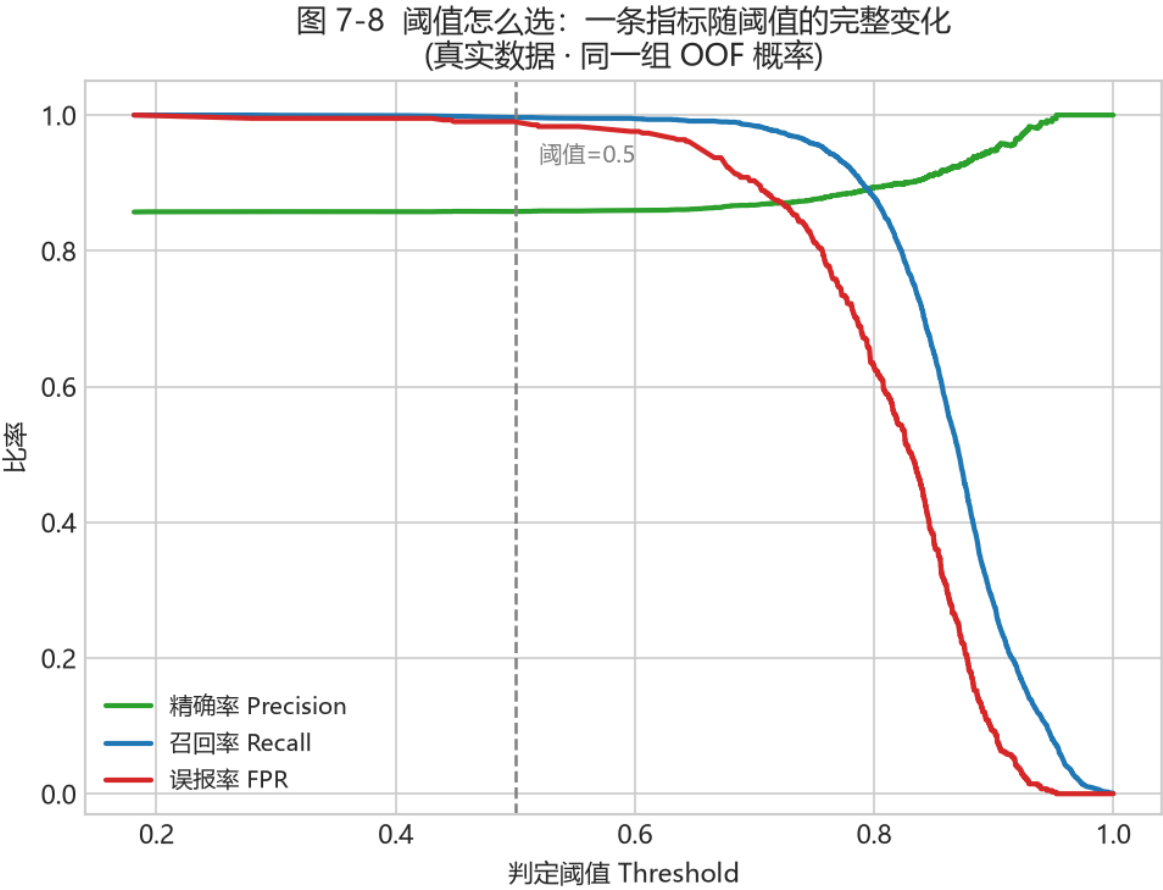


图 23: 图 7-8 指标随阈值变化

- **堆叠 (stacking)**：它不用一个模型打天下，而是把“逻辑回归、随机森林、梯度提升、XGBoost”这 4 个模型的答案都拿出来，再让一个小模型（逻辑回归）学“该听谁的”。

新手提示 |

堆叠有点像“评委团”：每个评委（基模型）先独立打分，再由一个“组长”（meta 学习器）综合定结论。它不挑单个最好，而是把几种思路的优势合并。第 8 章的图 8-4 会把这四个模型横向比一比，你就知道为什么它们能“互补”了。

7.9 小结与自我检查

一句话总结本章：机器学习 = 用「数据 → 特征 → 模型 → 评估」，把“一张图藏没藏东西”变成一个可判断、可评估、可解释的过程。

- 监督学习 = 用“带答案的样本”学一个从特征到答案的判断；
- 分类 = 用一条分界线把两类分开，逻辑回归 = 画线 + sigmoid 输出概率（图 7-1、7-2）；
- 损失 = 猜错扣多少分；训练 = 让损失变小；梯度下降 = 一点点朝下走（图 7-3）；
- 同源样本别随便切分，GroupKFold 按 photo_id 分组才是诚实的（图 7-4）；
- 准确率在不平衡时失真，看 AUC / 精确率 / 召回率；阈值决定“严一点还是松一点”（图 7-5 7-8）。

想一想 |

如果一篇论文说“隐写检测 AUC=0.95”，但没说数据是怎么切的，你会先怀疑什么？写下来——这就是你评估一切机器学习实验的起点。再想深一层：它有没有做概率校准？它的阈值是怎么选的？这些细节会不会也在“挑”一个好看的结果？

动手做 |

把图 7-6、7-8 看懂后，试着回答：如果领导要求“误报必须低于 5%”，你应该把阈值往高还是往低调？代价是什么？

第 8 章 · 用机器学习做隐写检测 (第 8–14 周)

动手做 |

本章目标：知道**为什么用“特征”而不是直接看像素**；认识 11 个特征各自在“量什么”（并配上真实分布图）；看懂 4 种分类器、4 个真实 ROC；跑通“生成数据 → 训练 → 判定”全流程。第 8.8 讲项目 v1.4 的完整演进。

8.0 本章路线图 (约 4–8 周)

1. **第 1 步 (约 1 周)**：8.1, 理解“为什么用特征而不是裸 CNN”——这是本书**最重要的一步**，请务必想通。
2. **第 2 步 (约 2 周)**：8.2, 逐个认识 11 个特征。配合图 8-1/8-2 的分布，对照公式和 `steganalysis.py`。
3. **第 3 步 (约 2 周)**：8.38.4, 看懂“数据怎么来 + 训练脚本怎么读”。配合图 8-3/8-4。
4. **第 4 步 (约 2 周)**：8.58.7, 学会读 ROC/检出率，跑一遍完整流程。
5. **第 5 步 (进阶, 约 1 周)**：8.8 的 v1.4 演进 (SRM、143 维、双版本)，这是“广、全、实”里最接近论文的部分。

8.1 一个反直觉的问题：为啥不直接让 AI “看图”？

你都听说过深度学习能看图识别猫狗，那为什么不直接把像素喂给神经网络、让它自己学“藏没藏东西”呢？项目试过，**验证 $AUC \approx 0.50$ ，等于完全瞎猜**。原因很好懂：

- 隐写改的是**极微弱的高频信号** (LSB 那一层)，比照片里的内容、纹理弱太多，信噪比极低；
- 神经网络很容易学到“这张是风景、那张是人像”，而不是“藏没藏”；
- 真正独立的样本是**照片的张数** (几百张)，而深度模型往往需要成千上万张。

所以项目换了一条路：**用人的领域知识去“量”隐写留下的统计痕迹 (特征)，再让一个简单的分类器判断**。v1 用 11 个特征，小样本时 AUC 约 0.750.79；v1.4 升级到 143 维 + LightGBM 后到 0.90+ (见 8.8)，但“特征法好过裸 CNN”的结论没变。

新手提示 |

这不是“深度学习没用”，而是一堂“在数据少的时候怎么选方法”的课：先用可解释的强特征验证信号到底存不存在，再考虑更复杂的模型。不少工程问题都是这个次序。

8.2 认识 11 个特征：它们在“量”什么

v1 的 11 个特征由 C++ 库 `cpp/fsfeatures.dll` 计算 (Python 封装在 `src/fsfeatures.py`) ; v2 的 143 维在 `src/featurize_v2.py` 组装。先记一个总印象：这 11 个数，就是 11 把尺子，专门量“LSB 平面是不是被搞乱过”。量什么可以分三组：

特征	量什么	藏过东西后会怎样
Rm / Sm / Gr	正掩码下“常规/奇异”组数与缺口	Gr 下降
Rn / Sn / Gn	负掩码下“常规/奇异”组数与缺口	Gn 明显塌缩 (核心信号)
chi2_stat / chi2_pvalue	灰度对是否被“抹均匀”	灰度对趋匀, p 上升
diff_entropy	图像本底粗糙度	用来认出天然噪声图
lsb_diff_entropy	LSB 位面乱不乱	随机化后趋近 1
median_prefix_p	20 段前缀卡方 p 的中位数	整体乱时升高

下面把三组“尺子”的本事讲清楚。先看分布图，再讲公式——图比公式直观。

8.2.1 RS 分析：核心信号 Gn 怎么来的

RS (Regular-Singular) 先把像素流每 4 个分成一组 $g = (g_1, g_2, g_3, g_4)$ ，定义“这组有多光滑”，用组内相邻像素的差加起来：

$$f(g) = |g_1 - g_2| + |g_2 - g_3| + |g_3 - g_4|.$$

选一个掩码 $M = (0, 1, 1, 0)$ ，对组里某些位置做 LSB 翻转，得到新的光滑度 f_M ；取补得到负掩码。于是每个组可以分为：

- 常规 (Regular)：翻转后 $f_M > f$ (更“光滑规则”)；
- 奇异 (Singular)：翻转后 $f_M < f$ (更“乱”)。

最后算两类组数的差值占总数比例，这就是缺口：

$$G_r = \frac{R_M - S_M}{n}, \quad G_n = \frac{R_{-M} - S_{-M}}{n}.$$

图 8-1 (真实数据：干净图 (蓝) 和含密图 (红) 的 RS 缺口分布。左：Gn (负掩码) ——干净图集中 在偏高位置、含密图塌向低处，这就是最强的判别信号；右：Gr (正掩码) 也有区分，但不如 Gn 干净)

图 8-1 RS 缺口特征分布: 干净 vs 含密

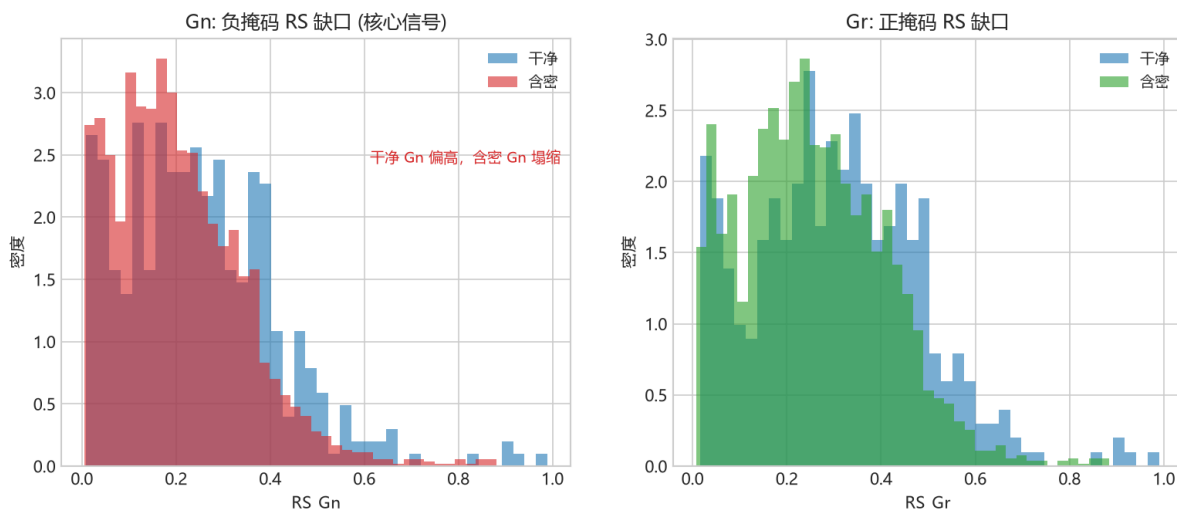


图 24: 图 8-1 RS 特征分布

大白话: 干净图 LSB 位面是有结构的 (相邻像素 LSB 往往相关), 负掩码一翻, 很多组会从“常规”变成“奇异”, 所以 G_n 明显为正 (干净图常在 $0.30.7$)。藏过东西后 LSB 被随机化, 正反掩码效果趋同, G_n 就塌下来接近 0。所以 G_n 塌缩 = 藏过的强力证据。

对照代码——`src/steganalysis.py::rs_metrics:`

```
gi = groups.astype(np.int16) # 一组 4 个像素
fg = np.abs(np.diff(gi, axis=1)).sum(axis=1) # f(g) = Σ|相邻差|
pos = ((gi ^ mask) & 0xFF) # 正掩码翻转后的组
fM = np.abs(np.diff(pos, axis=1)).sum(axis=1)
Rm = int(np.sum(fM > fg)); Sm = int(np.sum(fM < fg))
# 最后 {"Gr": (Rm - Sm) / n, "Gn": (Rn - Sn) / n}
```

8.2.2 卡方检验: 灰度对是不是“被抹匀了”

Westfeld 把灰度值两两配对 ($2i, 2i + 1$)。干净图里这两个灰度出现的频率往往不同; LSB 随机化会把它俩“拉平”。于是用卡方衡量“偏离均匀”的程度:

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}, \quad O_i = \text{观测到的偶灰度频率}, \quad E_i = \frac{f_{2i} + f_{2i+1}}{2}.$$

p 值由不完全伽马算出: $p = Q(df/2, \chi^2/2)$, 自由度 $df = (\text{有效灰度对数} - 1)$ 。

大白话: p 值越高, 说明观测和“完全均匀”的期望差异越小, 也就是这一对灰度已经被随机化/藏过了。

一个小细节 (看得懂就行)

: 代码里写的是 $(\text{even}-\text{odd})^2/(\text{even}+\text{odd})$, 正好是上面教科书式的 2 倍。因为只是把每个数乘个常数, 它比较“哪张图更可疑”的**排序不变**, 只是 p 值的刻度不同。

对照代码——`steganalysis.py::chi2_stats` 与 `_prefix20_p`:

```
even = counts[0::2]; odd = counts[1::2]
stat = float(np.sum((even[mask]-odd[mask])**2 / sums[mask])) # 卡方值
return stat, float(chi2_sf(stat, n-1)), n-1 # 卡方值, p, df
```

回到代码 |

`median_prefix_p` 把整图切成 20 段前缀, 每段算一个 p , 取中位数——这样单段的偶然波动就不影响判断。

8.2.3 熵: 图像“天然有多乱”

相邻像素绝对差值的**香农熵**, 度量图像本底的乱度:

$$H = - \sum_{d=0}^{255} p(d) \log_2 p(d), \quad p(d) = \frac{\#\{\text{相邻绝对差} = d\}}{\text{总对数}}.$$

- `diff_entropy`: 对灰度差分算熵。平滑/结构化的图低 (≈ 13), 高噪声/抖动的图接近 8。
- `lsb_diff_entropy`: 先只取 LSB 位面 (像素按位与 1), 再算相邻差熵, 范围 01。LSB 完全随机 $\rightarrow 1$, 明显有结构 $\rightarrow < 0.7$ 。

图 8-2 (真实数据: 左: 卡方统计量 (对数) ——干净图 (蓝) 更高 (奇偶更不齐), 含密图 (红) 偏低 (更“抹平”); 右: *LSB* 差分熵——含密图 (红) 偏向 1 (*LSB* 更随机), 干净图 (蓝) 偏低)

大白话: 这两个特征的作用是**认出“本来就乱”的图** (高噪声照片、抖动图), 别把它们误判成隐写——给“能不能可靠下结论”兜底。比如 `lsb_diff_entropy` 非常接近 1 的图, 可能是天然噪声, 也可能是藏过了, 需要结合其它特征交叉印证。

8.3 数据怎么来: `make_dataset.py` 的“1+6”设计

训练数据不能靠手工藏几条消息。`src/make_dataset.py` 对每张源照片生成 1 张干净图 + 6 个含密变体 (v1.4 支持 12 档, 见 8.8):

变体 (方法、 p 、密度)	强度	目的
------------------	----	----

clean	—	提供干净的负样本
nsF5, p=3, d=0.25	弱	贴近真实隐蔽通信的弱密度
nsF5, p=3, d=0.55	中	中等强度
nsF5, p=2, d=0.35	中弱	p 越小效率越低, 痕迹更强
nsF5, p=2, d=0.85	强	强嵌入, 足迹明显
matrix, p=3, d=0.50	中	对比另一种方法
matrix, p=2, d=0.80	强	普通矩阵编码的高密度
v1.4 再 +6 档	—	nsF5 p3 d0.40; matrix p3 d0.40/0.60; lsb d0.30/0.50/0.70

所有图统一缩放为 512×512 灰度; 特征由 C++ 提取, 含密变体由 C++ 嵌入路径生成, 保证训练与推理一致。

新手提示 |

注意右边的 “p、density” 就是 8.1 说的 “嵌入强度”: **档位越强, 藏得越多, 统计痕迹越明显, 分类器越好判。**这也解释了为什么后面 “弱密度更难检测” (见图 8-5)。

8.4 训练脚本怎么读: train_model.py

python src\train_model.py 是一条完整的 “诚实评估流水线”:

- 1. 读表, 删掉特征缺失的行;
- 2. 按 photo_id 预留 25% 作最终测试集 (这些照片绝不进训练池);
- 3. 训练池上做 5 折交叉验证, 比较**逻辑回归** / **随机森林** / **梯度提升** / XGBoost;
- 4. 选 CV 最好的模型, 在训练池内再切 20% 当校准集, 挑阈值;
- 5. 用全部训练池重训, 在留出的测试集上报 AUC / 准确率等;
- 6. 按 (方法、p、密度) 分组报每档检出率;
- 7. 保存模型与阈值到 models/stego_classifier.joblib。

8.4.1 四种分类器, 各是什么路数 (只讲思想 + 真实对比)

第 3 步比的是四个模型, 其实就三种思路:

- **逻辑回归 (LR)**: 第 7 章讲的 “画一条线 + sigmoid 输出概率”。**最直观、最稳、最好解释**; 缺点是线是一根直的, 不会拐弯。

图 8-2 卡方/熵特征分布: 干净 vs 含密

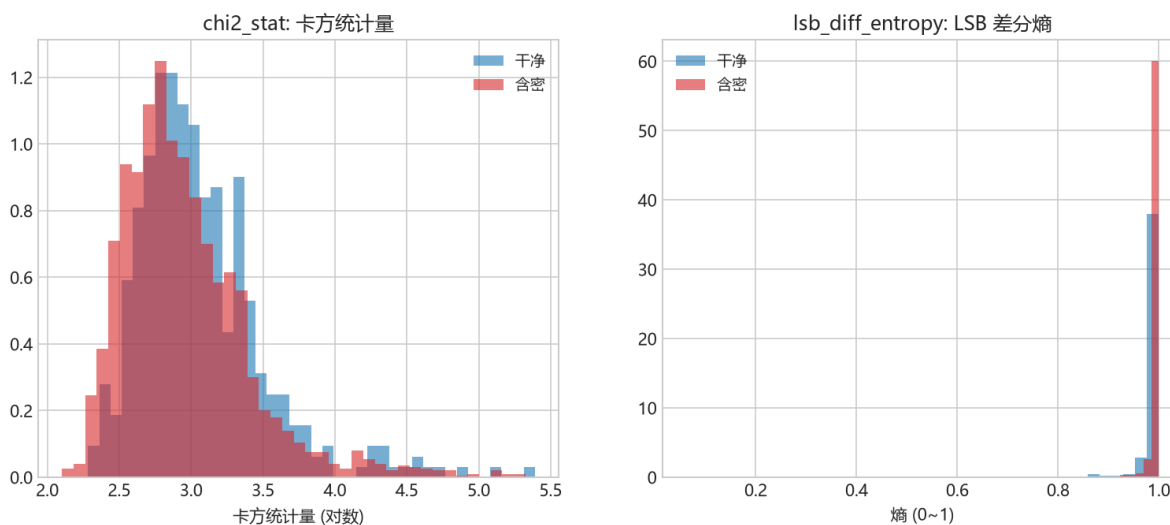


图 25: 图 8-2 卡方/熵特征分布

- **随机森林 (RF): 一群树投票。**每棵树都在问一连串“如果这个特征大于某数就分到左, 否则右”。多棵树分别用不同样本、不同特征训练, 最后少数服从多数。**稳、抗过拟合。**
- **梯度提升 / XGBoost (GB/XGB): 接力纠错。**不是同时种很多树, 而是一棵接一棵, 后一棵专门去补前一棵没做对的地方。XGBoost 是它的“升级版”, 多了些防过拟合的规矩, 跑得快、效果常最好。

```
def lr(seed=0): return make_pipeline(StandardScaler(), LogisticRegression(
    max_iter=2000, C=0.1, random_state=seed))
def rf(seed=0): return RandomForestClassifier(n_estimators=500, max_depth=None,
    min_samples_split=2, n_jobs=-1, random_state=seed)
def gb(seed=0): return GradientBoostingClassifier(n_estimators=300,
    learning_rate=0.05, max_depth=3, random_state=seed)
def xg(seed=0): return XGBClassifier(n_estimators=400, learning_rate=0.05,
    max_depth=4,
    subsample=0.9, colsample_bytree=0.8, eval_metric="logloss",
    random_state=seed, n_jobs=-1)
```

图 8-4 (真实数据 · 11 维特征 · 5 折 GroupKFold OOF: 四个分类器的 ROC 几乎叠在一起, AUC 都在 0.700.72 之间。这说明在这个 11 维特征上, 方法之间的差别不大——瓶颈更多在“特征”而非“分类器”。这反过来印证了 8.1 的思路: 先做对特征, 再挑分类器)

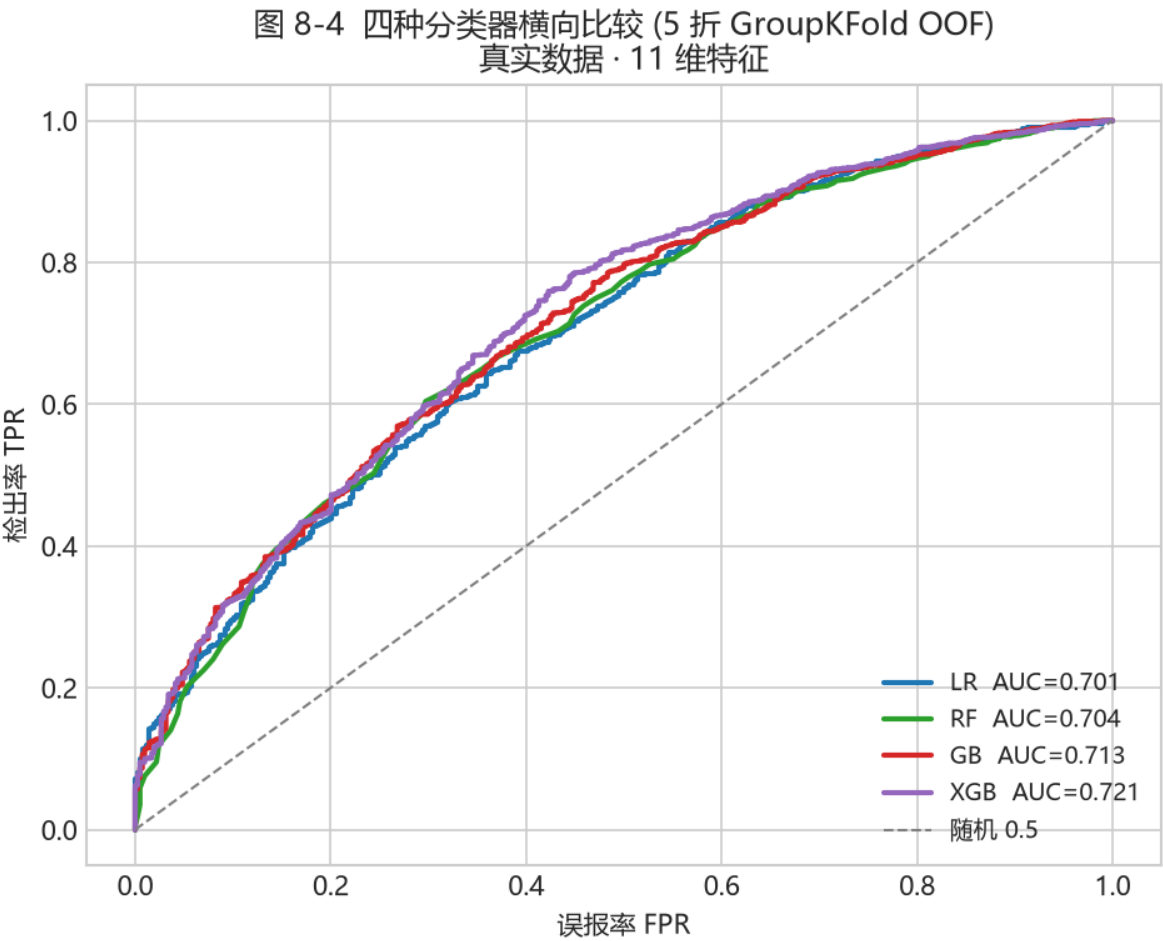


图 26: 图 8-4 四分类器 ROC 对比

想了解再点 |

树怎么“问”最合适：每次挑一个特征和一个阈值，把样本分成两堆，让每堆尽量“纯”。纯度常用**基尼不纯度** $G = 1 - \sum_c p_c^2$ 或信息熵衡量。`n_estimators=500` 指种 500 棵树；`learning_rate=0.05` 是“一步步别走太快”。公式不必背，知道“树找纯净的切分、XGBoost 更克制”即可。

8.4.2 哪个特征更重要：看看 LR 学到的系数

既然分类器差别不大，那**特征**才是关键。逻辑回归学出的权重 w 直接告诉我们每个特征“往判断里贡献了多少、方向如何”：

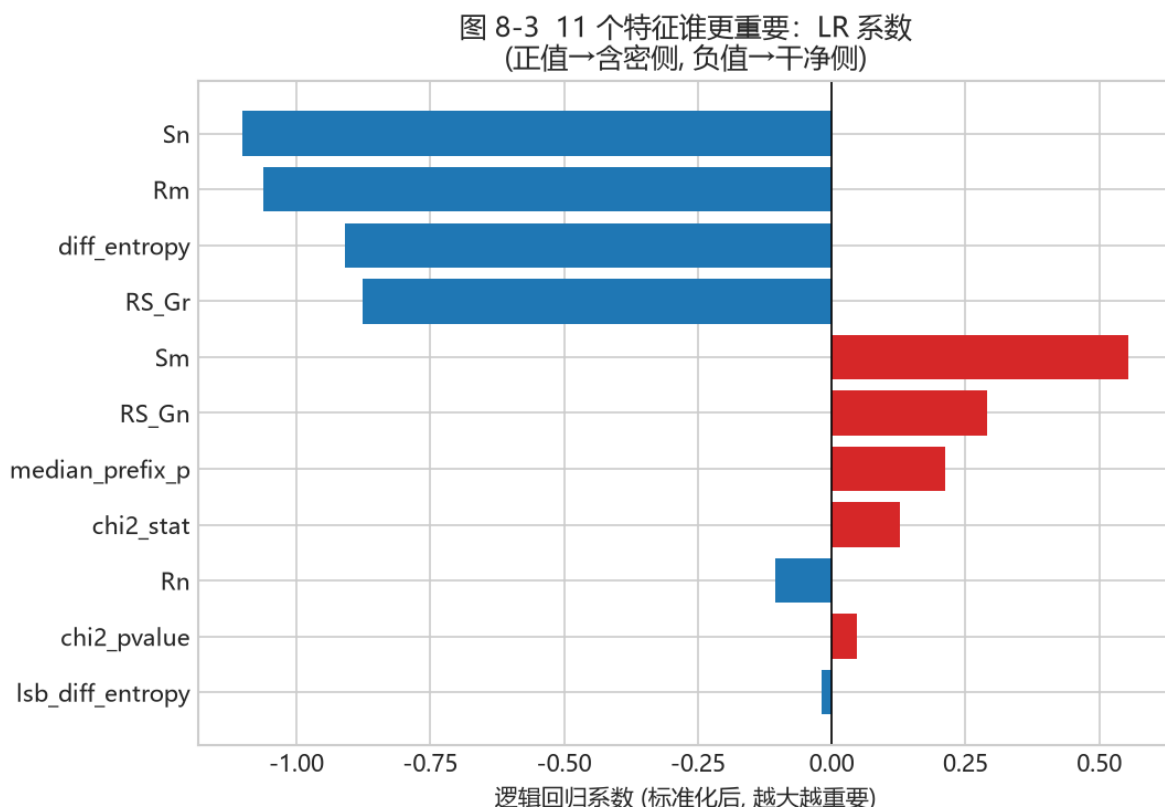


图 27: 图 8-3 特征重要度

图 8-3 (真实数据 · 11 维 · 标准化后 LR 系数: 正值 (红)→ 往“含密”方向推, 负值 (蓝)→ 往“干净”方向拉, 绝对值越大越重要。可以看到 *Sn*、*Rm*、*diff_entropy*、*RS_Gr* 贡献很大 (负), *Sm*、*RS_Gn*、*median_prefix_p*、*chi2_stat* 贡献较大 (正))

读图要点:

- 系数**绝对值大**的特征, 对判断影响大;
- 系数**为正**的特征, 值越大越像含密; **为负**则相反;
- 这正好呼应图 8-1: *Gn* 越大越像干净 (所以系数带一定方向)。

8.4.3 一个很关键的细节：OOF 与“别用自己考自己”

第 3 步的交叉验证不只是为了比较模型，还要得到一个干净的“模型可信度”数字。它的做法是：**每个样本只在“没见过它的那折”上被预测一次**，把这些预测收集起来（这叫 OOF, out-of-fold）。这样拿到的分数是“诚实的”，没有被“自己训练自己”污染。

避坑提醒 |

校准集用来挑阈值，测试集只许用一次。反复用测试集调阈值，测试集就“脏”了。项目用**训练/校准/测试**三层结构，是工业界认可的严谨做法。

8.5 怎么读结果：ROC、AUC 与按档检出率

综合评估时，除了整体 AUC (图 7-6)，还要看**每一档 (方法、p、密度) 的检出率**，因为不同档差异很大。

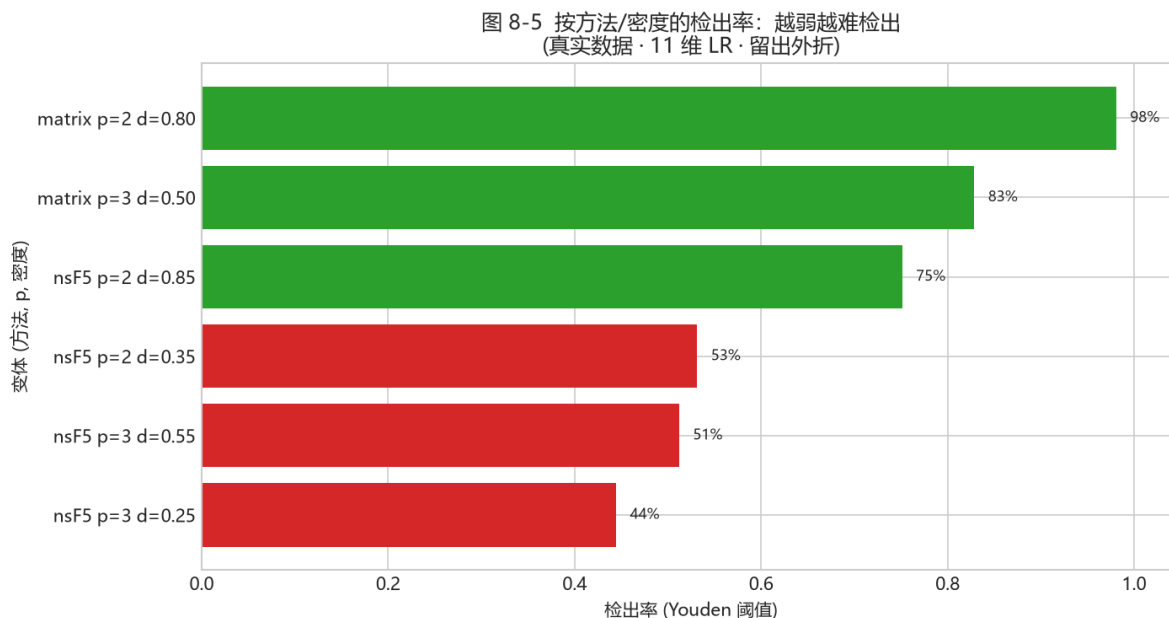


图 28: 图 8-5 按密度检出率

图 8-5 (真实数据 · 11 维 LR · 留出外折 · Youden 阈值：按 (方法、p、密度) 分组的检出率。可以看到**强密度 (matrix p2/p3) 接近 90%+**，**弱密度 (nsF5 p3 d0.25) 明显低 (约 50%+)**——这就是“嵌入越弱越难检测”的直接证据)

回到项目看代码 |

对比 thesis/data/det_density.csv: 弱档 nsF5 p3 d≈0.3 检出约 48%64%，强档 matrix p2/p3 接近 75%99%。这就印证了 8.3 的话——**小密度统计足迹弱，特征信号弱，所以更难检出，不是玄学。**

8.6 灵敏度：在 GUI 里怎么切换“松紧”

src/ml_predict.py 的 MLPredictor 预处理好图像 (转灰度 →512×512)，按模型记录的特征数自动走 11 维或 143 维，再输出含密概率。“灵敏度” 其实就是**换一个判定阈值**：

灵敏度	ML 阈值	效果
严格 (低误报)	threshold_low_fp	干净图误判最少，弱含密更易漏
均衡	Youden 阈值	检出与误报平衡，默认档
宽松 (高检出)	Youden×0.75	弱密度检出更多，误报略升

对照代码——ml_predict.py::_threshold:

```
if self.sensitivity.startswith("严格"): return float(d.get("threshold_low_fp", d["threshold"]))
if self.sensitivity.startswith("宽松"): return max(0.05, float(d["threshold"]) * 0.75)
return float(d["threshold"])
```

新手提示 |

“宽松” 并没有换模型，只是把**判定阈值下调 25%**。阈值越低越“激进”：藏得少的也判出来，但误报也变多——这就是图 7-7/7-8 说的“阈值 = 严一点还是松一点”，很直观。

GUI 的“分析” 页会把启发式概率和 ML 概率**并排显示**，要求你交叉印证、别只看一个数——这是项目刻意保留的“诚实” 设计。

8.7 动手实验：从训练到单图判定

完整链路 (v1 数据集；v2 与双版本命令见 8.8)

```
# 1) 生成数据集 (可换成自己的照片目录)
python src\make_dataset.py

# 2) 训练与评估
python src\train_model.py

# 3) 对单张图做 ML 判定
import sys; sys.path.insert(0, "src")
import numpy as np
from PIL import Image
from ns5_core import embed_string
from ml_predict import get_predictor
a = np.asarray(Image.open("img/cover.png").convert("L"))
```

```
s, _, _ = embed_string(a, "ML_test", method="nsF5", p=3)
print(get_predictor().predict(s))
```

动手做 |

做一个“诚实性小实验”：先对一张照片判定（应接近干净），再嵌入弱档消息（nsF5 p=3、密度 0.25 附近）重新判定。观察概率是不是**只微弱上升**——这正是 README 里“弱密度要交叉印证”的现场体会。

对照第 7 章公式 |

第 3 步 predict() 内部就是：特征 $x \rightarrow$ 加权求和 $z = w \cdot x + b \rightarrow$ sigmoid 变概率 $\rightarrow$ 和阈值比较得出判定。

8.8 v1.4.0 更新：SRM、143 维特征与双版本模型 (2026-09)

8.18.7 讲的是 v1.0~1.3 路线：11 维特征 + LR/XGB，AUC 约 0.750.79。v1.4.0 沿三条线升级：SRM 高通滤波预处理、143 维 v2 特征与 12 档变体、以及最终的 LightGBM 双版本模型。下面按演进顺序讲，帮你读懂 README 里的实验表。

8.8.1 SRM 高通滤波：同源有收益，跨源反而亏

SRM (Spatial Rich Model) 是一组**高通滤波核**。src/srm_filter.py 实现 30 个标准核，对图像逐核滤波后取最大绝对响应、clip 到 ± 4 再缩放成一张“增强图”，喂给原有特征器。CPU (numpy) 与 GPU (torch) 双实现，开关是 make_dataset.py --preprocess srm 和 GPU 的 use_srm=True。

指标	同源原图基线	同源 SRM 增强
5 折 CV-AUC	0.7594	0.7922
held-out 测试 AUC	0.7811	0.8085
弱档 nsF5 p3 d0.25 检出	51.9%	62.5%
nsF5 p2 d0.35 检出	76.0%	90.4%
低误报点含密检出	41.2%	50.3%
干净误报 (低误报阈值)	9.6%	9.6%

避坑提醒 |

SRM 的收益只出现在**同源**数据里。跨源合并 (校园 + BOSSbase 全量) 时，SRM 反而把 CPU 测试 AUC 从 0.741 拉到 0.704、GPU 验证 AUC 从 0.651 拉到 0.572——高通滤波在压掉图像内容的同时，也压平了不同相机/压缩源之间的差异。所以默认模型用**未 SRM** 的合并全量，SRM 单源校园模型另存为 campus_srm 专用。

8.8.2 v2 143 维特征与 12 档变体

src/featurize_v2.py 在 11 维基础上新增: 30 个 SRM 残差的均值/绝对均值/标准差 (90 维)、20 段整图前缀卡方 p、20 段 LSB 前缀卡方 p, 以及 texture_noise 与 est_rate (2 维), 合计 143。变体从 6 档扩到 12 档: 追加 nsF5 p3 d0.40、matrix p3 d0.40/0.60, 以及 lsb d0.30/0.50/0.70。

严格 A/B (同一测试集照片分组、5 折 GroupKFold OOF): v2 数据集本身带来 +0.019; 再加 143 维又 +0.027 (OOF 0.8143、held-out 0.8278), 累计约 +0.05 AUC。

新手提示 |

143 维不是凭空多出来的, 就是把 8.2 的三组“尺子”量得更细、从更多角度: BASE 11 是主干; SRM 90 是残差统计; PREFIX 20 + LSB-PREFIX 20 是卡方检验的“分 20 段 + LSB 版”。它们仍在测同一件事——统计足迹。

8.8.3 真实 JPEG 干净图暴露的部署问题与修复

v2 逻辑回归在训练分布内 AUC 很高, 但部署到真实 JPEG 干净照片时几乎全判 1.0: SRM 残差对 JPEG 高频噪声过于敏感。修复办法是把 414 张真实 JPEG 干净图作为独立 photo_id (与原训练集完全不相交) 加入训练, 得 dataset_campus_v2_jpeg.csv, 再做 LGB 网格调优。最终 LGB tuned 成为新默认: held-out AUC 0.8946、8-split 平均 0.9085, 弱档 nsF5 p3 d0.25 检出率从 67.9% 升到 85.3%。

新手提示 |

这是一个分布漂移的活例子: 模型在“校园 PGM/BMP 灰度”里学得好, 一到“真实 JPEG 相机噪声”就崩。项目用“把真实 JPEG 干净图加进训练”来纠正——训练数据必须覆盖真实部署场景, 不然分数再高也是自嗨。

8.8.4 双版本部署: 143d 稳健 vs 53d 可解释

可解释性实验发现: 143 维里约 73% 的增益来自 31 个可解释特征 (BASE 11 + PREFIX 20); SRM 90 维单个特征 AUC 平均只有 0.500.52 (接近随机), 但在 LGB 里起到“过滤 JPEG 噪声、提升 OOD 鲁棒性”的作用。于是保留两套模型:

维度	143d 默认 (稳健)	53d 可解释
特征构成	BASE11 + SRM90 + PREFIX20 + LSB-PREFIX20 + TEX/EST2	去 SRM 后的 53 维
8-split 平均 AUC	0.9085	0.9227
Held-out AUC	0.8946	0.9100
弱档 nsF5 p3 d0.25	85.3%	87.2%
真实 JPEG 干净 OOD	1/8 fp (median 0.011)	3/8 fp (median 0.028)
适用场景	通用部署 / 异构数据 / 真实图	论文 / 答辩 / 教学 / 单图可解释

MLPredictor() 默认加载 143d; 传 model_path 可切 53d。推理时按模型记录的特征数自动选 11 维或 143 维。GUI 的模型切换仍在规划, 当前通过 API:

v1.4.0 双版本模型切换示例

```
from ml_predict import MLPredictor

# 默认: 143d 稳健版 (models/stego_classifier.joblib)
pred_143 = MLPredictor()

# 53d 可解释版 (去 SRM 90 维, AUC 更高、可逐维解释)
pred_53 = MLPredictor(
    model_path='models/stego_classifier_v2_jpeg_lgb_51d.joblib',
    clip_outliers=False)

r = pred_53.predict(img)
# {'probability': ..., 'verdict': ..., 'threshold': ...}
```

动手做 |

实验任务: 用 \$env:DS_FILES = "dataset_campus_v2_jpeg.csv" 重新训练, 分别用 143d 与 53d 对 “真实 JPEG 干净图” 和 “弱档 nsF5 p3 d0.25 嵌图” 做判定, 把 AUC 与 OOD 行为写进实验报告。

避坑提醒 |

v2/53d 模型只在训练分布附近可靠。对新相机/压缩源, 先做小规模 OOD 测试再谈部署。

8.9 小结与自我检查

一句话总结本章: 在 “小样本 + 弱信号” 的约束下, **领域特征** + **简单分类器** 比裸 CNN 更可靠; 而 “特征” 的质量决定了上限, 分类器只是把特征里的信号挖出来。

- 11 个特征 = RS 家族 (Gn 塌缩) + 卡方家族 (灰度对被抹匀) + 熵/随机度基线 (认出天然乱图); 分布见图 8-1/8-2。
- 四个分类器 (LR/RF/GB/XGB) 在 11 维特征上差距不大 (图 8-4), 真正拉开差距的是**特征** (图 8-3) 与**数据**。
- **训练/校准/测试三层结构** + GroupKFold 是实验可信度的骨架;
- 越弱越难检测 (图 8-5), 检测能力有物理上限;

- ML 概率应与启发式判读**交叉印证**；v1.4 之后双版本模型很强，但真实部署前仍要做 JPEG OOD 验证 (8.8)。

想一想 |

同一张照片有 7 个样本，为什么不能用“随机 80/20 切分”？请设计一个最小例子，说明泄漏会让 AUC 虚高多少。再想一层：如果在 8.6 把“宽松”档阈值再下调，误报率会怎么变？为什么“检得更全”和“误报更少”总是一对矛盾？最后想想：如果给你 10 万张独立照片，你还会坚持“用特征而不是裸 CNN”吗？为什么？

第 9 章 · 工程化：C++、GPU、GUI 与测试（第 10 周）

动手做 |

本章目标：看懂“算法”如何变成“软件”；理解为什么需要 C++ 与 GPU 加速、如何保证跨语言一致；会运行测试与 CI 流程。

9.1 系统架构：分层而不耦合

系统总体架构（模块化分层）

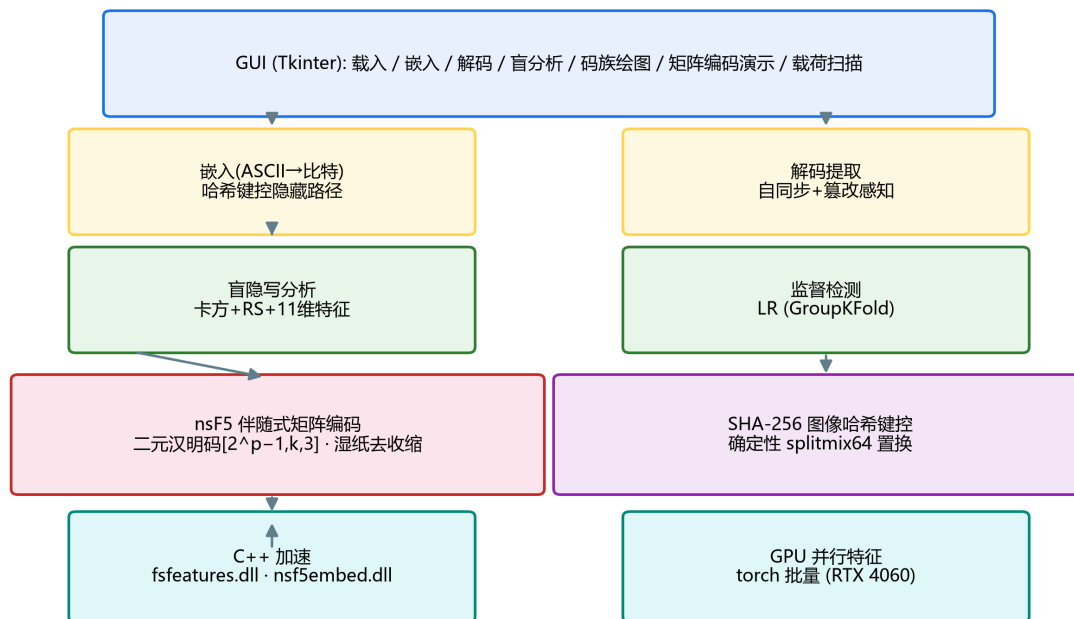


图 29: fig-5

图 9-1 系统总体架构：底层加速 → 算法安全核心 → 功能层 → GUI（论文图）

- **加速层**：cpp/fsfeatures.dll（特征）、cpp/nsf5embed.dll（嵌入/置乱）；

- **算法核心**：ns5_core.py——汉明码、湿纸、哈希键控、嵌入/解码；
- **功能层**：steganalysis.py、efficiency.py、make_dataset.py、train_model.py、ml_predict.py；
- **界面层**：gui.py + matrix_demo.py + scan_panel.py，所有功能一键可点。

分层的意义：算法层不依赖 GUI，可以在无界面环境（服务器/CI）里跑；加速层缺失时自动回退 Python，保证功能永远可用。

9.2 C++ 加速：为什么是“热路径”，以及如何保证没错

Python 逐元素循环极慢，但隐写有两个典型热路径：**确定性置换**（把 1600 万像素的顺序洗一遍）与**特征提取**（对每张图做 RS/卡方/熵统计）。把它们下沉到 C++ 动态库，性能可以提升几个数量级：置换加速最高约 **263 倍**（4096² 图：Python ≈12.3 秒 → C++ ≈223 毫秒）。

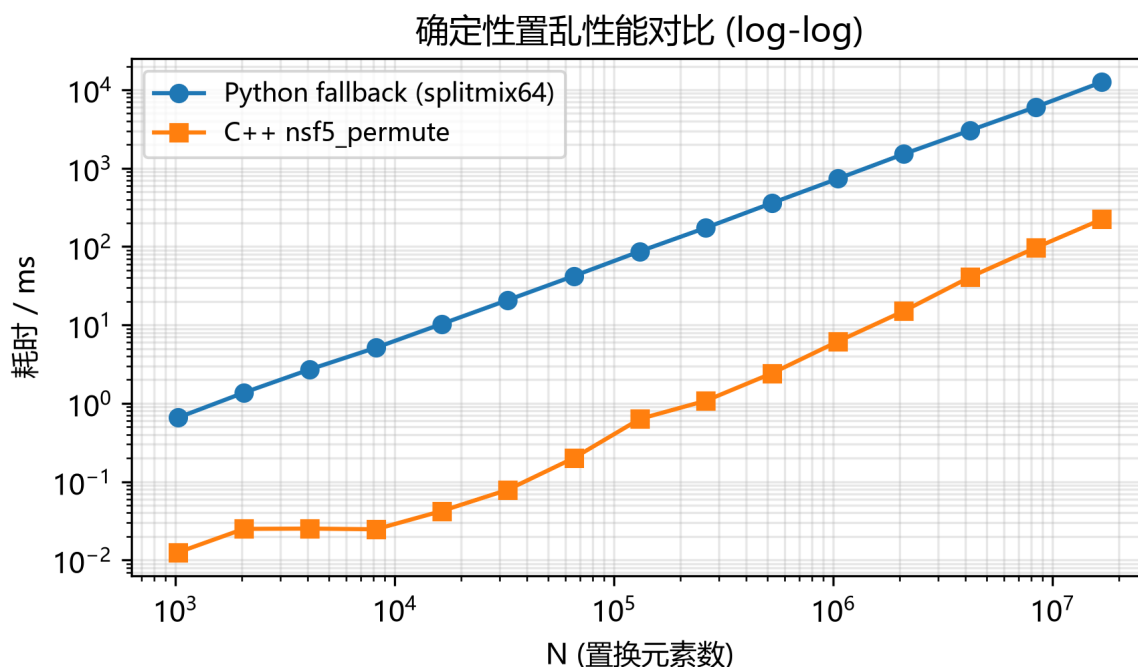


图 30: fig-6

图 9-2 确定性置换性能：Python vs C++ (log-log 坐标，论文图)

性能提升的前提是“**结果完全一样**”。Python 调 DLL 用的是 ctypes，项目为此设计了自校验：

- cppembed.py 用同一图像分别走 C++ 与 Python 路径嵌入，比较输出是否像素级一致；
- fsfeatures.py 的 check_against_python() 逐特征核对 C++ 与 Python 结果；
- DLL 缺失或加载失败时自动回退 Python 同算法，保证嵌入/解码两端序列恒定可逆。

避坑提醒 |

跨语言一致性是“可加速”的前提,也是调试的雷区: C++ 端位运算、取整、溢出行为与 Python/NumPy 不完全一致。遇到“有 DLL 能嵌、无 DLL 解不了”的诡异 bug, 先跑 `cppembed.selfcheck()` 和 `fsfeatures.check_against_python()`, 而不是去改算法。

9.3 GPU 版: 把 11 维特征“批量向量化”

`gpu/` 目录用 PyTorch 把特征计算改写成张量算子: 一次读入一批 512×512 灰度图, 在 GPU 上并行完成直方图、RS、卡方与熵计算, 再落回 CPU。v1.4 新增 `gpu/featurize_v2_gpu.py` 支持 143 维 v2 特征 (含 SRM 卷积)。README 实测 v1 特征 2070 张约 5 秒 (≈ 410 张/秒), 且与 CPU 参考实现逐位一致 (浮点误差 $1e-7$)。

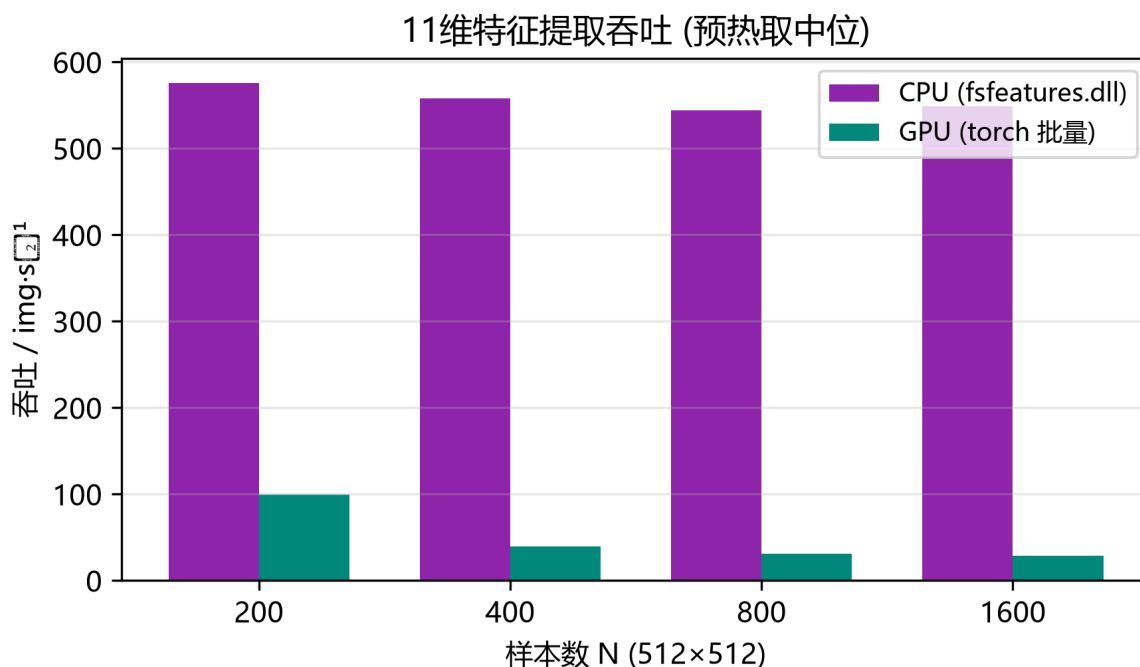


图 31: fig-7

图 9-3 v1 11 维特征提取吞吐: CPU(C++) vs GPU(torch 批量) (论文图)

回到项目看代码 |

读 `gpu/featurize_gpu.py` 的自检函数: 它把 GPU 结果与 `src/fsfeatures.py` 的 CPU 结果逐元素比较。bit 级一致是这套双实现能并存的前提。再读 README 关于吞吐边界的讨论: 小批量时主机 设备拷贝开销可能让 GPU 反而不如 C++——这是诚实评测, 不是“GPU 一定更快”的营销话术。

9.4 GUI：把研究工具变成可教学的应用

src/gui.py 的主界面围绕 “嵌入 → 解码 → 分析” 三步设计。真正出彩的是两个教学面板：

- **矩阵编码演示** (matrix_demo.py)：随机/手点一个块，实时计算 s、m、d，高亮命中的列与被改系数，直观展示 “至多改 1 位”；
- **载荷扫描** (scan_panel.py)：拖动 payload 0→0.4，逐档重新嵌入并刷新卡方 p 值、RS 估计率与 ML 概率三条曲线，观察 “藏得越满越容易被发现”。

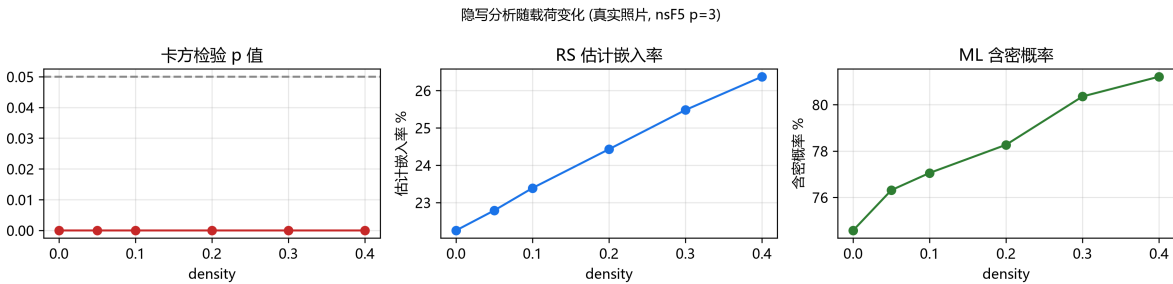


图 32: fig-8

图 9-4 载荷扫描：统计可检测性随嵌入密度变化（论文图）

避坑提醒 |

注意 GUI 文案里的严谨措辞：单张图没有 “AUC”（AUC 需要一组正负样本），所以扫描面板用的是模型概率作为趋势示意。阅读软件输出时，要区分 “单图概率” 与 “数据集指标”，这是容易被误用的一处。

9.5 测试与 CI：让重构不翻车

测试文件	验证内容	心智模型
test_core.py	汉明矩阵正确性、嵌入/解码往返、湿纸、口令错误	算法没坏
test_steg.py	盲分析能区分干净图与含密图	分析没坏
test_false_positive.py	大量干净图不误报（回归）	误报没失控
test_gui.py	窗口能构建、载入与预览	界面没坏

.github/workflows/ci.yml 会在每次推送到 main 或提 PR 时自动跑核心测试、构建 wheel 与 sdist；推送 v* 标签时自动发 GitHub Release。这套 “测试 + 打包 + 自动发布” 是算法项目走向可复用工具的标准姿势。

9.6 代码阅读路线图（按需选用）

- 1. 入口：src/run_e2e.py（最短路径，串起所有模块）；
- 1. 数据层：image_io.py → 数组怎么进出的；
- 1. 嵌入层：ns5_core.py 从上到下读一遍（哈希、置换、汉明、湿纸、高层 API）；
- 1. 分析层：steganalysis.py 的 analyze() 逆向追踪每个统计量；
- 1. ML 层：train_model.py 主流程 → featurize_v2.py / srm_filter.py（v1.4）→ ml_predict.py 双版本推理；
- 1. 加速层：cppembed.py / fsfeatures.py 的 ctypes 与自校验；
- 1. 界面层：gui.py 找按钮回调，顺藤摸瓜到算法函数。

想一想 |

为什么 ns5_core.py 的注释反复强调“改了置换算法会破坏可逆性”？请结合 v1.2.1 修复（DLL 缺失自动回退）想：如果 Python 与 C++ 排列不一致，会出现什么灾难性现象？

9.7 v1.4.0 更新：SRM、多源数据与模型工程（2026-09）

v1.4.0 新增/改动了若干工程模块，阅读新代码时先认目录：

新文件 / 模块	职责	学习要点
src/srm_filter.py	30 个标准 SRM 高通核， numpy/torch 双实现	核清单、归一化、增强图生成
src/featurize_v2.py	143 维 v2 特征组装与 CPU 实现	ALL_FEATURE_NAMES、 featurize_v2()
gpu/featurize_v2_gpu.py	GPU 批量 143 维特征与一致性自检	extract_features_v2_gpu()
src/make_dataset.py 扩展	多进程 -j、SRM/feature-set/variants	按图并行、输出逐行一致
src/train_model.py 扩展	DS_FILES 多数据集、Stacking、 LGB 调优	双版本模型保存与阈值
gpu/make_imageset.py 扩展	memmap 逐张落盘、-out/-id-offset	多源分工、避免 OOM

数据侧：项目建立了纯校园照片基准 data/campus_jpg（414 张，移除早期 DIP4E 教材图），并引入隐写分析事实标准 BOSSbase 1.01（1 万张 512² 灰度 PGM）做多源合并。CPU 11 维合并训练 72898 样本，held-out AUC≈0.741；GPU 合并 52070 样本，验证 AUC≈0.712；BOSSbase 单独跑 GPU 只有 AUC≈0.644——基准越难，弱密度信号越弱。make_dataset.py 现在支持多进程（16 核约 5.6 倍加速），GPU 图像集用 memmap 逐张落盘避免大数组 OOM。

工程配套：许可证从 MIT 改为 Apache-2.0（新增 NOTICE 与第三方声明），README 与 pyproject 的版本/主页同步为 v1.4.0；C++ DLL 与 Python 的像素级/特征级一致性自检仍然保留。

动手做 |

跑一次 `python gpu\featurize_v2_gpu.py` 自检；然后按 README 用 `data\campus.jpg` 与 `data\BOSSbase_1.01` 各生成一源数据，对比“单源校园 / BOSSbase 单源 / 两源合并”三组模型的 AUC。这是 v1.4 最值得亲手复现的“数据域”实验。

第 10 章 · 综合实践：从“读懂”到“做出来”（第 11—12 周）

动手做 |

本章目标：用两周时间完成一个可演示的改进实验。下面给出三个由易到难的方向与验收清单，你也可以把它们组合起来。但在动手之前，先记住“怎么做实验才算数”的骨架（图 10-1）。

10.0 实验方法论：先学会“怎么算才算数”

一个实验结论靠不靠谱，不取决于你“多努力”，而取决于你有没有守住**评估的诚实性**。这正是第 7、8 章反复讲的骨架，综合实践时它决定你的一切结论：

图 10-1 诚实评估流水线：train_model.py 的真实步骤（也是你做综合实验的骨架）

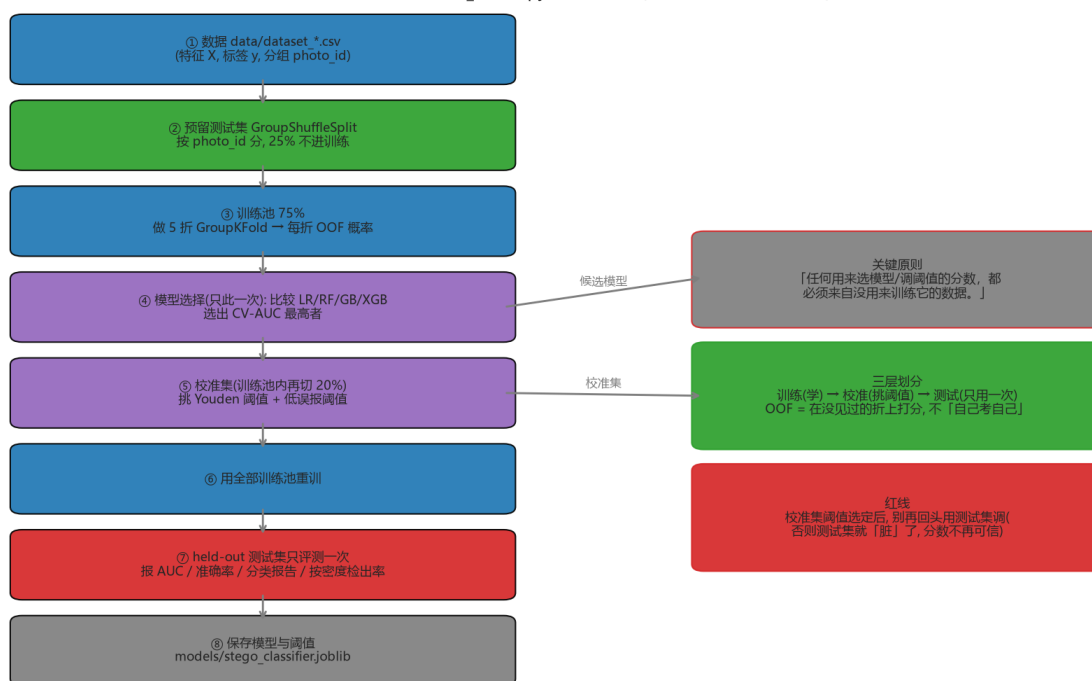


图 33: 图 10-1 诚实评估流水线

图 10-1 (`train_model.py` 的真实步骤——也是你做综合实验的骨架：数据 → 按 `photo_id` 预留测试集 → 训练池 5 折 `GroupKFold` 取 `OOF` → 只此一次选模型 → 校准集挑阈值 → 全量重训 → `held-out` 测试集只评一次 → 保存)

读图要点 |

- **测试集只许碰一次**：反复拿它调阈值，测试集就“脏”了，分数不再可信；- **校准集**：只在训练池内再切出，专门用来挑阈值（Youden / 低误报点）；- **OOF (out-of-fold)**：每个样本只在“没见过它的那一折”上被打分，用于选模型——这防止“用训练自己证明自己”。无论你选下面哪个方向，**都要回到这张图**：先跑通基线（图 10-1 全流程），再做小改动，用 `GroupKFold` 给出诚实对照。记住那句话：“**改了 A、结果变好**”不足以服人，要用**训练/校准/测试三层结构 + 分组切分证明它真的变好**。

10.1 方向 A：复现并批判性验证（最稳）

选论文/README 中的 1~2 个结论，自己重新跑并尝试“推翻”它：

- 复现码族图：理论 $\alpha(p)$ 是否与实测吻合？ p 取大后偏差从哪来？
- 复现检测实验：换你自己的照片训练，比较 v1 11 维基线（约 0.75–0.79）与 README 的 v1.4 双版本结果，观察照片风格对 AUC 的影响；
- 复现篡改实验：改 1 像素、改 1 行、改 1 块，解码失败率各是多少？

避坑提醒 |

复现 ≠ 照抄。复现的意义是**理解每个数字从哪来**：AUC 是 OOF 还是 held-out？用没用 `photo_id` 分组？阈值怎么定的？这些问题（见第 7 章）决定了你能否“批判性验证”而不是“盲信论文”。

验收项	通过标准
实验日志	记录环境、命令、参数、原始输出
图表	至少 2 张自绘曲线/对比图，坐标与图例完整
结论	能区分“验证了论文”与“与论文不一致”并给原因
可复现	提供一键脚本或逐条命令，别人能重跑

10.2 方向 B：功能扩展（最有成就感）

- **支持中文/UTF-8 消息**：把 `encode_string()` 的 ASCII 编码改为 UTF-8（注意长度头现在是字节数而非字符数）；
- **彩色通道扩展**：目前仅用 R 通道，可研究 R/G/B 三通道如何分配容量与安全性；
- **新增特征**：在 v1 11 维或 v2 143 维基础上加一个你认为能区分 clean/stego 的统计量，用 `GroupKFold` 评估 AUC 是否真的上升；

- **对比新算法**：实现朴素 LSB 替换，与 nsF5 在相同载荷下比较 G_n 塌缩与检出率。

避坑提醒 |

每次扩展都要跑 `test_core.py` 与 `test_steg.py`，保证“嵌入 → 解码”往返仍然成立。修改编码方案时，解码端必须同步修改——这是初学者最常忘的。

提示 |

“新增特征”这类扩展最容易“自我感觉良好”：加了一个特征、AUC 微涨就以为赢了。请务必回到图 10-1——用同一组测试照片、5 折 GroupKFold OOF 做 A/B，否则你的“+0.01”可能只是随机波动（这正是 7.8 讲的数据泄漏/泛化的实操）。

10.3 方向 C：教学演示再包装

把 GUI 里的矩阵编码演示扩展成一套可讲解的教材页/动画脚本，例如：

- 逐步动画：随机块 → 显示 H → 算 s → 给 m → 求 d → 高亮 → 校验；
- 题库模式：随机出题让你手算“该翻转哪一位”，自动判分；
- 对比模式：同一消息分别用 LSB、matrix p3、nsF5 p3 嵌入，并排显示改动数与 G_n 。

10.4 论文与代码对照阅读表

论文章节	对应代码	读它之前先掌握
第 2 章技术基础	<code>ns5_core.py</code> / <code>steganalysis.py</code>	LSB、汉明码、湿纸、卡方/RS
第 3 章需求分析	README 功能总览	隐写与隐写分析博弈观
第 4 章总体设计	arch 图 + <code>ns5_core.py</code>	分层架构、哈希键控流程
第 4.5 节加速	<code>cppembed.py</code> / <code>fsfeatures.py</code>	ctypes 与一致性自校验
第 4.6 节 GPU	<code>gpu/*.py</code>	张量化、批量、吞吐边界
第 5 章实验	<code>run_e2e</code> / <code>make_dataset</code> / <code>train_model</code>	GroupKFold、AUC、Youden

提示：论文草稿已更新为 `thesis/thesis1.pdf` (v1.4.0 实验，含 SRM/143d/53d 与多源数据)，上表以早期论文章节为例，阅读时按新论文目录对照。

10.5 汇报/答辩常见问题与应答要点

高频问题	应答要点
------	------

LSB 隐写为什么能被发现？	相邻灰度对被拉平（卡方）+ LSB 结构塌缩（RS），先讲直觉再讲统计量
矩阵编码为什么“改得少”？	$n=2^{-1}$ 个位置承载 p 位， H 列互异 $\rightarrow$ 任一伴随式差对应唯一位置
nsF5 相比 F5 好在哪里？	湿纸预标记湿点、只在干点解方程，无收缩无重嵌
湿纸编码怎么解方程？	$GF(2)$ 线性方程 $H_{dry} \cdot y = d$ ，先试单列/双列，再高斯消元兜底
为什么不用裸 CNN？	小样本弱信号下学不到泛化特征；特征 + 简单模型更稳（AUC 实证）
AUC 0.756 说明什么？	排序能力尚可但弱密度难检；AUC 与“具体阈值下检出率”是两回事
哈希键控能防什么？	防位置可预测/复制；能感知普通篡改，但不是密钥化 MAC
C++/GPU 加速有没有风险？	有：必须做跨语言一致性校验，否则嵌入解码会不可逆
“你的模型更好”怎么证明？	回到图 10-1：同测试照片、5 折 GroupKFold OOF、训练/校准/测试分离；报告单图概率 $\neq$ 数据集 AUC

10.6 交付清单（最后一周）

- 改进代码 diff（自己能讲清每处改动）；
- 实验脚本 + 原始输出（不加工、不挑数）；
- 2~3 张图表（效率/ROC/密度检出/篡改任选）；
- 3~5 页报告：问题 $\rightarrow$ 方法 $\rightarrow$ 结果 $\rightarrow$ 局限 $\rightarrow$ 下一步；
- 5 分钟演示：从嵌入到分析的一条龙运行；
- 能脱稿回答 10.5 表中的任意三个问题。

想一想 |

你的改进实验，打算采用图 10-1 的哪一条“诚实保证”来讲清“结果可信”？如果别人质疑“你是不是反复调测试集才得到这个结果”，你该怎么回应？（提示：把“测试集只用一次”这条红线讲清楚，并展示 OOF/校准集的结果。）

第 11 章 · 边界、伦理与继续前进（穿插学习）

11.1 法律与伦理边界

隐写是双刃剑：它支撑版权水印、媒体溯源、隐蔽认证等正当应用，也可能被用于恶意隐蔽通信。学习与研究的正确姿势是：

- 只在自有数据、公开数据集或经授权场景做实验；
- 把工具用于“检测与取证”而非“协助隐藏”；
- 发表结果时说明局限与误报风险，不夸大检测能力；
- 不把他人的隐私图像当作练习载体（照片数据要脱敏、授权）。

避坑提醒 |

这个项目的隐写分析部分（卡方/RS/ML）正是“盾”的用途：监管与取证需要它来发现可能的隐蔽信道。研究攻击面之前，先想清楚你的实验数据与使用目的。

11.2 项目已知局限（看懂边界才算真正读懂）

- 消息默认仅支持 ASCII，中文需 UTF-8 扩展；
- 彩色图只使用 R 通道，容量利用率低；
- 内容是哈希键控而非密钥化 MAC，无法防“重嵌并更新头部”的强攻击者；
- v1 数据集的独立源图仍有限（414 张校园 + BOSSbase 后扩到 1 万张）；弱密度与跨源仍是检出难点，任何新模型都要先做 OOD 测试；
- 启发式分析输出的是“隐写倾向概率”，不是严格统计意义上的真实概率；
- 像素域 nsF5 是有损格式的天敌——JPEG 域需要 yccstego 那样的 DCT 系数实现。
- v2/143 维模型对训练分布外的真实 JPEG 干净图曾“过激”，修复依赖把 JPEG 干净样本加入训练——任何模型都要先做 OOD 测试再部署；

- SRM 只在同源数据上提升检测，跨异构源反而下降，预处理收益有严格适用范围；
- 双版本模型各有取舍：143d 更稳（OOD 1/8 fp）、53d AUC 更高更可解释（OOD 3/8 fp），GUI 切换仍在规划，当前用 API 指定 model_path；
- 项目许可证已改为 Apache-2.0 并含 NOTICE，分发/引用时需保留第三方声明。

11.3 现代信息隐藏与隐写分析地图

方向	代表思想/方法	与本科生的距离
内容自适应嵌入	HUGO / WOW / UNIWARD：在纹理复杂处嵌入	需要较深最优化背景
深度学习隐写	编码器-解码器端到端训练隐藏模型	可做入门复现
深度学习隐写分析	SRNet / 大规模 CNN 检测	需要大算力与大数据
JPEG 域隐写	yccstego：Y 通道量化 DCT 系数 nsF5	本项目即可扩展
可逆/鲁棒水印	兼顾不可见性、鲁棒性与容量	工程性强，适合就业方向

如果你的兴趣被点燃，推荐的下一步：把本手册第 5 章“湿纸”读进原始论文 (Fridrich et al.)，再用 yccstego 对照学习 JPEG 量化域，最后挑一个现代算法做文献综述——附录 E 给了具体入口。

11.4 结语

12 周前你也许以为隐写就是“把字藏进图片”，现在你应该能讲出：它为什么依赖载体冗余、为什么直接改 LSB 会留下统计指纹、汉明码如何用最少的改动传递信息、湿纸编码如何让嵌入“无收缩”，以及机器学习如何诚实地说出“我有多确定”。这些知识串起来的，不只是 F:\Steganography 这一个项目，而是信息安全、概率统计与机器学习交汇处的一种思维方式。

动手做 |

最后做一个收尾练习：不看任何资料，用 500 字向一位同学介绍“nsF5 为什么比朴素 LSB 更好”。写完后打开 README 对照，查漏补缺——能写清，才算真正学完。

附录 A · 术语中英对照

按出现顺序整理，方便随时查阅。掌握粗体项即可覆盖本手册绝大部分内容。

中文	英文	一句话解释
隐写术	Steganography	把秘密信息藏进正常载体，隐藏“通信行为本身”
隐写分析	Steganalysis	通过统计/学习判断载体是否藏有秘密
载体 / 含密图	cover / stego	嵌入前的原始图像 / 嵌入后的图像
最低有效位	LSB	像素二进制的最低位，翻转后视觉几乎不变
位平面	bit plane	按二进制位拆出的二值图层，共 8 层
冗余	redundancy	载体中可被修改而不易察觉的部分
嵌入容量	capacity / payload	一张图最多能承载的秘密比特数
相对载荷	relative payload	嵌入比特数与可嵌位置数之比
嵌入效率	embedding efficiency	平均每次修改所携带的比特数 α
矩阵嵌入	matrix embedding	用分组编码在更少改动中嵌入更多位
二元汉明码	binary Hamming code	能纠正/定位 1 位错误的线性码 $[n,k,3]$
校验矩阵	parity-check matrix H	$p \times n$ 矩阵，列互异，用于算伴随式
伴随式	syndrome	$s = H \cdot x \pmod{2}$ ，块的“现状摘要”
GF(2)	Galois field of 2	只含 0/1、按异或运算的有限域
收缩	shrinkage	减幅时系数变 0 导致块作废的现象
湿纸编码	wet paper coding	湿点不动、只在干点解方程完成嵌入
干点 / 湿点	dry / wet positions	可修改的位置 / 碰不得的位置
F5 / nsF5	F5 / nsF5 algorithm	JPEG 矩阵嵌入；nsF5 用湿纸消除收缩
减幅	coefficient shrinking	让系数绝对值减小 1 的修改方式
SHA-256	SHA-256	输出 256 位摘要的密码学哈希函数
键控	keying	由口令/内容密钥决定隐藏位置等行为

确定性置换	deterministic permutation	同种子必得同排列的洗牌算法
splitmix64	splitmix64	项目使用的 64 位伪随机数生成器
Fisher-Yates	Fisher-Yates shuffle	等概率洗牌的标准算法
自同步	self-synchronization	解码端无需外部参数即能找到正文
篡改感知	tamper perception	图像被改动后能够被察觉
盲隐写分析	blind steganalysis	无原始载体时的统计检测
卡方检验	chi-square test	检验相邻灰度对是否被拉平
p 值	p-value	观测与假设吻合的概率（此处高 p 反而可疑）
RS 分析	RS analysis	通过正负掩码扰动统计规则/奇异组
常规/奇异组	regular / singular	扰动后判别值变大 / 变小的像素组
差分熵	difference entropy	相邻像素差分布的香农熵，衡量纹理随机度
香农熵	Shannon entropy	信息量/不确定度的度量，单位 bit
监督学习	supervised learning	用带标签样本训练模型
特征向量	feature vector	描述样本的数值列表（本项目 11 维）
标签	label	样本的正确答案（0= 干净，1= 含密）
二分类	binary classification	输出属于两类之一的预测
逻辑回归	logistic regression	线性加权 + sigmoid 的可解释分类器
sigmoid	sigmoid	把任意实数压到 0~1 的 S 形函数
交叉熵损失	cross-entropy loss	分类常用的损失函数
梯度下降	gradient descent	沿损失下降方向迭代更新参数
过拟合	overfitting	背下训练样本而失去泛化能力
交叉验证	cross-validation	轮流用不同折验证模型的流程
数据泄漏	data leakage	验证信息混入训练导致指标虚高
GroupKFold	GroupKFold	按组切分的交叉验证（同照片样本同组）
混淆矩阵	confusion matrix	真实 vs 预测的四格计数表
精确率/召回率	precision / recall	判对比例 / 抓全比例
ROC / AUC	ROC curve / AUC	全阈值下的检出-误报曲线及其面积
Youden 准则	Youden' s J	在 tpr-fpr 最大处选阈值
误报 / 漏报	false positive / negative	把干净判成含密 / 把含密判成干净
ctypes	ctypes	Python 调用 C/C++ 动态库的接口
DLL	dynamic-link library	Windows 动态链接库
CUDA / 批处理	CUDA / batching	GPU 并行计算 / 多样本同时处理
一致性校验	consistency check	C++/GPU 与 Python 结果逐位比对

ASCII / UTF-8	ASCII / UTF-8	英文字符编码 / 通用字符编码 (含中文)
空间富模型	SRM / Spatial Rich Model	一组高通滤波残差核, 用于突出嵌入噪声
残差图	residual map	高通滤波后保留的 “噪声残差”
LightGBM	LightGBM / LGB	梯度提升树实现 (v1.4 双版本模型使用的分类器)
特征组	feature group	同一起来源的一批特征 (BASE/SRM/PREFIX/LSB-PREFIX 等)
分布外	out-of-distribution (OOD)	与训练分布不同的输入 (如真实 JPEG 干净图)
BOSSbase	BOSSbase 1.01	隐写分析标准基准库: 1 万张 512^2 灰度 PGM
PGM	PGM	无损灰度图像格式 (BOSSbase 载体格式)
Stacking	stacking / meta-learner	用次级模型组合多个基模型预测的集成方法
内存映射	memmap	大数组分批落盘、按需读写, 避免一次性 OOM
校准集	calibration set	单独用于选择阈值的样本子集
网格调优	grid tuning	在超参数网格上搜索并比较模型
双版本模型	dual-version model	143d 稳健版 + 53d 可解释版并存部署策略

附录 B · 常用命令速查

以下命令均在项目根目录 F:\Steganography 下执行。

目的	命令
安装核心依赖	<code>pip install numpy pillow</code>
安装全部依赖 (ML/绘图)	<code>pip install -e .</code> 或 <code>pip install numpy pillow matplotlib scikit-learn joblib pandas</code>
核心算法自测	<code>python src\test_core.py</code>
隐写分析自测	<code>python src\test_steg.py</code>
误报回归测试	<code>python src\test_false_positive.py</code>
GUI 冒烟测试	<code>python src\test_gui.py</code>
端到端验证	<code>python src\run_e2e.py</code>
启动图形界面	<code>python src\gui.py</code>
生成码族/效率图	<code>python -c "import sys; sys.path.insert(0,'src'); from efficiency import plot_code_family_and_efficiency; print(plot_code_family_and_efficiency())"</code>
生成特征数据集	<code>python src\make_dataset.py data\campus.jpg -out campus</code>
训练/评估分类器	<code>\$env:DS_FILES = "dataset.csv"; python src\train_model.py</code>
单图 ML 判定	<code>python -c "import sys; sys.path.insert(0,'src'); from ml_predict import get_predictor; import numpy as np; from PIL import Image; print(get_predictor().predict(np.asarray(Image.open('img/cover.png'))))"</code>
C++/Python 一致性自检	<code>python -c "import sys; sys.path.insert(0,'src'); import cppembed; cppembed.selfcheck()"</code>
GPU 数据集生成	<code>python gpu\make_imageset.py</code>
GPU 特征自检	<code>python gpu\featurize_gpu.py</code>
GPU 训练	<code>python gpu\train_ml_gpu.py</code>
GPU 单图检测	<code>python gpu\predict_gpu.py img\cover.png</code>
构建发布包	<code>python -m build</code>

安装 wheel	<code>pip install dist\nsf5stego-1.4.0-py3-none-any.whl</code>
生成 v2 数据集 (12 档 / 143 维)	<code>python src\make_dataset.py -out campus_v2 -feature-set v2 -variants all</code>
生成 SRM 增强数据集 (同源)	<code>python src\make_dataset.py data\campus.jpg -out campus_srm -preprocess srm -j 16</code>
指定 v2+JPEG 数据集训练	<code>\$env:DS_FILES = "dataset_campus_v2_jpeg.csv"; python src\train_model.py</code>
GPU v2 特征自检	<code>python gpu\featurize_v2_gpu.py</code>
BOSSbase 多源生成 + GPU 训练	<code>py gpu\make_imageset.py data\BOSSbase_1.01 -out bossbase (再运行 python gpu\train_ml_gpu.py)</code>
切换 53d 可解释模型 (API)	<code>python -c "import sys; sys.path.insert(0,'src'); from ml_predict import MLPredictor; import numpy as np; from PIL import Image; print(MLPredictor(model_path='models/stego_classifier_v2_jpeg_ clip_outliers=False).predict(np.asarray(Image.open('img/cover.png'))</code>

附录 C · 12 周打卡检查表

每完成一项就在日期列写下当天日期。两周内完成一章不丢人，跳过动手实验才丢。

周	主题	必须完成的动作	日期
1	图像与二进制	运行位运算示例；说出 LSB 定义	
2	Python 环境	pip 装依赖；跑通 run_e2e.py；保存一张灰度图	
3	LSB 与盲分析	嵌入 → 分析实验；看懂 test_steg.py 的输出	
4	卡方/RS 原理	给同学讲清 Gn 塌缩；跑一遍 test_false_positive.py	
5	矩阵编码/F5	手算一个 p=3 例子；用 matrix_demo 验证	
6	nsF5/湿纸	读 ns5_core.py 湿纸段；跑 test_wet_paper_needed	
7	哈希键控	口令错误实验；篡改 1 像素实验；写出键控边界	
8	ML 基础	跑通 v1 训练；能解释特征/泄漏/GroupKFold/AUC	
9	ML 隐写检测	对比 11d/143d/53d 判定；解释 SRM 与双版本策略	
10	工程化	跑 cppembed.selfcheck 与 featurize_v2_gpu 自检；读 SRM/多源管线	
11	综合项目	选定方向并完成 70% 代码/实验	
12	汇报	完成报告与演示；脱稿回答 10.5 的三个问题	

附录 D · 概念 → 代码定位表

学习后期 “忘了某概念在哪个文件”，查这张表最快。

概念	文件	优先阅读内容
消息编码/长度头	src/ns5_core.py	encode_string / decode_string
图像哈希/种子	src/ns5_core.py	derive_seed / get_image_hash
确定性置换	src/ns5_core.py	permute_index
汉明码/伴随式	src/ns5_core.py	build_hamming / syndrome
矩阵嵌入	src/ns5_core.py	MatrixEmbedding._embed
湿纸求解	src/ns5_core.py	solve_wet_paper / gauss_solve_GF2
像素域 nsF5	src/ns5_core.py	nsF5Pixel._embed
高层嵌入/解码	src/ns5_core.py	embed_string / extract_string
卡方统计	src/steganalysis.py	chi2_stats / chi2_sf
RS 统计	src/steganalysis.py	rs_metrics
综合判读	src/steganalysis.py	analyze
特征顺序	src/fsfeatures.py / ml_predict.py	FEAT_KEYS
训练流水线	src/train_model.py	main()
数据集生成	src/make_dataset.py	main()
C++ 嵌入封装	src/cppembed.py	embed_string / selfcheck
GPU 特征	gpu/featurize_gpu.py	批量算子与 check_vs_cpu
GUI 回调	src/gui.py	按钮事件 → 核心函数
v2 特征计算	src/featurize_v2.py	ALL_FEATURE_NAMES / featurize_v2()
SRM 预处理	src/srm_filter.py	srm_residuals_np / preprocess_batch_torch
模型推理（双版本）	src/ml_predict.py	MLPredictor(model_path=..., clip_outliers=...)
GPU v2 特征	gpu/featurize_v2_gpu.py	extract_features_v2_gpu() 与自检

附录 E · 推荐资源

书籍（中文先入门）

- 《机器学习》（周志华，清华大学出版社）——第 2 章“模型评估与选择”务必精读；
- 《统计学习方法》（李航）——逻辑回归与感知机的经典推导；
- 《数字图像处理》（Gonzalez & Woods）——图像处理经典教材：位平面、直方图与频域基础（早期实验用过其中教材图，v1.4 数据源已改为纯校园照片与 BOSSbase）；
- 《深入浅出信息隐藏》类中文教材可作为扩展阅读。

核心论文（按阅读顺序）

- Westfeld, “F5—A Steganographic Algorithm” , 2001——F5 与矩阵嵌入出处；
- Westfeld & Pfitzmann, “Attacks on Steganographic Systems” , 1999——卡方检验；
- Fridrich, Goljan & Du, “Reliable Detection of LSB Steganography in Color and Grayscale Images” , 2001——RS 分析；
- Fridrich, Goljan, Lisonek & Soukal, “Writing on Wet Paper” , 2005——湿纸编码；
- Pevný, Filler & Bas, “Using High-Dimensional Image Statistics to Perform Unsigned Steganalysis” ——现代特征法（SPAM）入口。

在线资料与工具

- 项目 README 与论文草稿 thesis/thesis1.pdf (SRM/143d/53d 与多源实验见论文实验章) ;
GitHub: Yukinoshita-lin/nsf5-steganography;
- scikit-learn 官方文档: LogisticRegression、GroupKFold、roc_curve;
- PyTorch 官方教程 (GPU 版依赖);
- GitHub Actions 文档: 了解 CI 中如何跑测试与发版。

动手做 |

到这里，手册主体结束。回到导读 0.4 的能力清单，逐条给自己打分：全部能打勾，就给自己发一张“nsF5 入门结业证”吧。

附录 F · 6—12 个月深入自学路线图

动手做 |

导读 0.3 给的是 12 周“能入门”的压缩路线；这一节给的是 6—12 个月“能搞懂并做出成果”的深入路线。适合想真正吃透数字图像处理、信息隐藏与机器学习三大块、并动手做研究/工程的人。两条路线用的是同一批内容，只是时长的分配不同。

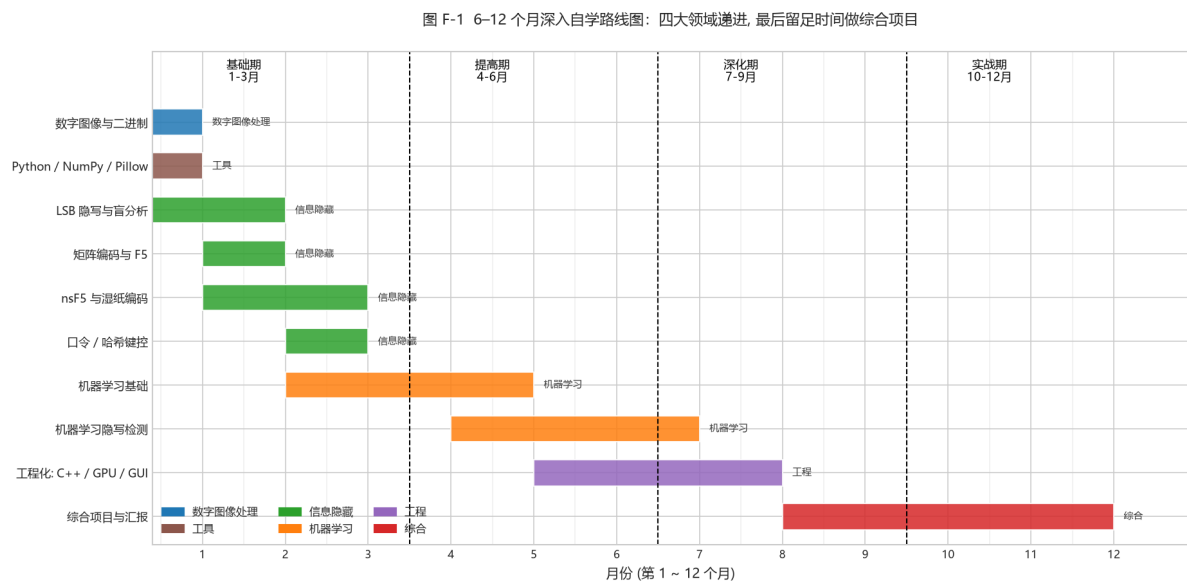


图 34: 图 F-1 路线图

图 F-1 (6—12 个月甘特图: 横轴 = 月份, 纵轴 = 主题, 颜色 = 领域; 虚线把全程分成基础期 (1—3) / 提高期 (4—6) / 深化期 (7—9) / 实战期 (10—12) 四段)

F.1 四段式：每一段的目标与产出

阶段	月份	目标	主攻内容	结束时你能做什么
----	----	----	------	----------

基础期	1–3	把三大领域的“地基”打实	ch01 数字图像与二进制、ch02 Python 工具链、ch03 LSB 隐写与盲分析、ch04 矩阵编码与 F5、ch05 nsF5 湿纸、ch06 哈希键控	能独立“藏一句话 → 用卡方/RS 抓到它”；手算一个 $p=3$ 汉明嵌入；讲清湿纸解码
提高期	4–6	进入机器学习，建立“特征 → 模型 → 评估”框架	ch07 机器学习基础、ch08 机器学习隐写检测（11 维 → 143 维 → 双版本）	能讲清过拟合/数据泄露/GroupKFold/AUC；看懂并复现 v1 11 维训练管线
深化期	7–9	吃透工程与“为什么这么设计”	ch08.8 SRM 与 143d/53d 策略、ch09 工程化（C++ / GPU / GUI / 多源数据）	读懂 SRM 滤波与 143 维特征；理解同源/跨源收益差异；跑通 GPU 批量特征与自校验
实战期	10–12	做一件“自己的”完整工作	ch10 综合项目、ch11 汇报；并选择一条研究或工程主线深入	完成一个可复现的改进实验并给出 诚实 结果分析；能脱稿答辩

新手提示 |

“基础期”几乎与 12 周路线一样，只是每周多花时间把每个**为什么**想透（尤其第 3 章的统计检测与第 5 章的湿纸）。真正的差距在“深化期”与“实战期”。

F.2 四条主线：贯穿“广、全、实”

想做出成果，别平均用力。选一条主线深挖，其余保持“看得懂”即可：

主线	聚焦	深度学习参考
算法主线	nsF5/F5、矩阵嵌入、湿纸、伴随式	ch04–05、项目 ns5_core.py；往外可延伸 JPEG 域与 DCT 系数隐写
检测 / ML 主线	特征工程、SRM、143d/53d、OOD	ch08、featurize_v2.py、train_model.py；往外可延伸现代高维特征与深度学习隐写分析
工程主线	C++ DLL、GPU 批量、GUI、多源数据集、CI	ch09、cppembed.py、gpu/*、.github/workflows；往外可延伸部署与鲁棒性
研究主线	复现论文 → 找 gap → 做改进 → 诚实评估	论文 (appE) 与 thesis/；强调“分/校准/测试”严谨性 (ch07)

F.3 可选的“往外走”课题（按主线挑 1–2 个）

- **JPEG 域隐写**：把 ch04–05 的思路搬到量化 DCT 系数（项目 yccstego 扩展即此方向），体会“空间域 vs 变换域”的差异；
- **深度学习隐写分析**：在小样本约束下，如何用可解释特征先验证信号，再谨慎引入深度模型（ch08.1 的教训）；
- **域适应 / 跨源鲁棒性**：SRM 同源有收益、跨源反而亏（ch08.8.1），可研究如何用域适应让特征跨相机/压缩源更稳；
- **对抗鲁棒性与误报控制**：研究“宽松 vs 严格”阈值与 Youden/低误报点的取舍（ch07），以及不可检测嵌入的物理上限；
- **可解释性**：143d 稳健 vs 53d 可解释（ch08.8.4），可研究哪些特征真正贡献了增益、如何做逐维解释。

回到项目看代码 |

把上述任一课题落到代码上：先跑通基线，再做小改动，用 GroupKFold 给出诚实对照。“改了 A、结果变好”不足以服人，要用训练/校准/测试三层结构 + 分组切分证明它真的变好。（ch07、ch08 反复强调这一点。）

F.4 与导读 0.3（12 周路线）的关系

- **想快速入门**：按 0.3，12 周跑一遍，做到附录 C 的打卡表；
- **想深入并出成果**：按本节，6–12 个月，把 0.3 的每周任务放慢、加深、关联，并在“深化期/实战期”加入 0.5 项目地图里那些只在研究时才真正需要的部分（SRM、多源、GPU、双版本、OOD 验证）。
- 两者的**共同底线**：不要略过“动手实验”，也不要略过“诚实评估”。这本手册最想教你的不是某个算法，而是“怎么判断一个实验结论靠不靠谱”。

想一想 |

你打算选哪条主线？打算用多长的周期？（12 周 vs 6–12 月，意味着你把时间花在“宽度”还是“深度”上。）把它写下来，作为你学习这本书的“个人目标”。